

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ – THÔNG TIN



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

THIẾT KẾ VÀ KIỂM THỬ DỰA TRÊN UVM CHO 10GBASE-R PCS

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

GVHD: PGS. TS. PHẠM NGỌC SƠN

SVTH: NGUYỄN TRUNG KIÊN

MSSV: 22161274

NGUYỄN CHÍ PHƯƠNG

MSSV: 22161307

TP. HỒ CHÍ MINH - 06/2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ – THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP

THIẾT KẾ VÀ KIỂM THỬ DỰA TRÊN
UVM CHO 10GBASE-R PCS

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

GVHD: PGS. TS. PHẠM NGỌC SƠN

SVTH: NGUYỄN TRUNG KIÊN

MSSV: 22161274

NGUYỄN CHÍ PHƯƠNG

MSSV: 22161307

TP. HỒ CHÍ MINH – 06/2026

THÔNG TIN KHÓA LUẬN TỐT NGHIỆP

1. Thông tin sinh viên

Họ và tên sinh viên: Nguyễn Trung Kiên

Email: kien.ngtr.24@gmail.com

Họ và tên sinh viên: Nguyễn Chí Phương

Email: phuongngchi@gmail.com

MSSV: 22161274

Điện thoại: 0971832965

MSSV: 22161307

Điện thoại: 0962507318

2. Thông tin đề tài

- Tên đề tài: Thiết kế và kiểm thử dựa trên UVM cho 10GBASE-R PCS
- Đơn vị quản lý: Bộ môn Kỹ Thuật Điện tử – Thông tin, Khoa Điện-Điện Tử, Trường Đại học Công Nghệ Kỹ Thuật TP.Hồ Chí Minh.
- Thời gian thực hiện: Từ ngày / / 20... đến ngày / / 20...
- Thời gian bảo vệ trước hội đồng: Ngày / / 20...

3. Lời cam đoan của sinh viên

Nhóm thực hiện cam đoan KLTN là công trình nghiên cứu của tôi, dưới sự hướng dẫn của Phó Giáo Sư Tiến Sĩ Phạm Ngọc Sơn. Kết quả công bố trong KLTN là trung thực và không sao chép từ bất kì công trình nào khác.

Tp. HCM, ngày ... tháng ... năm 20...

SV thực hiện đồ án

(Ký và ghi rõ họ tên)

.....

Giảng viên hướng dẫn xác nhận quyền báo cáo đã được chỉnh sửa theo đề nghị được ghi trong biên bản của Hội đồng đánh giá Khóa luận tốt nghiệp.

.....

Xác nhận của Bộ Môn

Tp. HCM, ngày ... tháng ... năm 20..

Giảng viên hướng dẫn

(Ký, ghi rõ họ tên và học hàm – học vị)

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: THIẾT KẾ VÀ KIỂM THỬ DỰA TRÊN UVM CHO 10GBASE-R PCS

Sinh viên: + Nguyễn Trung Kiên
+ Nguyễn Chí Phương

MSSV: 22161274
MSSV: 22161307

Hướng dẫn: PGS.TS. Phạm Ngọc Sơn

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

- Hợp lý.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

- Nghiên cứu dựa trên tài liệu lý thuyết, thực hiện mô phỏng và đánh giá.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá / kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ an toàn.

- Kết quả mô phỏng chứng minh chức năng.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

- Kết luận phù hợp với đặt vấn đề.

5. Hình thức trình bày và bố cục báo cáo:

Vấn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tổ, văn bản trau chuốt.

- Hợp lý.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

- Kỹ năng làm việc nhóm hiệu quả.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

- Có trích dẫn tài liệu phù hợp.

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

- Chưa phát hiện trùng lặp với các đề tài trong nước.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

- Bổ sung giải thích các kết quả mô phỏng.

- Cập nhật tài liệu tham khảo.

- Đọc và chỉnh sửa trình bày toàn bộ báo cáo.

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

- Nhóm sinh viên làm việc nghiêm túc.

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Báo cáo đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép bảo vệ khóa luận tốt nghiệp.

Tp. HCM, ngày 25 tháng 06 năm 2026

Người nhận xét

(Ký và ghi rõ họ tên)


PGS.TS. Phạm Ngọc Sơn

* Giảng viên hướng dẫn có thể ghi tiếp vào mặt sau

LỜI CẢM ƠN

Nhóm xin gửi lời cảm ơn đến TS. Phạm Ngọc Sơn, giảng viên hướng dẫn Đồ án tốt nghiệp của nhóm, vì những đánh giá và góp ý về quá trình thực hiện đề tài.

Nhóm xin chân thành cảm ơn quý thầy cô Khoa Điện – Điện Tử, Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh, đã tận tình giảng dạy, truyền đạt kiến thức và hỗ trợ nhóm trong suốt quá trình học tập tại trường. Sự định hướng cùng nền tảng kiến thức vững chắc mà quý thầy cô cung cấp đã tạo điều kiện thuận lợi để nhóm thực hiện đề tài này.

Nhóm cũng xin gửi lời cảm ơn chân thành đến Bộ môn Kỹ thuật Điện tử – Thông tin đã tạo điều kiện, hỗ trợ và đồng hành cùng nhóm trong quá trình thực hiện đồ án tốt nghiệp.

Do nhiều hạn chế về kiến thức và sự hiểu biết của nhóm, bài báo cáo có thể không tránh khỏi những sai sót. Nhóm kính mong nhận được những sự góp ý trên nhiều khía cạnh để cải thiện chất lượng của bài báo cáo này.

Xin chân thành cảm ơn.

Thành phố Hồ Chí Minh, 06/2026

Nhóm thực hiện Đồ án tốt nghiệp

Nguyễn Trung Kiên

Nguyễn Chí Phương

TÓM TẮT

Đề tài thực hiện thiết kế một lớp mã hóa vật lý phụ (PCS) theo nguyên lý của chuẩn Ethernet, gồm các khối thu, khối phát và khối quản lý. Thiết kế này được kiểm tra bằng một UVM testbench do nhóm tự thiết kế. Testbench có khả năng thực hiện truyền thông tin trên giao diện truyền thông độc lập 10 Gigabit (XGMII) và đánh giá tín hiệu nhận được trên phía nhận của PCS. UVM testbench thực hiện truyền tín hiệu ngẫu nhiên theo dải dữ liệu XGMII trên mô phỏng. Kết quả cho thấy thiết kế PCS truyền và nhận tín hiệu đúng với mong đợi trên XGMII.

MỤC LỤC

LỜI CẢM ƠN	i
TÓM TẮT	ii
MỤC LỤC	iii
DANH SÁCH BẢNG.....	v
DANH SÁCH HÌNH.....	vi
DANH MỤC TỪ VIẾT TẮT	viii
CHƯƠNG 1: TỔNG QUAN.....	1
1.1. Giới thiệu	1
1.2. Mục tiêu nghiên cứu	1
1.3. Giới hạn của đề tài.....	2
1.4. Nội dung nghiên cứu.....	2
1.5. Bố cục nội dung.....	3
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	4
2.1. Lớp mã hóa vật lý phụ (PCS)	4
2.1.1. XGMII	5
2.1.2. Mã hóa/giải mã 64B/66B.....	9
2.1.3. Xáo trộn và giải xáo trộn bit.....	12
2.1.4. Chuyển đổi độ rộng dữ liệu	14
2.1.5. Đồng bộ khối	15
2.1.6. Giám sát tỉ lệ lỗi bit và bit đồng bộ	16
2.1.7. Cơ chế truyền nhận dữ liệu.....	16
2.2. Phương pháp kiểm thử chung (UVM)	18
2.2.1. uvm_component	19
2.2.2. uvm_transaction	20
2.2.3. Các cơ chế trong UVM.....	21
2.2.4. Kiến trúc thông thường của UVM.....	23
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG	24
3.1. PCS.....	24

3.1.1. Sơ đồ khối tổng quát.....	24
3.1.2. Khối giao tiếp XGMII	31
3.1.3. Khối mã hóa và giải mã 64B/66B	33
3.1.4. Khối xáo trộn và giải xáo trộn.....	42
3.1.5. Khối chuyển độ rộng dữ liệu	44
3.1.6. Khối đồng bộ khối	47
3.1.7. Khối giám sát BER và Sync Header.....	53
3.1.8. Khối giám sát và quản lý trạng thái PCS.....	55
3.2. UVM Testbench	58
3.2.1. pcs_test, pcs_env, pcs_agent.....	58
3.2.2. pcs_driver	60
3.2.3. pcs_transaction, pcs_sequence, pcs_sequencer.....	61
3.2.4. pcs_monitor	62
3.2.5. pcs_scoreboard	62
3.2.6. pcs_stress_test và pcs_stress_sequence.....	64
CHƯƠNG 4: KẾT QUẢ MÔ PHỎNG.....	66
4.1. Mô phỏng chức năng	66
4.2. Mô phỏng dạng sóng	71
4.2.1. Quá trình truyền dữ liệu.....	71
4.2.2. Quá trình nhận dữ liệu	73
4.2.3. Kiểm tra dạng khung truyền nhận trên XGMII.....	78
4.3. Mô phỏng kịch bản kiểm tra chịu tải.....	80
CHƯƠNG 5: TỔNG KẾT	82
5.1. Kết quả.....	82
5.2. Hướng phát triển	83
TÀI LIỆU THAM KHẢO.....	84
PHỤ LỤC	85

DANH SÁCH BẢNG

Bảng 2.1: Các đường tín hiệu trên XGMII.....	6
Bảng 2.2: Các mã tín hiệu của đường truyền XGMII	6
Bảng 2.3: Tín hiệu preamble và sfd.....	8
Bảng 2.4: Các mã tín hiệu của đường nhận XGMII.....	8
Bảng 2.5: Các ký hiệu điều khiển và các mã điều khiển tương ứng	9
Bảng 3.1: Các chân tín hiệu của khối PCS_10GBASE_R.....	26
Bảng 4.1: Các dữ liệu ngẫu nhiên được gửi đi trên XGMII.....	67
Bảng 4.2: Các dữ liệu kiểm tra mã hóa ký tự Ordered Set (/O/) gửi đi XGMII	69
Bảng 4.3: Tín hiệu gửi dữ liệu 0x6a12_a8a7_faeb_028d_1a71 theo XGMII.....	79

DANH SÁCH HÌNH

Hình 2.1: Mối quan hệ giữa 10GBASE-R PCS với mô hình OSI [4]	4
Hình 2.2: Sơ đồ khối của 10GBASE-R PHY [4]	5
Hình 2.3: Các tín hiệu XGMII [4]	6
Hình 2.4: Một dải dữ liệu XGMII [4].....	7
Hình 2.5: Một khung truyền thông thường trên XGMII [4].....	7
Hình 2.6: Một khung nhận thông thường trên XGMII [4]	9
Hình 2.7: Định dạng khối 64B/66B [4]	11
Hình 2.8: Cấu trúc bộ scrambler trong PCS 10GBASE-R.....	12
Hình 2.9: Cấu trúc bộ descrambler trong PCS 10GBASE-R	13
Hình 2.10: Quá trình tạo khối 64B/66B và xáo trộn dữ liệu trong PCS 10GBASE-R	17
Hình 2.11: Các lớp cơ sở của UVM	19
Hình 2.12: Các lớp kế thừa của uvm_component	20
Hình 2.13: Các lớp kế thừa của uvm_transaction	21
Hình 2.14: Kiến trúc thông thường của một UVM Testbench.....	23
Hình 2.15: Kiến trúc thông thường của một UVM Agent.....	23
Hình 3.1: Sơ đồ khối tổng quát lớp mã hóa vật lý phụ 10GBASE R.....	24
Hình 3.2: Các tín hiệu đầu vào và ra của khối PCS_10GBASE_R.....	26
Hình 3.3: Mô hình khối nhận dữ liệu XGMII	31
Hình 3.4: Mô hình truyền dữ liệu XGMII.....	32
Hình 3.5: Mô hình khối mã hóa Encoder 64B/66B.....	33
Hình 3.6: Sơ đồ trạng thái của khối mã hóa Encoder 64B/66B	35
Hình 3.7: Mô hình khối giải mã Decoder 64B/66B	37
Hình 3.8: Sơ đồ trạng thái khối giải mã Decoder 64B/66B	40
Hình 3.9: Mô hình khối xáo trộn dữ liệu Scrambler	42
Hình 3.10: Mô hình khối giải xáo trộn dữ liệu Desrambler	43
Hình 3.11: Mô hình khối chuyển độ rộng dữ liệu Gearbox	44
Hình 3.12: Mô hình khối đồng bộ khối	47
Hình 3.13: Sơ đồ trạng thái khối đồng bộ khối 64B/66B.....	50
Hình 3.14: Mô hình khối giám sát BER và Sync Header.....	53
Hình 3.15: Sơ đồ trạng thái của khối BER Monitor.....	54

Hình 3.16: Khối giám sát và quản lý trạng thái.....	56
Hình 3.17: Cấu trúc của top module.....	58
Hình 3.18: Cấu trúc của pcs_test, pcs_env và pcs_agent.....	59
Hình 3.19: Cấu trúc của pcs_driver.....	60
Hình 3.20: Lưu đồ hàm điều khiển tín hiệu trên driver.....	61
Hình 3.21: Lưu đồ hàm lấy mẫu tín hiệu trên monitor.....	62
Hình 3.22: Lưu đồ hàm write_tx	63
Hình 3.23: Lưu đồ hàm write_rx	64
Hình 4.1: Thông báo của một hoạt động ghi lại được trên phía truyền	66
Hình 4.2: Thông báo của một hoạt động ghi lại được trên phía nhận	66
Hình 4.3: Thông báo của một hoạt động phía nhận trùng khớp với phía truyền	66
Hình 4.4: Thông báo lỗi của một hoạt động phía nhận không trùng khớp với phía truyền	66
Hình 4.5: Thống kê các số lượng thông báo trên UVM.....	67
Hình 4.6: Kết quả mô phỏng thực hiện gửi 30 lần các dữ liệu ngẫu nhiên và mã hóa Ordered Set.....	71
Hình 4.7: Dạng sóng tín hiệu trong quá trình truyền dữ liệu khởi tạo ban đầu ở phía phát	72
Hình 4.8: Dạng sóng tín hiệu sau quá trình xáo trộn và và chuyển đổi độ rộng dữ liệu phía phát	73
Hình 4.9: Dạng sóng tín hiệu trong quá trình truyền dữ liệu khởi tạo ban đầu ở phía thu	74
Hình 4.10: Dạng sóng tín hiệu của phía thu khi bắt đầu nhận được dữ liệu từ phía phát.	74
Hình 4.11: Dạng sóng xử lý dữ liệu ở phía thu khi đã khóa được khung	76
Hình 4.12: Dạng sóng mô phỏng quá trình truyền frame đầu tiên.....	77
Hình 4.13: Dạng sóng mô phỏng quá trình nhận frame đầu tiên từ phía phát	78
Hình 4.14: Kết quả mô phỏng dạng sóng của một hoạt động truyền dữ liệu trên phía truyền và phía nhận.....	79
Hình 4.15: Một khung truyền dẫn thông thường trên XGMII [4].....	79
Hình 4.16: Kết quả log UVM của kịch bản kiểm tra chịu tải.....	81

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Định nghĩa
OSI	Open Systems Interconnection
PCS	Physical Coding Sublayer
PHY	Physical Layer Device
RS	Reconciliation Sublayer
UVM	Universal Verification Methodology
XGMII	10 Gigabit Media Independent Interface
10GBASE-R	10 Gigabit Ethernet BASE-R
ASIC	Application-Specific Integrated Circuit
CDC	Clock Domain Crossing
CDR	Clock and Data Recovery
DDR	Double Data Rate
DUT	Design Under Test
EEE	Energy-Efficient Ethernet
EFD	End of Frame Delimiter
FIFO	First In, First Out
FSM	Finite State Machine
HDL	Hardware Description Language
IEEE	Institute of Electrical and Electronics Engineers
LPI	Low Power Idle
LSB	Least Significant Bit
MAC	Media Access Control
MDIO	Management Data Input/Output
OOP	Object-Oriented Programming
PMA	Physical Medium Attachment
RTL	Register Transfer Level
RX	Receive

SDR	Single Data Rate
SFD	Start of Frame Delimiter
SoC	System on Chip
TLM	Transaction-Level Modeling
TX	Transmit
WIS	WAN Interface Sublayer
XAUI	10 Gigabit Attachment Unit Interface
XOR	Exclusive OR
IPG	Inter-Packet Gap

CHƯƠNG 1: TỔNG QUAN

1.1. Giới thiệu

Qua thời gian, sự phát triển của Ethernet qua các hệ thống mạng có thể được cảm nhận rõ rệt. Nhờ chi phí áp dụng thấp, độ ổn định cao và đơn giản trong việc thiết lập và bảo trì, sự phổ biến của Ethernet đã đạt đến điểm mà gần như mọi lưu thông trên Internet đều khởi đầu và kết thúc bằng một kết nối Ethernet [1]. Từ sự thích ứng của Ethernet với tốc độ cao, tiêu chuẩn được cải tiến qua nhiều năm với chuẩn mới nhất là IEEE Std 802.3-2022, ra đời năm 2022. Do khả năng hỗ trợ tốc độ cao, lên tới hàng chục hoặc hàng trăm gbyte trên một giây, Ethernet dần trở thành một lựa chọn đáng tin cậy cho các mạch tích hợp để giao tiếp giữa các chip [2], [3].

Lớp mã hóa vật lý phụ (Physical Coding Sublayer, PCS) là một phần trong chuẩn 10 Gigabit Ethernet. Lớp này được dùng trong các thiết bị lớp vật lý (Physical Layer device, PHY) nhóm 10GBASE-R. 10GBASE-R được dựa trên mã 64B/66B. Các PHY là một phần trong lớp vật lý trên mô hình OSI. [4]

Các phương pháp kiểm thử vi mạch cũng được cải tiến qua nhiều năm. Phương pháp kiểm thử chung (Universal Verification Methodology, UVM) trở thành một lựa chọn phổ biến. UVM giúp giảm chi phí kiểm thử và tăng chất lượng thiết kế trong công nghiệp [5].

Việc sử dụng UVM cho bài toán kiểm thử PCS là một hướng tiếp cận thực tế trong ngành thiết kế vi mạch và có tiềm năng cải thiện các hệ thống đang sử dụng PCS nói riêng và Ethernet nói chung. Từ đây nhóm quyết định chọn “Thiết kế và kiểm thử dựa trên UVM cho 10GBASE-R PCS” là đề tài của nhóm cho Đồ án tốt nghiệp.

1.2. Mục tiêu nghiên cứu

Thiết kế module PCS đơn giản có khả năng truyền và nhận tín hiệu theo dải dữ liệu của giao diện truyền thông độc lập 10 Gigabit (10 Gigabit media independent interface, XGMII).

Xây dựng môi trường kiểm thử dựa trên UVM có khả năng kiểm tra hoạt động truyền nhận dữ liệu theo dải dữ liệu XGMII của module PCS.

1.3. Giới hạn của đề tài

Thiết kế chỉ dừng lại ở mức mô phỏng chức năng: Toàn bộ quá trình kiểm chứng được thực hiện trên môi trường mô phỏng phần mềm, chưa tiến hành tổng hợp mạch Synthesis và triển khai trên nền tảng FPGA thực tế. Thiết kế được xây dựng bằng ngôn ngữ SystemVerilog. Các tập tin thiết kế và kiểm chứng trong đề tài được biên dịch, mô phỏng và đánh giá trên phần mềm ModelSim.

Đề tài không đi sâu vào các chức năng nâng cao của UVM. Một tiêu chuẩn quan trọng trong việc kiểm thử vi mạch, được gọi là độ bao phủ (coverage), không được kiểm tra trong đề tài này, do thiếu điều kiện tiếp cận các công cụ đọc các tệp kết quả của độ bao phủ.

1.4. Nội dung nghiên cứu

Nội dung nghiên cứu của đề tài bao gồm:

- Tìm hiểu về PCS và hoạt động của các khối bên trong PCS.
- Tìm hiểu lý thuyết về UVM, các thành phần của UVM và hoạt động bên trong một UVM testbench.
- Nghiên cứu và thiết kế mô tả phần cứng cho các khối của PCS bằng SystemVerilog HDL có khả năng thực hiện các chức năng của các khối trong PCS.
- Xây dựng một UVM testbench có khả năng mô phỏng thực hiện các hoạt động truyền tín hiệu theo dải dữ liệu XGMII lên thiết kế được kiểm tra (design under test, DUT).

- Mô phỏng từ DUT và testbench đã thiết kế.
- Đánh giá, kiểm tra hoạt động của DUT qua mô phỏng.

1.5. Bố cục nội dung

Báo cáo đồ án tốt nghiệp của nhóm bao gồm 5 chương:

Chương 1: Tổng quan – Giới thiệu tổng quan về đề tài, bao gồm lý do chọn đề tài, mục tiêu và những giới hạn trong phạm vi của đề tài.

Chương 2: Cơ sở lý thuyết – Trình bày các nền tảng lý thuyết về lớp mã hóa vật lý phụ (Physical Coding Sublayer, PCS), phương pháp kiểm thử chung (Universal Verification Methodology, UVM) được sử dụng trong đề tài.

Chương 3: Thiết kế hệ thống – Trình bày chi tiết về quy trình thực hiện, từ việc thiết kế module PCS và các khối bên trong, thiết kế sơ đồ khối, đến việc xây dựng các thành phần dựa trên UVM.

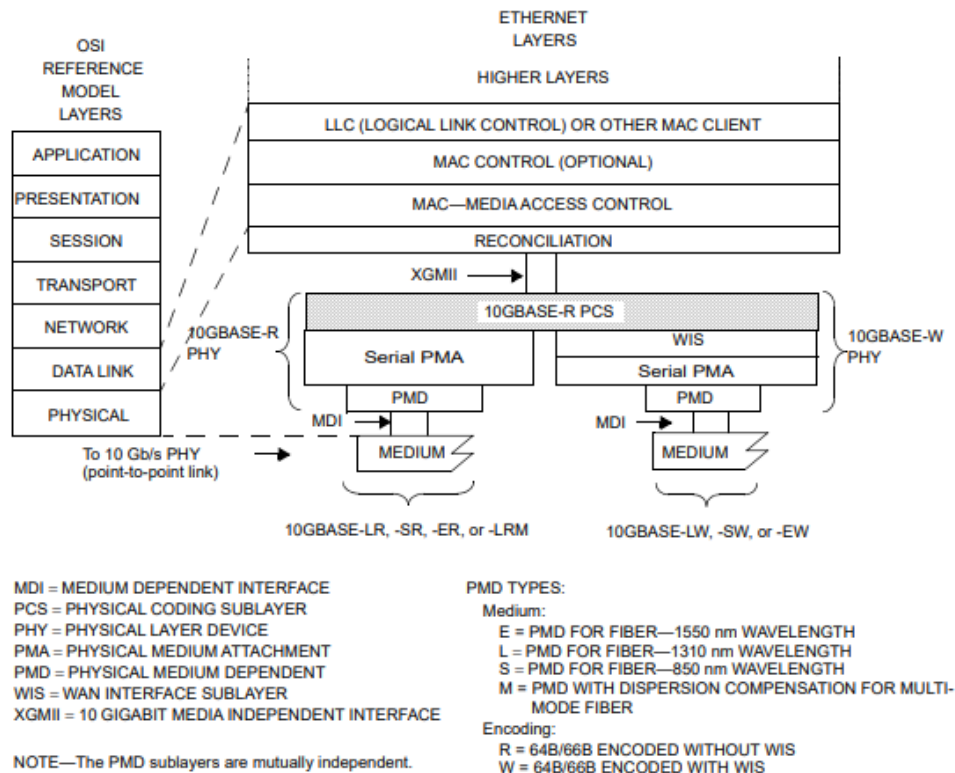
Chương 4: Kết quả – Trình bày các kết quả đạt được sau khi thiết kế các module và sau khi kiểm tra hoạt động dựa trên UVM.

Chương 5: Kết luận – Tổng kết lại những gì đã đạt được so với mục tiêu ban đầu và đề xuất các hướng phát triển, cải tiến cho đề tài trong tương lai..

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Lớp mã hóa vật lý phụ (PCS)

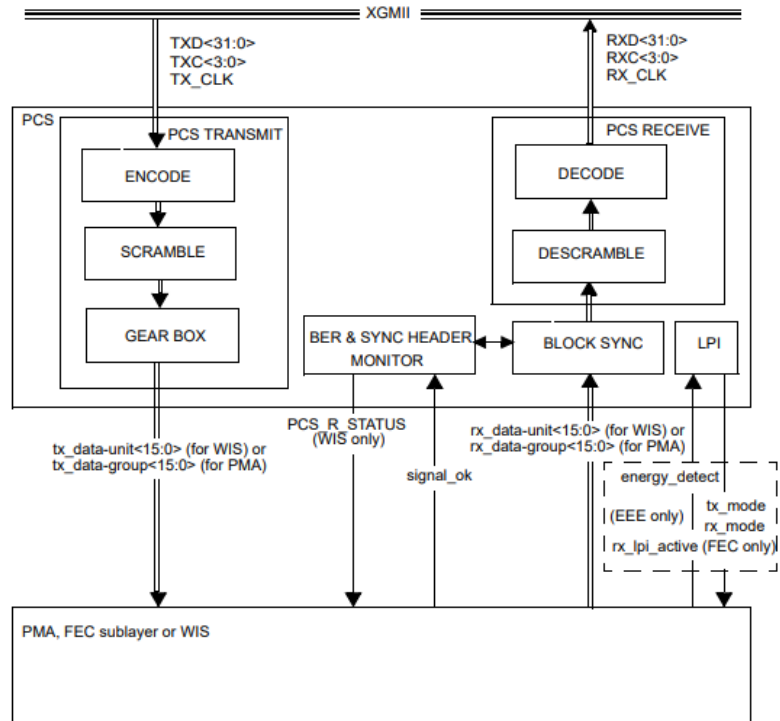
Mối quan hệ giữa 10GBASE-R PCS với mô hình OSI được biểu diễn trong hình 2.1 [4].



Hình 2.1: Mối quan hệ giữa 10GBASE-R PCS với mô hình OSI [4]

Từ hình trên, ta có thể thấy 10GBASE-R PCS là một phần của PHY. PCS giao tiếp với RS (Reconciliation Sublayer) qua giao diện truyền thông độc lập 10 Gigabit (10 Gigabit media independent interface, XGMII). PHY thuộc lớp thứ bảy, tức lớp vật lý trên mô hình OSI. 10GBASE-R khác với 10GBASE-W ở việc mã hóa không có lớp mạng diện rộng (Wide Area Network Interface Sublayer, WIS).

Sơ đồ khối của 10GBASE-R PHY được biểu diễn trong hình [4].

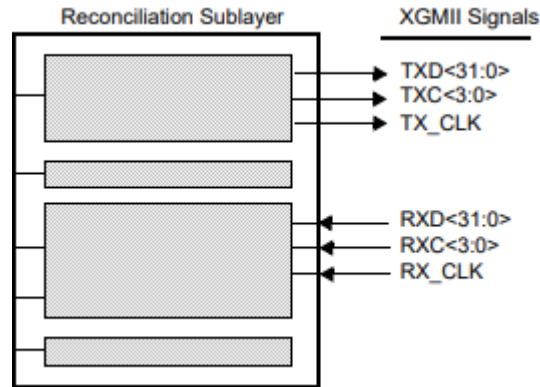


Hình 2.2: Sơ đồ khối của 10GBASE-R PHY [4]

Hình 2.2 cho thấy PCS giao tiếp với giao diện XGMII và lớp PMA. Bên trong PCS gồm các khối mã hóa/giải mã, khối xáo trộn/giải xáo trộn, khối Gearbox, khối đồng bộ khối và khối giám sát tỉ lệ lỗi bit (bit error rate, BER) và các bit đồng bộ (sync header).

2.1.1. XGMII

Các tín hiệu XGMII trên RS được biểu diễn trên hình [4].



Hình 2.3: Các tín hiệu XGMII [4]

Hình trên cho thấy XGMII gồm các đường truyền và đường nhận độc lập với nhau. Mỗi hướng gồm có 32 bit tín hiệu dữ liệu (TXD và RXD), 4 bit tín hiệu điều khiển (TXC và RXC) và một tín hiệu xung (TX_CLK và RX_CLK).

Các tín hiệu dữ liệu và điều khiển được chia thành bốn đường dữ liệu như bảng.

Bảng 2.1: Các đường tín hiệu trên XGMII

TXD, RXD	TXC, RXC	Đường
<7:0>	<0>	0
<15:8>	<1>	1
<23:16>	<2>	2
<31:24>	<3>	3

Các đường dữ liệu này sử dụng chung một tín hiệu xung, TX_CLK đối với đường truyền và RX_CLK đối với đường nhận.

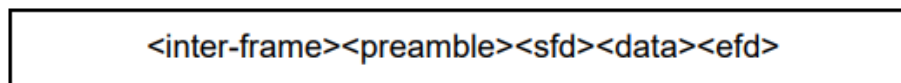
Các mã tín hiệu của TXC và TXD được mô tả theo bảng:

Bảng 2.2: Các mã tín hiệu của đường truyền XGMII

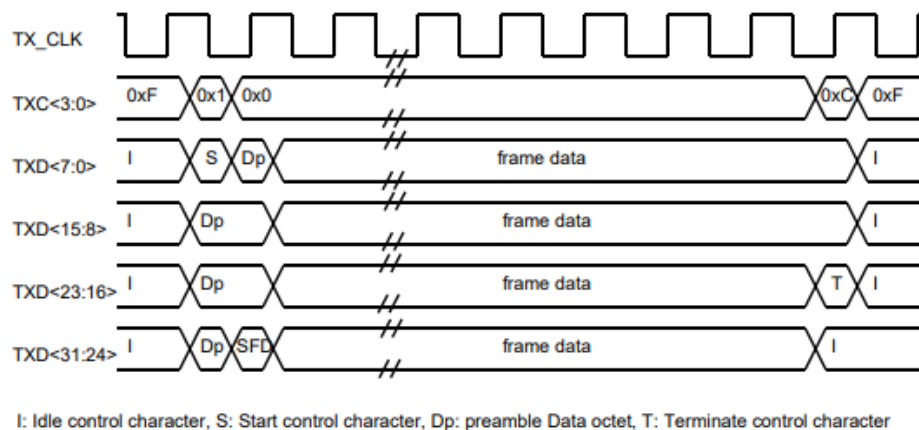
TXC	TXD	Mô tả
-----	-----	-------

0	00 đến FF	Truyền dữ liệu thông thường
1	06	Chỉ hợp lệ khi xuất hiện trên cả bốn đường. Mã yêu cầu LPI (Low Power Idle)
1	07	Idle
1	9C	Sequence (Chỉ hợp lệ trên đường 0)
1	FB	Start (Chỉ hợp lệ trên đường 0)
1	FD	Terminate
1	FE	Truyền lỗi (Transmit error propagation)
1	Khác	Dự trữ

Một dải dữ liệu XGMII và ví dụ về khung truyền thông thường được thể hiện trong hình 2.4 và hình 2.5.



Hình 2.4: Một dải dữ liệu XGMII [4]



Hình 2.5: Một khung truyền thông thường trên XGMII [4]

Trong dải dữ liệu XGMII, một inter-frame là khoảng thời gian không có hoạt động, tức bắt đầu từ thời điểm có ký hiệu điều khiển Terminate, kéo dài bởi các ký

hiệu điều khiển Idle, và kết thúc tại ký hiệu điều khiển Idle ngay trước ký hiệu điều khiển Start.

Trong XGMII, preamble và sfd (start of frame delimiter) được quy định như trong bảng 2.3:

Bảng 2.3: Tín hiệu preamble và sfd

Đường 0	Đường 1	Đường 2	Đường 3
Start	10101010	10101010	10101010
10101010	10101010	10101010	10101011

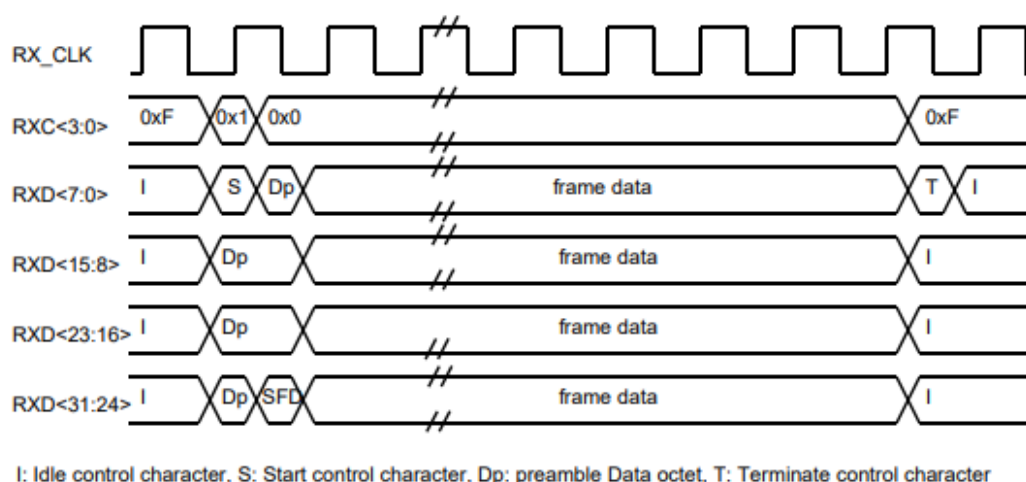
Sau sfd, data là các byte dữ liệu được truyền đi và được kết thúc bằng efd. Một ký hiệu Terminate trên một đường đồng nghĩa với efd đối với dải truyền dữ liệu. [4]

Tương tự, ta có các mã tín hiệu của RXC và RXD cùng ví dụ về khung nhận cơ bản trong bảng 2.4 và hình 2.6

Bảng 2.4: Các mã tín hiệu của đường nhận XGMII

RXC	RXD	Mô tả
0	00 đến FF	Nhận dữ liệu thông thường
1	06	Chỉ hợp lệ khi xuất hiện trên cả bốn đường. Cho thấy LPI được kích hoạt
1	07	Idle
1	9C	Sequence (Chỉ hợp lệ trên đường 0)
1	FB	Start (Chỉ hợp lệ trên đường 0)
1	FD	Terminate
1	FE	Nhận lỗi

		(Receive error)
1	Khác	Dự trữ



Hình 2.6: Một khung nhận thông thường trên XGMII [4]

Các tín hiệu đường nhận hoạt động giống với các tín hiệu đường truyền.

2.1.2. Mã hóa/giải mã 64B/66B

PCS mã hóa 8 byte dữ liệu XGMII thành khối 66 bit và giải mã khối 66 bit này thành 8 byte dữ liệu. Đây được gọi là mã hóa 64B/66B.

Các ký hiệu điều khiển được mã hóa thành mã điều khiển 7 bit hoặc mã O 4 bit. Các ký hiệu điều khiển tương ứng với các mã điều khiển 10GBASE-R và mã điều khiển XGMII được biểu diễn trong bảng 2.5.

Bảng 2.5: Các ký hiệu điều khiển và các mã điều khiển tương ứng

Ký hiệu	Chú thích	Mã điều khiển XGMII	Mã điều khiển 10GBASE-R	Mã O 10GBASE-R
Idle	/I/	0x07	0x00	
LPI	/LI/	0x06	0x06	

Start	/S/	0xFB		
Terminate	/T/	0xFD		
Error	/E/	0xFE	0x1E	
Sequence ordered set	/O/	0x9C		0x0
Reserved0		0x1C	0x2D	
Reserved1		0x3C	0x33	
Reserved2		0x7C	0x4B	
Reserved3		0xBC	0x55	
Reserved4		0xDC	0x66	
Reserved5		0xF7	0x78	
Signal ordered set		0x5C		0xF

Các mã Reserved được sử dụng bởi 10 Gigabit Attachment Unit Interface (XAUI), một tùy chọn mở rộng của XGMII. Các mã điều khiển khác mã /O/, /S/ và /T/ được chú thích chung là mã /C/. Byte dữ liệu được chú thích là /D/.

Định dạng khối 64B/66B được biểu diễn trong hình 2.7.

Input Data	S y n c	Block Payload									
Bit Position:	0 1 2	65									
Data Block Format:											
D ₀ D ₁ D ₂ D ₃ /D ₄ D ₅ D ₆ D ₇	01	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇		
Control Block Formats:		Block Type Field									
C ₀ C ₁ C ₂ C ₃ /C ₄ C ₅ C ₆ C ₇	10	0x1E	C ₀	C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇	
C ₀ C ₁ C ₂ C ₃ /O ₄ D ₅ D ₆ D ₇	10	0x2D	C ₀	C ₁	C ₂	C ₃	O ₄	D ₅	D ₆	D ₇	
C ₀ C ₁ C ₂ C ₃ /S ₄ D ₅ D ₆ D ₇	10	0x33	C ₀	C ₁	C ₂	C ₃			D ₅	D ₆	D ₇
O ₀ D ₁ D ₂ D ₃ /S ₄ D ₅ D ₆ D ₇	10	0x66	D ₁	D ₂	D ₃	O ₀			D ₅	D ₆	D ₇
O ₀ D ₁ D ₂ D ₃ /O ₄ D ₅ D ₆ D ₇	10	0x55	D ₁	D ₂	D ₃	O ₀	O ₄	D ₅	D ₆	D ₇	
S ₀ D ₁ D ₂ D ₃ /D ₄ D ₅ D ₆ D ₇	10	0x78	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇		
O ₀ D ₁ D ₂ D ₃ /C ₄ C ₅ C ₆ C ₇	10	0x4B	D ₁	D ₂	D ₃	O ₀	C ₄	C ₅	C ₆	C ₇	
T ₀ C ₁ C ₂ C ₃ /C ₄ C ₅ C ₆ C ₇	10	0x87				C ₁	C ₂	C ₃	C ₄	C ₅	C ₆
D ₀ T ₁ C ₂ C ₃ /C ₄ C ₅ C ₆ C ₇	10	0x99	D ₀			C ₂	C ₃	C ₄	C ₅	C ₆	C ₇
D ₀ D ₁ T ₂ C ₃ /C ₄ C ₅ C ₆ C ₇	10	0xAA	D ₀	D ₁			C ₃	C ₄	C ₅	C ₆	C ₇
D ₀ D ₁ D ₂ T ₃ /C ₄ C ₅ C ₆ C ₇	10	0xB4	D ₀	D ₁	D ₂			C ₄	C ₅	C ₆	C ₇
D ₀ D ₁ D ₂ D ₃ /T ₄ C ₅ C ₆ C ₇	10	0xCC	D ₀	D ₁	D ₂	D ₃			C ₅	C ₆	C ₇
D ₀ D ₁ D ₂ D ₃ /D ₄ T ₅ C ₆ C ₇	10	0xD2	D ₀	D ₁	D ₂	D ₃	D ₄			C ₆	C ₇
D ₀ D ₁ D ₂ D ₃ /D ₄ D ₅ T ₆ C ₇	10	0xE1	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅			C ₇
D ₀ D ₁ D ₂ D ₃ /D ₄ D ₅ D ₆ T ₇	10	0xFF	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆		

Hình 2.7: Định dạng khối 64B/66B [4]

Một khối 66 bit có thể là khối dữ liệu hoặc khối điều khiển. 2 bit đồng bộ đầu (sync header) là 01 (0 là LSB) đối với khối dữ liệu và 10 (1 là LSB) đối với khối điều khiển.

Khối dữ liệu gồm có 8 ký hiệu dữ liệu. Khối điều khiển bắt đầu với 8 bit vùng kiểu khối (block type field) cho biết định dạng của phần còn lại trong khối. Các ký hiệu không có mã điều khiển 10GBASE-R (/S/, /T/ và /O/) được mã hóa theo vùng kiểu khối này.

Một khối không hợp lệ khi các trường hợp này xảy ra [4]:

- Vùng đồng bộ có giá trị 00 hoặc 11.
- Vùng kiểu khối chứa một giá trị dự trữ.

- Bất kỳ ký hiệu điều khiển nào chứa giá trị không nằm trong bảng mã điều khiển.
- Bất kỳ mã O nào chứa giá trị không nằm trong bảng mã điều khiển, hoặc mã O 0xF được nhận trên Ethernet PHY.
- Nhóm 8 ký hiệu XGMII không có định dạng tương ứng được quy định trong bảng định dạng khối.

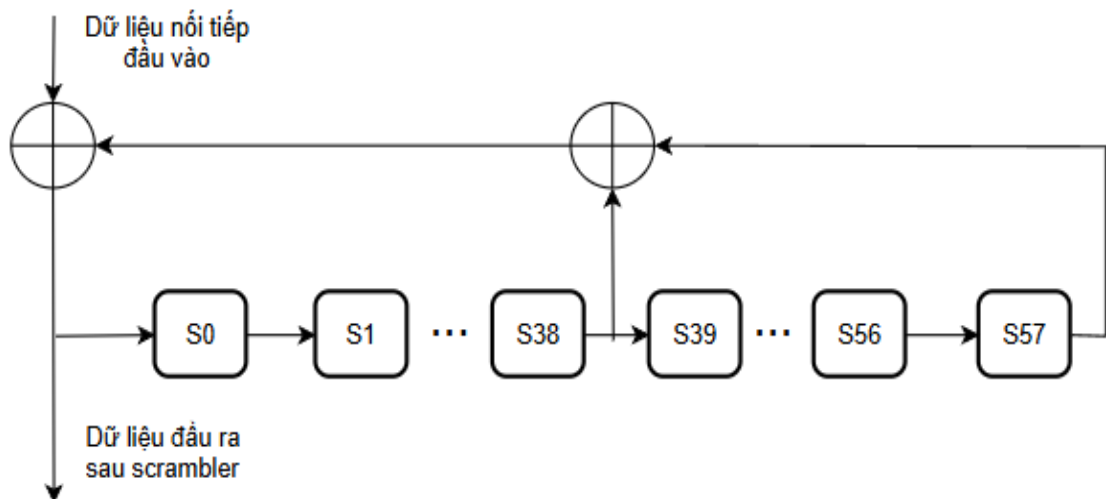
Với trường hợp LPI, nếu PHY không hỗ trợ Energy-Efficient Ethernet (EEE), tín hiệu LPI được coi là không hợp lệ nếu nhận được

2.1.3. Xáo trộn và giải xáo trộn bit

2.1.3.1. Khối xáo trộn

Khối xáo trộn (Scrambler) trong PCS 10GBASE-R là loại self-synchronizing scrambler. Theo IEEE 802.3 Clause 49, scrambler chỉ xử lý phần payload của block, chạy liên tục trên tất cả các bit payload và không yêu cầu giá trị khởi tạo cố định.

Theo chuẩn IEEE 802.3-2022, payload của block 64B/66B được scramble bằng self-synchronizing scrambler với đa thức: $G(x) = 1 + x^{39} + x^{58}$ [4].



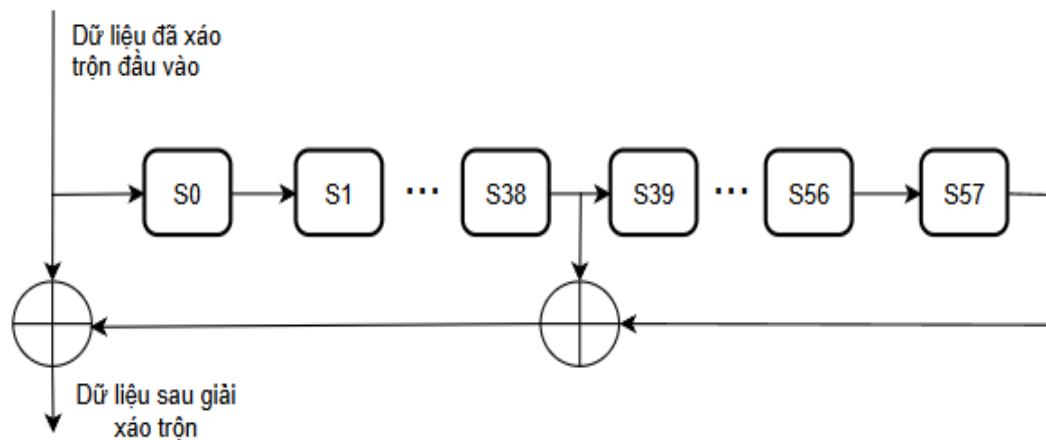
Hình 2.8: Cấu trúc bộ scrambler trong PCS 10GBASE-R

Hình 2.8 trình bày cấu trúc bộ scrambler trong PCS 10GBASE-R. Bộ scrambler sử dụng một thanh ghi dịch gồm 58 phần tử nhớ, ký hiệu từ S0 đến S57. Dữ liệu nối tiếp đầu vào được XOR với các bit hồi tiếp tại hai vị trí S38 và S57 để tạo ra dữ liệu đầu ra đã được xáo trộn. Cấu trúc này tương ứng với đa thức.

Dữ liệu đầu ra được tạo bằng cách kết hợp dữ liệu đầu vào với các bit hồi tiếp tại các vị trí trễ tương ứng. Mục đích của scrambler là làm ngẫu nhiên hóa chuỗi bit truyền đi, giảm các chuỗi bit lặp dài, tăng mật độ chuyển mức tín hiệu và hỗ trợ quá trình khôi phục clock ở phía thu.

2.1.3.2. Khối giải xáo trộn

Khối giải xáo trộn (Descrambler) có nhiệm vụ đảo ngược data của scrambler. Bộ descrambler xử lý payload để khôi phục dữ liệu bằng cách sử dụng cùng đa thức với scrambler. Đa thức được sử dụng trong 10GBASE-R PCS tương tự như scrambler: $G(x) = 1 + x^{39} + x^{58}$ [4].



Hình 2.9: Cấu trúc bộ descrambler trong PCS 10GBASE-R

Hình 2.9 trình bày cấu trúc bộ descrambler trong PCS 10GBASE-R. Bộ descrambler sử dụng một thanh ghi dịch gồm 58 phần tử nhớ, ký hiệu từ S0 đến S57. Dữ liệu nối tiếp đầu vào của descrambler là chuỗi bit đã được xáo trộn ở phía phát.

Tại phía thu, bit đầu vào hiện tại được XOR với các bit đã nhận trước đó tại hai vị trí trễ tương ứng là S38 và S57 để tạo ra dữ liệu nối tiếp đầu ra sau khi giải xáo trộn. Cấu trúc này tương ứng với đa thức.

Do scrambler ở phía phát đã làm ngẫu nhiên hóa chuỗi bit truyền đi, descrambler ở phía thu sẽ thực hiện quá trình ngược lại nhằm loại bỏ tác động xáo trộn. Nhờ đó, dữ liệu sau khi đi qua descrambler có thể được giải mã về dạng dữ liệu ban đầu trên giao diện XGMII

2.1.4. Chuyển đổi độ rộng dữ liệu

Sau khi dữ liệu được mã hóa 64B/66B và đi qua khối scrambler, khối chuyển đổi độ rộng dữ liệu (gearbox) có nhiệm vụ chuyển đổi độ rộng dữ liệu từ block 66 bit sang các đơn vị dữ liệu 16 bit để truyền xuống giao diện PMA hoặc WIS [4]. Mục đích của Gearbox là thực hiện chức năng chuyển đổi độ rộng dữ liệu giữa khối 64B/66B PCS và giao diện truyền dữ liệu phía dưới.

Trong đường truyền phát gearbox được đặt sau khối Scrambler và trước giao diện PMA/WIS. Dữ liệu đầu vào của gearbox là các block 66 bit, gồm 2 bit sync header và 64 bit payload đã được scramble. Do giao diện PMA hoặc WIS sử dụng đường dữ liệu rộng 16 bit, gearbox sẽ đóng gói liên tục chuỗi bit 66 bit thành các nhóm dữ liệu 16 bit, gọi là transmit data-units, trước khi gửi xuống lớp PMA hoặc WIS [4].

Quá trình đóng gói của gearbox được thực hiện theo thứ tự bit của block. Các bit trong block 64B/66B được đưa tuần tự vào các nhóm 16 bit. Vì kích thước block là 66 bit không chia hết cho 16, nên ranh giới block 66 bit không luôn trùng với ranh giới của nhóm dữ liệu 16 bit. Do đó, một nhóm dữ liệu 16 bit có thể chứa phần cuối của một block và phần đầu của block kế tiếp. Khi đó, các bit thuộc block xuất hiện trước sẽ được đặt ở các vị trí bit có số thứ tự thấp hơn trong nhóm dữ liệu 16 bit, còn các bit của block kế tiếp được đặt tiếp theo sau [4].

Nhiệm vụ của khối là sắp xếp lại luồng bit theo đúng độ rộng giao diện truyền bên dưới, giúp dữ liệu sau mã hóa 64B/66B có thể truyền qua giao diện PMA/WIS 16 bit một cách liên tục các block 66 bit sau khi được tạo ra trong PCS sẽ được gearbox đóng gói thành các đơn vị dữ liệu 16 bit và gửi đến PMA hoặc WIS thông qua các primitive tương ứng [6].

2.1.5. Đồng bộ khối

Trong PCS 10GBASE-R, khối Block Sync nằm ở phía thu, sau giao diện PMA/WIS và trước khối Descrambler. Khối này có nhiệm vụ xác định đúng ranh giới của các block 64B/66B trong luồng bit nhận được. Dữ liệu từ PMA hoặc WIS được đưa vào PCS dưới dạng các nhóm 16 bit, sau đó Block Sync ghép các nhóm này thành luồng bit liên tục và tìm vị trí bắt đầu của từng block 66 bit dựa trên synchronization header [4].

Khối thực hiện kiểm tra các bit sync header để xác định ranh giới block 64B/66B. Trong mã hóa 64B/66B, mỗi block gồm 66 bit, trong đó 2 bit đầu là synchronization header và 64 bit còn lại là payload. Sync header có giá trị 01 đối với data block và 10 đối với control block. Hai giá trị 00 và 11 không hợp lệ và được xem là lỗi khi xuất hiện ở vị trí sync header [6], [4].

Nguyên lý hoạt động của Block Sync dựa trên đặc điểm sync header luôn xuất hiện ở đầu mỗi block 66 bit. Khi chưa đồng bộ, khối Block Sync sẽ kiểm tra các vị trí bit khác nhau trong luồng dữ liệu nhận được để tìm vị trí có mẫu sync header hợp lệ lặp lại theo chu kỳ 66 bit. Khi phát hiện đủ số lượng sync header hợp lệ theo quy định, khối Block Sync xác nhận trạng thái khóa đồng bộ block lock. Sau khi đạt block lock, các block 66 bit sẽ được tách ra đúng ranh giới và đưa đến khối Descrambler để tiếp tục xử lý [4].

Trong trường hợp phát hiện nhiều sync header không hợp lệ, khối Block Sync sẽ mất trạng thái đồng bộ và thực hiện thao tác slip để kiểm tra vị trí đồng bộ mới. Quá trình kiểm tra sync header được thực hiện theo từng cửa sổ 64 block; nếu số lượng

sync header không hợp lệ vượt ngưỡng cho phép, hệ thống sẽ bỏ vị trí đồng bộ hiện tại và chuyển sang kiểm tra vị trí kế tiếp [4]. Cơ chế này giúp PCS duy trì khả năng căn chỉnh block chính xác ngay cả khi luồng dữ liệu nhận được bị lệch bit hoặc xuất hiện lỗi trên đường truyền.

2.1.6. Giám sát tỉ lệ lỗi bit và bit đồng bộ

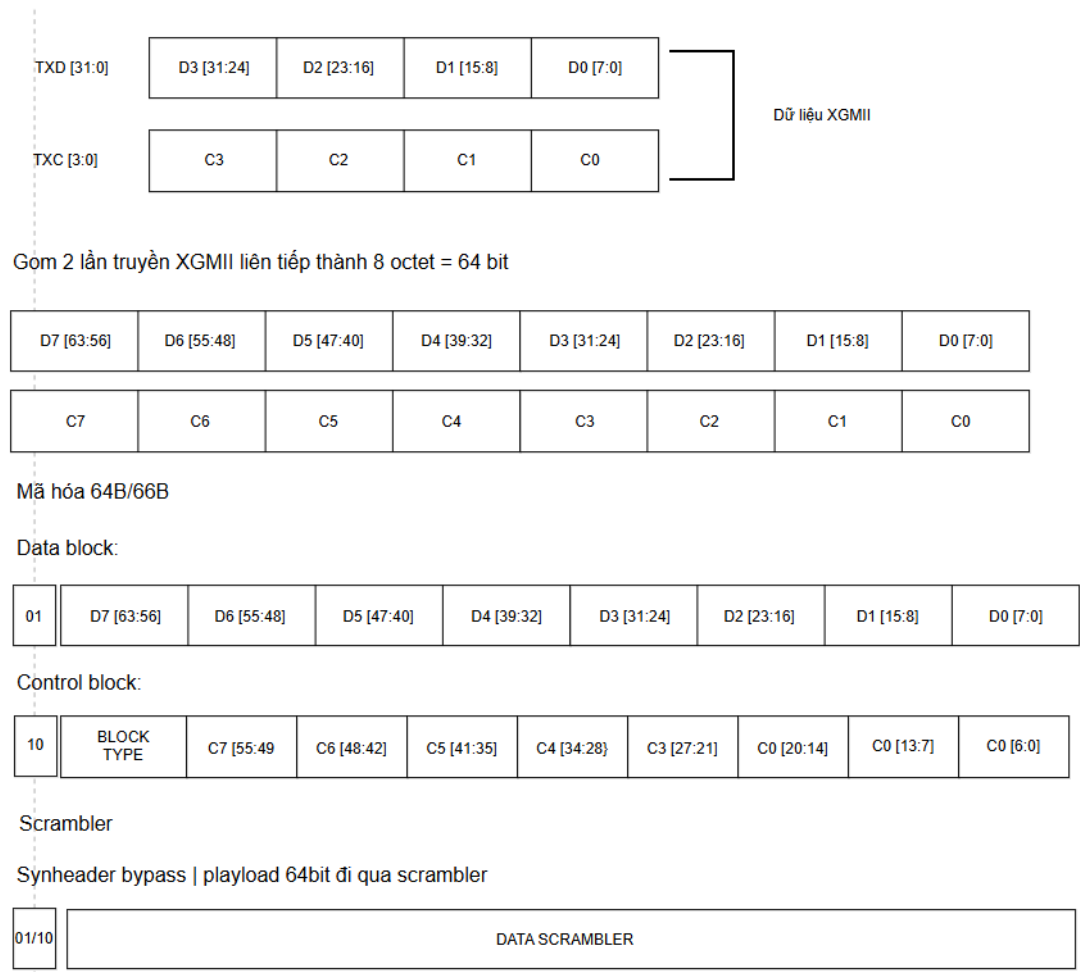
Khối giám sát tỉ lệ lỗi bit và bit đồng bộ (BER & Sync Header Monitor) được đặt ở phía thu và có nhiệm vụ giám sát chất lượng tín hiệu nhận được thông qua các sync header của block 64B/66B [4]. Sau khi khối Block Sync đạt được trạng thái đồng bộ block, khối này tiếp tục kiểm tra các sync header để phát hiện tình trạng lỗi bit cao trên đường truyền.

Nguyên lý hoạt động của khối này dựa trên việc đếm số lượng sync header không hợp lệ trong một khoảng thời gian xác định. Bộ đếm `ber_cnt` được sử dụng để đếm số sync header không hợp lệ trong chu kỳ 125 us. Khi số lượng sync header lỗi vượt ngưỡng quy định, tín hiệu `hi_ber` sẽ được kích hoạt để báo hiệu đường truyền đang có tỷ lệ lỗi bit cao [4].

Khối này có nhiệm vụ là giám sát lỗi dựa trên sync header, từ đó đánh giá chất lượng đường truyền và phát hiện trạng thái lỗi nghiêm trọng. PCS 10GBASE-R có thể bảo đảm rằng dữ liệu chỉ được tiếp tục xử lý khi đường truyền đã đồng bộ và không xuất hiện tỷ lệ lỗi bit vượt ngưỡng cho phép.

2.1.7. Cơ chế truyền nhận dữ liệu

PCS trong 10GBASE-R có nhiệm vụ chuyển đổi dữ liệu giữa giao diện XGMII ở phía trên và PMA/WIS ở phía dưới. Khi giao tiếp với XGMII, PCS sử dụng đường dữ liệu đồng bộ rộng 4 octet, tương ứng với các tín hiệu `TXD<31:0>`, `TXC<3:0>` ở phía phát và `RXD<31:0>`, `RXC<3:0>` ở phía thu. Khi giao tiếp với PMA hoặc WIS, PCS sử dụng đường dữ liệu đồng bộ rộng 16 bit để truyền các bit đã được mã hóa [4].



Hình 2.10: Quá trình tạo khối 64B/66B và xáo trộn dữ liệu trong PCS 10GBASE-R

Hình 2.10 mô tả quá trình xử lý dữ liệu ở chiều phát của PCS 10GBASE-R. Dữ liệu từ giao diện XGMII được đưa vào PCS thông qua hai nhóm tín hiệu chính là TXD<31:0> và TXC<3:0>. Trong đó, TXD<31:0> chứa bốn octet dữ liệu hoặc mã điều khiển, còn TXC<3:0> cho biết từng octet tương ứng là dữ liệu hay mã điều khiển. Nếu bit TXC bằng 0 thì octet tương ứng được xem là dữ liệu, nếu bit TXC bằng 1 thì octet tương ứng là mã điều khiển.

Trong quá trình mã hóa 64B/66B, hai lần truyền liên tiếp từ XGMII được ghép lại để tạo thành tám ký tự, tương ứng với 64 bit dữ liệu đầu vào cho khối Encode. Dựa

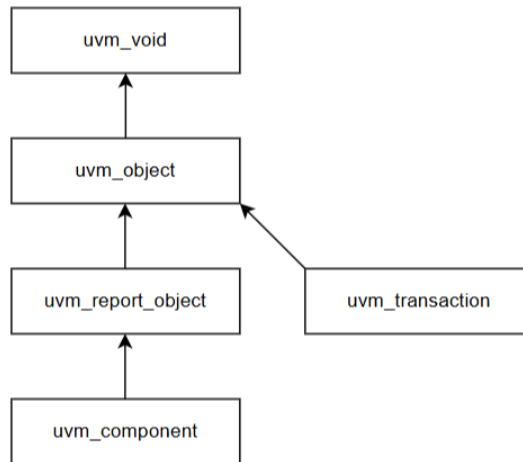
vào trạng thái của các bit điều khiển TXC, khối Encode sẽ phân loại dữ liệu để tạo thành data block hoặc control block. Đối với data block, sync header có giá trị 01; đối với control block, sync header có giá trị 10. Như vậy, sau khối Encode, mỗi block có độ dài 66 bit, gồm 2 bit sync header và 64 bit payload [4]. Sau khi block 66 bit được tạo ra, phần payload 64 bit được đưa qua khối Scrambler để xáo trộn dữ liệu, trong khi sync header được giữ nguyên và không đi qua Scrambler. Việc giữ nguyên sync header giúp phía thu có thể xác định ranh giới block trong quá trình đồng bộ. Sau đó, khối Gearbox đóng gói luồng bit 66 bit thành các nhóm dữ liệu 16 bit để truyền xuống PMA hoặc WIS.

Ở chiều thu, quá trình xử lý được thực hiện theo hướng ngược lại. Dữ liệu từ PMA hoặc WIS được đưa vào PCS dưới dạng các nhóm 16 bit. Khối Block Sync có nhiệm vụ xác định đúng ranh giới các block 66 bit dựa trên sync header. Sau khi đạt trạng thái đồng bộ block, khối BER & Sync Header Monitor tiếp tục kiểm tra các sync header để phát hiện tình trạng lỗi bit cao trên đường truyền. Khi dữ liệu nhận được hợp lệ, phần payload của block 64B/66B được đưa qua khối Descrambler để khôi phục lại dữ liệu ban đầu trước khi xáo trộn. Sau đó, khối Decode giải mã block 64B/66B để tạo lại dữ liệu và tín hiệu điều khiển trên giao diện XGMII, tương ứng với RXD<31:0> và RXC<3:0>.

2.2. Phương pháp kiểm thử chung (UVM)

UVM là một phương pháp chuẩn hóa cho kiểm thử thiết kế mạch tích hợp. UVM được xây dựng trên khái niệm hướng đối tượng (OOP) của ngôn ngữ đặc tả phần cứng SystemVerilog. UVM cho phép người dùng tạo nên các thành phần có thể tái sử dụng cho việc kiểm thử các thiết kế mạch số hoặc SoC.

UVM được triển khai thành một thư viện lớp cơ sở (base class), cho phép người dùng xây dựng các thành phần bằng cách kế thừa các lớp, từ đó thêm các chức năng vào các thành phần của UVM trong môi trường kiểm thử. UVM tổ chức các thành phần bên trong nó theo các cấp bậc khác nhau, được biểu diễn như hình 2.4.



Hình 2.11: Các lớp cơ sở của UVM

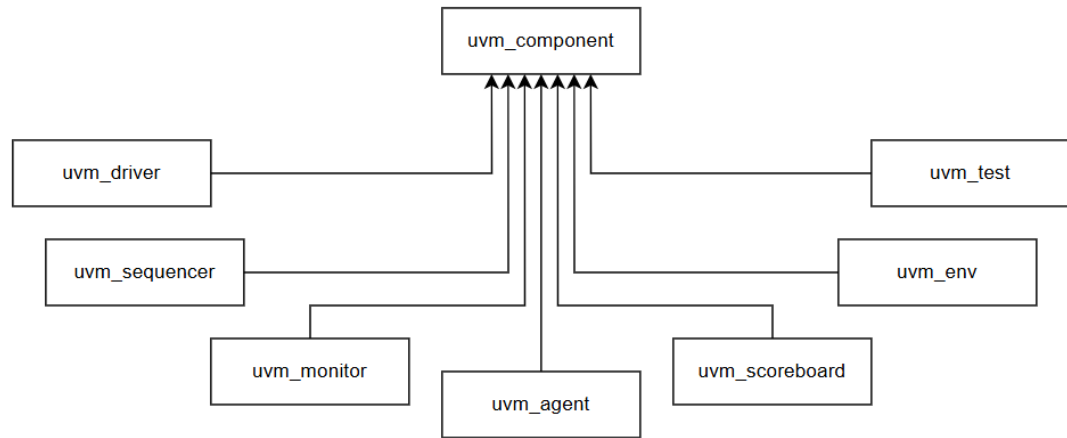
Ở cấp bậc cao nhất là lớp `uvm_void`. Lớp `uvm_void` là lớp cơ bản của mọi lớp khác trong UVM. Nó không chứa bất kì hàm hay dữ liệu nào nhưng cho phép các thành phần chứa (container) của các đối tượng (object) được tạo ra bên trong nó. Lớp tiếp theo trong phân cấp của UVM là `uvm_object`. Vai trò chính của lớp này là định nghĩa các phương thức cho các hoạt động chung như *create* (tạo), *compare* (so sánh), *copy* (sao chép), v.v.. Lớp `uvm_report_object` là nơi giao tiếp của các chức năng báo cáo (reporting facility) của UVM [7].

Hai lớp chính mà người dùng thường phải mở rộng để tự định nghĩa các thành phần (component) và trao đổi (transaction) là `uvm_component` và `uvm_transaction`. Lớp `uvm_component` định nghĩa các thành phần phục vụ việc kiểm thử như *agent*, *driver*, v.v. và lớp `uvm_transaction` định nghĩa các dữ liệu trong một trao đổi mà các thành phần thực hiện.

2.2.1. *uvm_component*

Lớp `uvm_component` là lớp cơ bản của các thành phần trong UVM. Lớp này kế thừa các chức năng có trong `uvm_object` và `uvm_report_object`. Các thành phần trong

lớp `uvm_component` được người dùng kế thừa để xây dựng một UVM testbench. Các cấp bậc này được biểu diễn như hình 2.5.



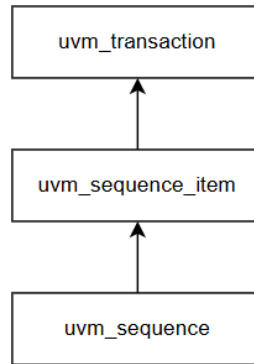
Hình 2.12: Các lớp kế thừa của `uvm_component`

Lớp `uvm_test` là thành phần ở cấp cao nhất trong một UVM testbench. Lớp này khởi tạo môi trường kiểm thử. Ở cấp bậc tiếp theo là `uvm_env`. Lớp này chứa các thành phần kiểm thử tập trung vào DUT. Các cấp bậc sau có các thành phần phụ thuộc vào việc xây dựng kiến trúc cho môi trường kiểm thử.

Lớp `uvm_scoreboard` có vai trò kiểm tra hành vi của một DUT. Lớp `uvm_agent` gom các thành phần xử lý giao diện (interface) của DUT. Lớp `uvm_sequencer` điều khiển dòng các trao đổi từ các chuỗi kích thích ngõ vào (stimulus). Lớp `uvm_driver` nhận trao đổi từ sequencer và đặt nó vào giao diện của DUT. Lớp `uvm_monitor` ghi lại thông tin của các trao đổi được gửi đi tại giao diện của DUT và gửi đi cho phần còn lại của testbench để phân tích [8].

2.2.2. *uvm_transaction*

Lớp `uvm_transaction` là lớp cơ bản của các trao đổi trong UVM. Có hai lớp chính kế thừa lớp này là `uvm_sequence_item` và `uvm_sequence`. Các cấp bậc này được biểu diễn như hình 2.6.



Hình 2.13: Các lớp kế thừa của uvm_transaction

Lớp `uvm_sequence_item` là lớp cho người dùng tự định nghĩa một trao đổi, hay ở đây còn được gọi là một mục trình tự (sequence item). Lớp `uvm_sequence` phục vụ việc tạo ra một chuỗi các trao đổi [7].

2.2.3. Các cơ chế trong UVM

2.2.3.1. Pha

Pha là một tập hợp những *function* (hàm) hoặc *task* (tác vụ) của SystemVerilog được các thành phần trong UVM thực hiện đồng thời. UVM sử dụng cơ chế pha để kiểm soát trạng thái khi mô phỏng [7].

Các pha cơ bản được giới thiệu trong [7] như sau:

- *uvm_build_phase*: Tạo và thiết lập cấu trúc của UVM testbench.
- *uvm_connect_phase*: Thiết lập kết nối giữa các thành phần.
- *uvm_end_of_elaboration_phase*: Tinh chỉnh testbench.
- *uvm_start_of_simulation_phase*: Chuẩn bị cho DUT được mô phỏng.
- *uvm_run_phase*: Kích mô phỏng DUT.
- *uvm_extract_phase*: Trích xuất dữ liệu tại các điểm khác nhau trong môi trường kiểm thử
- *uvm_check_phase*: Kiểm tra các điều kiện bất thường trong môi trường kiểm thử.

- *uvm_report_phase*: Báo cáo kết quả kiểm tra.
- *uvm_final_phase*: Kết thúc tại điểm cuối.

Đối với pha *run_phase*, để đồng bộ các thành phần và đảm bảo xác định đúng thời điểm kết thúc pha, phương thức *raise_objection* phải được gọi khi bắt đầu pha. Sau khi kết thúc các hoạt động, phương thức *drop_objection* phải được gọi ở cuối pha.

2.2.3.2. *Factory*

Factory là cơ chế tạo ra các đối tượng và thành phần của UVM [7]. UVM factory cho phép các thành phần trong UVM có thể tạo đối tượng mà không cần chỉ rõ lớp của đối tượng được tạo. Điều này hữu ích cho việc thay đổi hoặc ghi đè một thành phần bằng một thành phần mới [8].

Cơ chế này cần các thành phần trong UVM được đăng ký vào factory. Có nhiều cách để đăng ký một thành phần vào factory, tuy nhiên các cách được khuyến nghị là sử dụng macro *`uvm\_object\_utils* hoặc *`uvm_component_utils* [8].

2.2.3.3. *Config_db*

Lớp *uvm_config_db* là một cơ sở dữ liệu thường được dùng để truyền các cấu hình hoặc thông tin trên testbench. Các thông tin thường được truyền qua cơ chế này là cấu hình của các thành phần trong UVM hoặc các giao diện phục vụ các thành phần [7].

Có hai phương thức chính thường được dùng trong *config_db*. Thứ nhất là *set()*, dùng để đưa thông tin vào cơ sở dữ liệu và thứ hai là *get()* để lấy thông tin từ cơ sở dữ liệu.

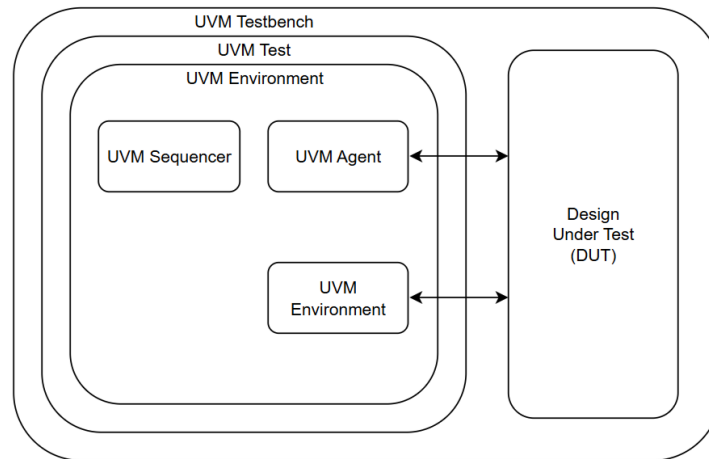
2.2.3.4. *Transaction-Level Modeling (TLM)*

UVM cung cấp một nhóm các kênh và giao diện mà người dùng có thể dùng để kết nối các thành phần tại mức trao đổi (transaction-level). Khi kết hợp với các cơ chế xây dựng theo pha của UVM, TLM cho khả năng tái sử dụng bằng cách cho phép cho

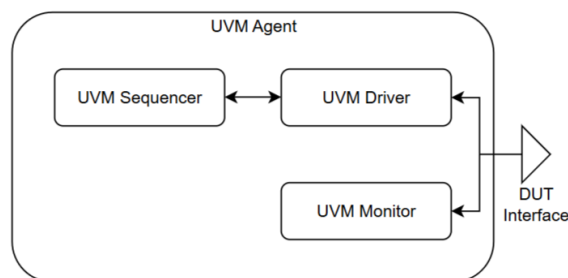
phép một thành phần có thể được thay thế bởi một thành phần khác có cùng giao diện [8]. TLM có 4 giao diện: *TLM1*, *TLM2*, *sequencer port* và *analysis* [7].

2.2.4. Kiến trúc thông thường của UVM

UVM có khuyến nghị một kiến trúc thông thường cho một UVM testbench [8]. Các kiến trúc này được biểu diễn như hình 2.14 và 2.15.



Hình 2.14: Kiến trúc thông thường của một UVM Testbench



Hình 2.15: Kiến trúc thông thường của một UVM Agent

Kiến trúc này được dùng làm một góc nhìn để giới thiệu về các thuật ngữ của UVM trong tài liệu [8]. Trên lý thuyết, người dùng có thể thiết kế kiến trúc bất kì cho một testbench sử dụng UVM.

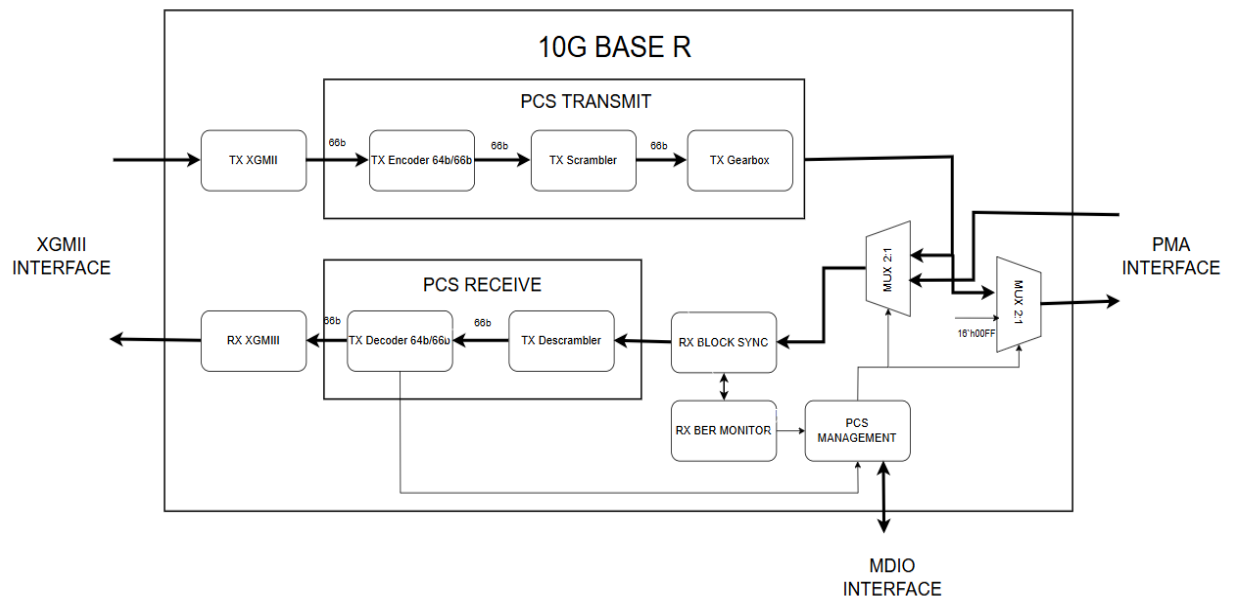
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1. PCS

Trong phạm vi đồ án, khối PCS_10GBASE_R được xây dựng bằng SystemVerilog ở mức mô phỏng chức năng nhằm kiểm tra luồng truyền – nhận dữ liệu theo giao diện XGMII. Thiết kế tập trung hiện thực các khối chính của PCS 10GBASE-R gồm khối xử lý XGMII, mã hóa và giải mã 64B/66B, xáo trộn và giải xáo trộn dữ liệu, chuyển đổi độ rộng dữ liệu, đồng bộ block, giám sát lỗi BER và khối thanh ghi quản lý trạng thái.

3.1.1. Sơ đồ khối tổng quát

Sơ đồ khối tổng quát của thiết kế 10GBASE R được thể hiện ở hình 3.1 như sau:



Hình 3.1: Sơ đồ khối tổng quát lớp mã hóa vật lý phụ 10GBASE R

Chức năng của từng khối trong thiết kế:

Đường phát dữ liệu: Nhiệm vụ truyền dữ liệu từ lớp MAC XGMII, tiến hành tiến hành nén, mã hóa và định dạng lại cấu trúc bit để truyền xuống lớp PMA.

- Khối TX XGMII DDR: Giao tiếp XGMII ở tốc độ 10 Gbps sử dụng xung nhịp 156.25 MHz ở chế độ DDR với bus 32-bit. Khối Adapter đóng vai trò chuyển đổi bắt dữ liệu ở cả sườn lên và sườn xuống, rồi gộp lại thành bus 64-bit SDR.
- Khối TX Encoder 64b66b: Nhiệm vụ chính là chuyển đổi dữ liệu từ định dạng XGMII sang định dạng 66-bit block để truyền xuống PMA.
- Khối TX Scrambler: Xáo trộn 64-bit payload của khối 66-bit nhằm tạo ra sự luân phiên trạng thái bit đều đặn, tránh các chuỗi 0 hoặc 1 kéo dài gây mất đồng bộ CDR ở phía thu.
- Khối TX Gearbox: Làm cầu nối về mặt kích thước bus dữ liệu giữa lớp PCS và lớp PMA chuyển đổi dữ liệu từ 66-bit xuống 16-bit.

Đường thu dữ liệu: Nhận dữ liệu thô từ cáp vật lý PMA, căn chỉnh khung, khôi phục lại dữ liệu nguyên bản và đẩy ngược lên lớp MAC.

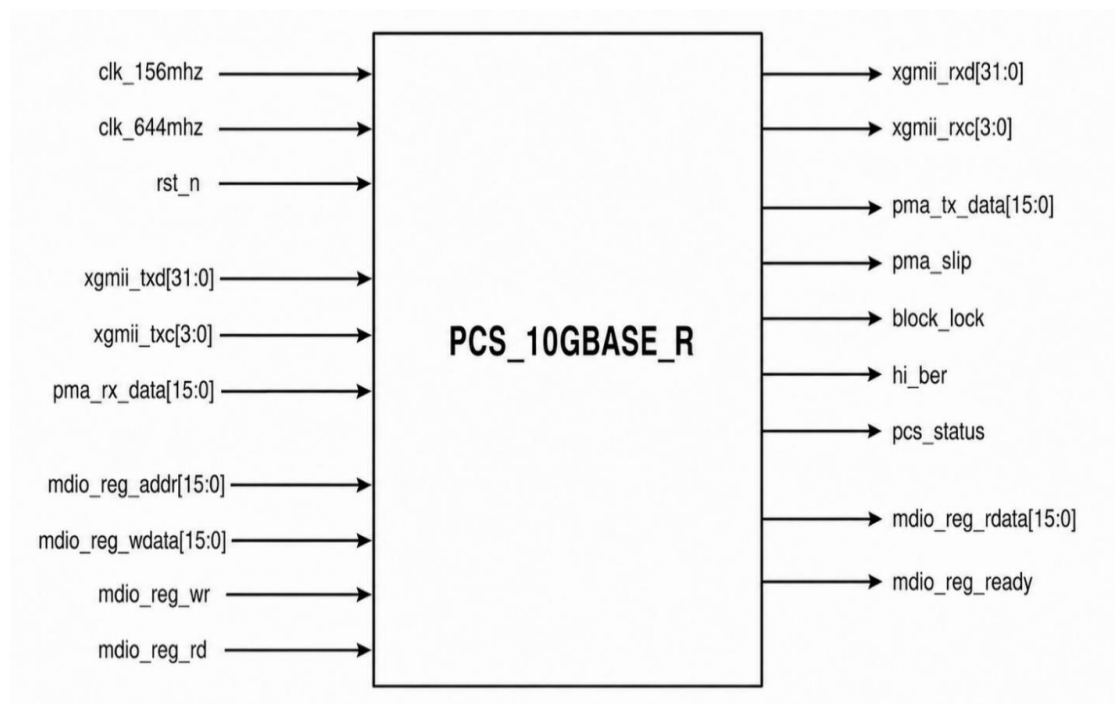
- Khối RX Block sync: Nhận dòng bit 16-bit liên tục từ PMA và tìm đúng điểm cắt căn chỉnh thành các khối 66-bit hoàn chỉnh có Sync Header hợp lệ.
- Khối RX Descrambler: Thực hiện phép đảo nghịch so với tx_scrambler để phục hồi 64 bit payload gốc.
- Khối RX Decoder: Phân tích Sync Header và Block Type của khối 66-bit, sau đó ánh xạ ngược 56-bit payload trở lại thành 64-bit dữ liệu XGMII gốc.
- Khối RX XGMII: Thực hiện quá trình nhận 64-bit SDR từ RX Decoder và tách thành 32bit DDR để giao tiếp với lớp MAC.

Khối quản lý và giám sát:

- Khối RX Ber monitor: Theo dõi chất lượng đường truyền sau khi block_lock = 1 bằng cách đếm Sync Header lỗi trong khung thời gian 125 us. Kích hoạt hi_ber nếu chất lượng quá xấu.

- Khối PCS Management: Cung cấp giao diện thanh ghi song song để mô phỏng một phần chức năng quản lý PCS.

Sơ đồ chân tín hiệu của 10GBASE R:



Hình 3.2: Các tín hiệu đầu vào và ra của khối PCS_10GBASE_R

Các chân tín hiệu của khối PCS sẽ được mô tả ở bảng 3.1

Bảng 3.1: Các chân tín hiệu của khối PCS_10GBASE_R

STT	Tín hiệu	Số bit	Phân loại	Mô tả
1	Clk_156mhz	1	Ngõ vào	Xung clock chính 156.25 MHz dùng cho toàn bộ datapath TX/RX và khối quản lý.
2	Clk_644mhz	1	Ngõ vào	Xung clock tốc độ cao 644.53 MHz dùng cho

				giao diện PMA (TX Gearbox xuất, RX Block Sync nhận)
3	rst_n	1	Ngõ vào	Tín hiệu reset đồng bộ tích cực mức thấp.
Giao diện XGMII				
4	xgmii_txd	32	Ngõ vào	Dữ liệu phát từ lớp MAC, truyền qua giao diện XGMII ở chế độ DDR 32bit/nửa chu kỳ, tổng 64bit/chu kỳ.
5	xgmii_txc	4	Ngõ vào	Tín hiệu điều khiển XGMII phía phát, DDR 4-bit/nửa chu kỳ, tổng 8-bit/chu kỳ. Mỗi bit tương ứng 1 lane: 1=Control (Idle/Start/Terminate/Error), 0 = Data
6	xgmii_rxd	32	Ngõ ra	Dữ liệu thu trả về lớp MAC, định dạng XGMII DDR 32bit. Là kết quả cuối cùng sau khi Block Sync, giải trộn và giải mã 64B/66B
7	xgmii_rxc	4	Ngõ ra	Tín hiệu điều khiển XGMII phía thu, DDR 4

				bit. Báo hiệu cho MAC biết byte tương ứng là Data hay ký tự điều khiển.
Giao diện PMA				
8	pma_tx_data	16	Ngõ ra	Luồng bit 16 bit/chu kỳ ở 644 MHz xuất xuống lớp PMA để truyền ra cáp. Ở chế độ Internal Loopback, giá trị này được thay bằng 16'h00FF để không gây nhiễu cáp vật lý.
9	pma_rx_data	16	Ngõ vào	Luồng bit 16 bit/chu kỳ ở 644 MHz nhận vào từ lớp PMA. Ở chế độ Internal Loopback, ngõ vào này bị bỏ qua, thay thế bằng dữ liệu từ đường TX nội bộ.
10	pma_slip	1	Ngõ ra	Tín hiệu yêu cầu PMA thực hiện bit slip để căn chỉnh ranh giới block 66-bit trong quá trình đồng bộ RX PCS. Tín hiệu này được kích hoạt khi PCS cần dịch pha/vị trí dữ liệu

				nhận nhằm đạt block_lock.
Tín hiệu trạng thái				
11	block_lock	1	Ngõ ra	Báo hiệu khởi rx_block_sync đã căn chỉnh thành công và khóa khung. Đã khóa 64 Sync Header hợp lệ liên tiếp. Là điều kiện tiên quyết để dữ liệu RX hợp lệ.
12	hi_ber	1	Ngõ ra	Báo hiệu tỷ lệ lỗi bit cao khi phát hiện ≥ 16 Sync Header lỗi trong thời gian 125 us. Khi hi_ber = 1, hệ thống cưỡng bức nhả khóa và tìm đồng bộ lại.
13	pcs_status	1	Ngõ ra	Tín hiệu tổng hợp trạng thái PCS: pcs_status = block_lock & ~hi_ber. 1 = Đường truyền hoạt động bình thường và ổn định.
Giao diện thanh ghi quản lý				
14	mdio_reg_addr	16	Ngõ vào	Địa chỉ thanh ghi cần đọc hoặc ghi. Các địa chỉ hỗ

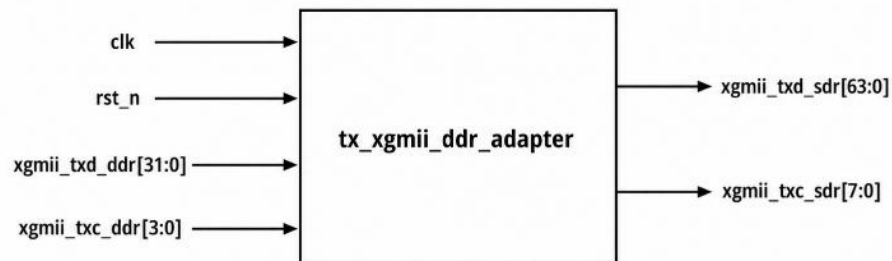
				trợ: 0x0000 PCS Control 1, 0x0001 PCS Status 1, 0x0008 PCS Status 2, 0x0020 BASE-R Status.
15	mdio_reg_wdata	16	Ngõ vào	Dữ liệu 16-bit cần ghi vào thanh ghi.
16	mdio_reg_wr	1	Ngõ vào	Tín hiệu cho phép ghi. Khi mdio_reg_wr = 1 trong 1 chu kỳ clock, giá trị mdio_reg_wdata được ghi vào thanh ghi tại địa chỉ mdio_reg_addr.
17	mdio_reg_rd	1	Ngõ vào	Tín hiệu cho phép đọc. Khi mdio_reg_rd = 1, dữ liệu của thanh ghi tại mdio_reg_addr được nạp vào mdio_reg_rdata ở chu kỳ tiếp theo.
18	mdio_reg_rdata	16	Ngõ ra	Dữ liệu 16-bit đọc về từ thanh ghi được chỉ định. Hợp lệ sau 1 chu kỳ clock kể từ khi mdio_reg_rd = 1 và mdio_reg_ready = 1.
19	mdio_reg_ready	1	Ngõ ra	Báo hiệu giao dịch MDIO hoàn tất. 1=Dữ liệu đọc mdio_reg_rdata hợp lệ

				hoặc lệnh ghi đã được chấp nhận.
--	--	--	--	----------------------------------

3.1.2. Khối giao tiếp XGMII

Khối giao tiếp XGMII có nhiệm vụ chuyển đổi dữ liệu giữa dạng 32-bit DDR trên giao diện XGMII và dạng 64-bit SDR dùng bên trong PCS. Trong mô phỏng, khối này lấy mẫu dữ liệu ở hai cạnh xung để ghép thành một word 64-bit ở phía phát, và tách word 64-bit thành hai chu kỳ 32-bit ở phía nhận.

3.1.2.1. Khối nhận dữ liệu từ XGMII



Hình 3.3: Mô hình khối nhận dữ liệu XGMII

Các chân tín hiệu có chức năng sau:

Các ngõ vào:

- clk: Tín hiệu xung clock khởi phát 156.25 MHz.
- rst_n: Tín hiệu reset (tích cực mức thấp).
- xgmii_txd_dds[31:0]: Bus dữ liệu ngõ vào DDR có độ rộng 32-bit.
- xgmii_txc_dds[3:0]: Bus điều khiển ngõ vào DDR có độ rộng 4-bit.

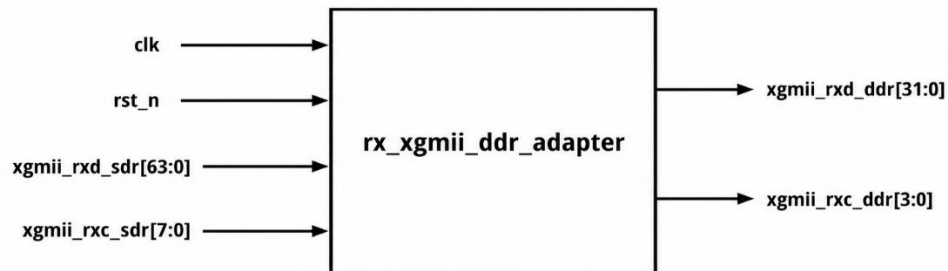
Các ngõ ra:

- xgmii_txd_sdr[63:0]: Bus dữ liệu ngõ ra SDR đã được ghép, có độ rộng 64-bit.

- xgmii_txc_sdr[7:0]: Bus điều khiển ngõ ra SDR đã được ghép, có độ rộng 8-bit

Khởi nhận dữ liệu XGMII Adapter thực hiện chuyển đổi dữ liệu từ dạng DDR 32-bit sang SDR 64-bit. Ở cạnh lên của xung clock, khối lấy mẫu 32-bit dữ liệu và 4-bit điều khiển đầu tiên. Ở cạnh xuống tiếp theo, khối tiếp tục lấy mẫu 32-bit dữ liệu và 4-bit điều khiển còn lại. Sau đó, tại cạnh lên kế tiếp, hai phần dữ liệu được ghép lại thành một word 64-bit cùng 8-bit điều khiển để đưa vào các khối xử lý bên trong PCS.

3.1.2.2. Khối truyền dữ liệu ra XGMII



Hình 3.4: Mô hình truyền dữ liệu XGMII

Các chân tín hiệu có chức năng sau:

Các ngõ vào:

- clk: Tín hiệu xung clock khởi thu 156.25 MHz.
- rst_n: Tín hiệu reset tích cực mức thấp.
- xgmii_rxd_sdr[63:0]: Bus dữ liệu ngõ vào SDR có độ rộng 64bit.
- xgmii_rxc_sdr[7:0]: Bus điều khiển ngõ vào SDR có độ rộng 8bit.

Các ngõ ra:

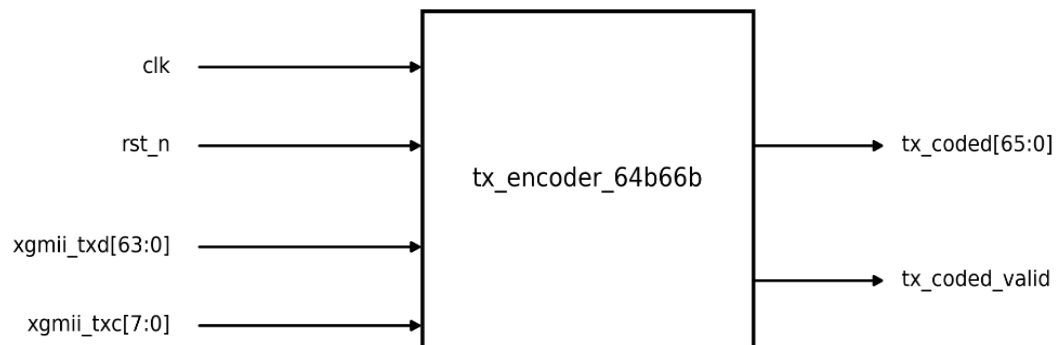
- xgmii_rxd_ddr[31:0]: Bus dữ liệu ngõ ra DDR đã được phân tách, có độ rộng 32bit.

- xgmii_rxc_ddr[3:0]: Bus điều khiển ngõ ra DDR đã được phân tách, có độ rộng 4bit.

Khối XGMII Adapter phía nhận thực hiện chuyển đổi dữ liệu từ dạng SDR 64-bit/8-bit control sang dạng DDR 32-bit/4-bit control. Ở cạnh lên của xung clock, khối chốt gói dữ liệu 64-bit đầu vào, đồng thời tách thành hai nửa 32-bit dữ liệu và 4-bit điều khiển. Nửa thấp được lưu vào các thanh ghi xgmii_rxd_pos và xgmii_rxc_pos, còn nửa cao được lưu tạm trong thanh ghi trung gian. Ở cạnh xuống kế tiếp, nửa dữ liệu còn lại được chốt vào các thanh ghi xgmii_rxd_neg và xgmii_rxc_neg. Sau đó, ngõ ra được chọn theo mức của tín hiệu clock: khi clock ở mức cao, khối xuất nửa dữ liệu thứ nhất; khi clock ở mức thấp, khối xuất nửa dữ liệu thứ hai.

3.1.3. Khối mã hóa và giải mã 64B/66B

3.1.3.1. Khối mã hóa 64B/66B



Hình 3.5: Mô hình khối mã hóa Encoder 64B/66B

Chức năng của tín hiệu trong khối:

Các ngõ vào:

- clk: Tín hiệu xung clock khởi phát 156.25 MHz.
- rst_n: Tín hiệu reset tích cực mức thấp.
- xgmii_txd [63:0]: Bus dữ liệu ngõ vào có độ rộng 64-bit.
- xgmii_txc [7:0]: Bus điều khiển ngõ vào có độ rộng 8-bit.

Các ngõ ra:

- tx_coded [65:0]: Block 66-bit đã được mã hóa theo chuẩn 64b/66b.
- tx_coded_valid: Tín hiệu báo hiệu cho biết ngõ ra tx_coded hợp lệ.

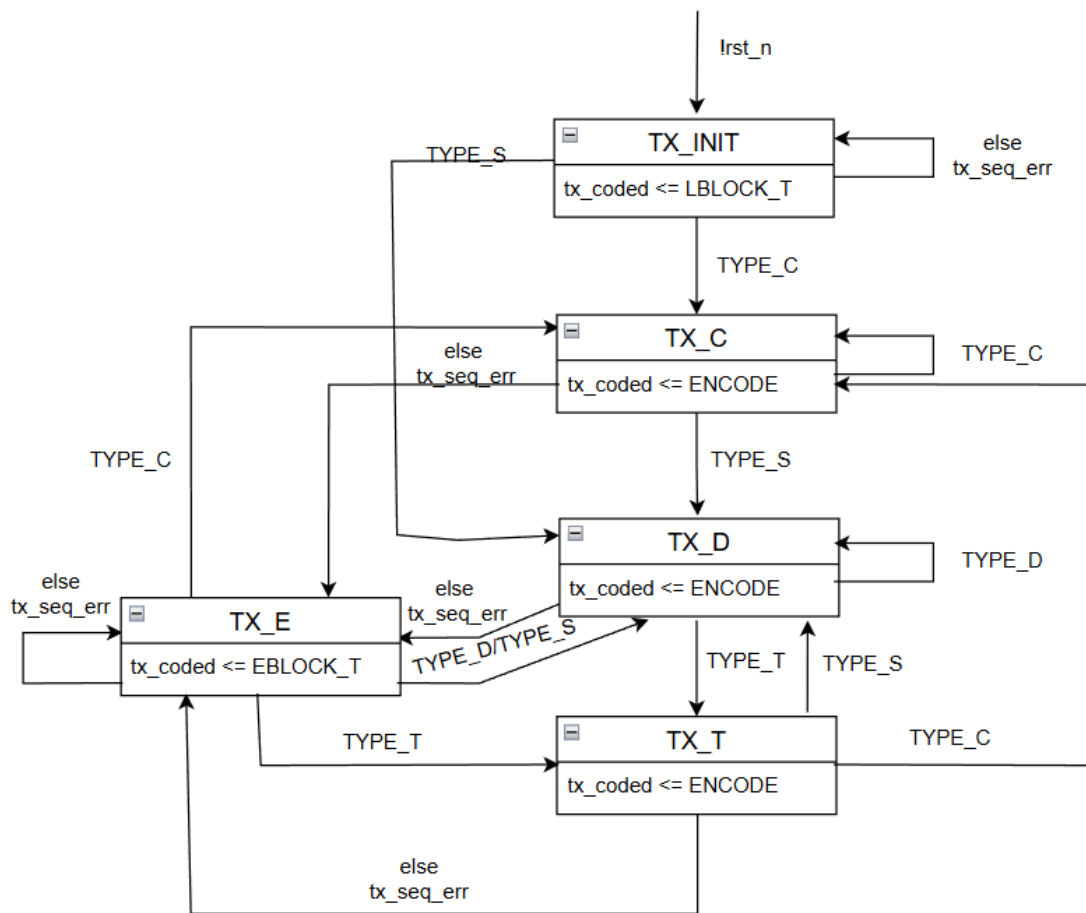
Nguyên lý hoạt động:

Dựa vào 8-bit của tín hiệu điều khiển xgmii_txc, xác định loại khối Block Type hiện tại và gom lại thành 3 thành phần chính: Header đồng bộ sync_gen 2bit, loại khối type_gen 8bit và dữ liệu 8bit đã được ánh xạ thành 7bit và được gom lại thành khung 56bit.

Khối Dữ liệu (TYPE_D): Khi xgmii_txc bằng 8'b0000_0000, khối được gắn header đồng bộ là 2'b01 và 64bit từ xgmii_tx dữ liệu được đưa trực tiếp vào tạo thành khối 66-bit.

Khối Điều khiển/Bắt đầu/Kết thúc: Khi xgmii_txc chứa các bit 1, module sẽ xác định đó là các khối Control TYPE_C, Start TYPE_S, hoặc Terminate TYPE_T và thiết lập header đồng bộ là 2'b10.

Khối sử dụng một máy trạng FSM với 5 trạng thái TX_INIT, TX_C, TX_D, TX_T và TX_E để kiểm tra tính hợp lệ của luồng dữ liệu liên tục như hình 3.6.



Hình 3.6: Sơ đồ trạng thái của khối mã hóa Encoder 64B/66B

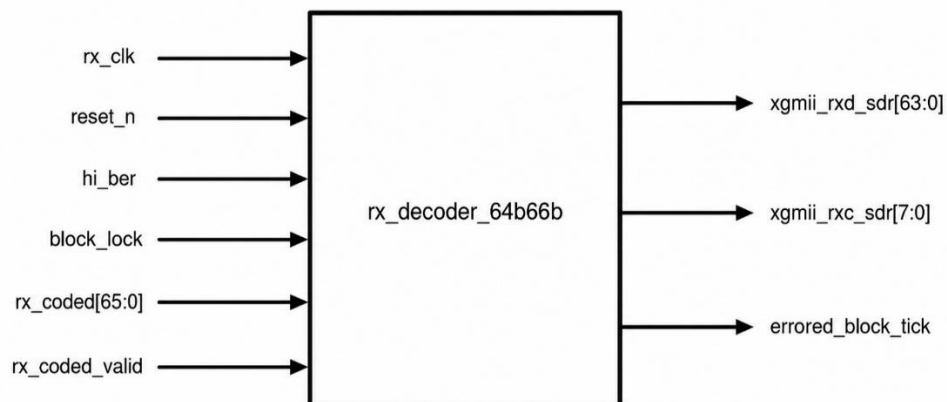
- Khi tín hiệu reset rst_n được kích hoạt, FSM được đưa về trạng thái TX_INIT. Đây là trạng thái khởi tạo của khối truyền. Trong trạng thái này, dữ liệu mã hóa đầu ra tx_coded được gán bằng LBLOCK_T, tức block báo Local Fault ở phía phát, nhằm thông báo rằng kênh truyền chưa sẵn sàng hoặc luồng dữ liệu đầu vào chưa ở trạng thái hợp lệ để truyền bình thường. Từ trạng thái TX_INIT, nếu khối nhận được block điều khiển TYPE_C, FSM chuyển sang trạng thái TX_C. Nếu nhận được block bắt đầu frame TYPE_S, FSM chuyển sang trạng thái TX_D để bắt đầu quá trình truyền frame dữ liệu. Nếu nhận được kiểu block khác không

phù hợp với trình tự, FSM giữ tại TX_INIT kích hoạt tín hiệu lỗi trình tự tx_seq_err.

- Tại trạng thái TX_C, khối đang xử lý các control block. Trong trạng thái này, ngõ ra tx_coded được tạo bằng quá trình mã hóa block đầu vào, tức tx_coded <= ENCODE. Nếu tiếp tục nhận được TYPE_C, FSM giữ nguyên ở trạng thái TX_C, vì các control block liên tiếp là hợp lệ khi đường truyền đang ở trạng thái rồi hoặc điều khiển. Nếu nhận được TYPE_S, FSM chuyển sang TX_D, cho biết một frame dữ liệu mới bắt đầu được truyền. Ngược lại, nếu xuất hiện các loại block không đúng trình tự là data block hoặc terminate block khi chưa có start block, FSM kích hoạt tx_seq_err và chuyển sang trạng thái TX_E.
- Tại trạng thái TX_D, khối đang trong quá trình truyền phần dữ liệu của frame. Ngõ ra tx_coded được tạo từ quá trình mã hóa tx_coded <= ENCODE. Nếu block đầu vào là TYPE_D, FSM giữ nguyên ở TX_D để tiếp tục truyền các data block liên tiếp trong frame. Khi nhận được block kết thúc TYPE_T, FSM chuyển sang trạng thái TX_T để xử lý việc kết thúc frame. Nếu trong trạng thái đang truyền dữ liệu mà nhận được một kiểu block không hợp lệ hoặc không đúng trình tự hoặc không thuộc các block hợp lệ, FSM sẽ kích hoạt cờ lỗi tx_seq_err và chuyển sang TX_E.
- Tại trạng thái TX_T, khối đã nhận được terminate block, nghĩa là frame hiện tại đã kết thúc. Trong trạng thái này, tx_coded vẫn được tạo bằng quá trình mã hóa block đầu vào. Sau khi kết thúc frame, nếu block tiếp theo là TYPE_C, FSM chuyển về TX_C để tiếp tục truyền các block điều khiển hoặc idle. Nếu block tiếp theo là TYPE_S, FSM chuyển sang TX_D để bắt đầu truyền frame mới ngay sau frame trước. Nếu block tiếp theo không thuộc các kiểu hợp lệ trong ngữ cảnh này, FSM xem đó là lỗi trình tự, kích hoạt tx_seq_err và chuyển sang TX_E.

- Trạng thái TX_E là trạng thái xử lý lỗi. Trong trạng thái này, khối phát block lỗi EBLOCK thay cho dữ liệu không hợp lệ nhằm tránh truyền sai dữ liệu lên đường truyền. Khi nhận được một kiểu block hợp lệ TYPE_C hoặc TYPE_D máy trạng thái sẽ thoát khỏi TX_E và chuyển về trạng thái tương ứng để tiếp tục hoạt động bình thường.

3.1.3.2. Khối giải mã 64B/66B



Hình 3.7: Mô hình khối giải mã Decoder 64B/66B

Chức năng của tín hiệu trong khối:

Các ngõ vào:

- rx_clk: Tín hiệu xung clock của khối thu 156.25 MHz.
- reset_n: Tín hiệu reset tích cực mức thấp.
- hi_ber: Tín hiệu báo lỗi BER cao từ khối BER monitor.
- block_lock: Tín hiệu báo khối Block Sync đã khóa được biên block 66-bit.
- rx_coded[65:0]: Block 66-bit đầu vào sau khi đã được đồng bộ và giải xáo trộn.
- rx_coded_valid: Tín hiệu báo block đầu vào hợp lệ.

Các ngõ ra:

- xgmii_rxd_sdr[63:0]: Bus dữ liệu XGMII sau giải mã, có độ rộng 64-bit.
- xgmii_rxc_sdr[7:0]: Bus điều khiển XGMII sau giải mã, có độ rộng 8-bit.
- errored_block_tick: Tín hiệu báo có block lỗi được phát hiện.

Nguyên lý hoạt động:

Khởi decoder thực hiện tách block 66-bit đầu vào thành ba thành phần chính: sync_header 2-bit, block_type 8-bit và payload 56-bit.

Dựa vào giá trị của sync header, khởi xác định loại block hiện tại:

- Nếu sync_header = 2'b01, block được xác định là block dữ liệu TYPE_D.
- Nếu sync_header = 2'b10, block được xác định là block điều khiển. Khi đó, khởi tiếp tục kiểm tra trường block_type để phân loại thành TYPE_C, TYPE_S, TYPE_T hoặc TYPE_E.
- Nếu sync_header có giá trị 00 hoặc 11, block được xem là không hợp lệ và chuyển sang loại lỗi TYPE_E.

Đối với block dữ liệu, khi sync_header bằng 2'b01, toàn bộ 64-bit payload được đưa trực tiếp ra bus xgmii_rxd_sdr, đồng thời xgmii_rxc_sdr được gán bằng 8'h00 để báo rằng cả 8 byte đều là dữ liệu.

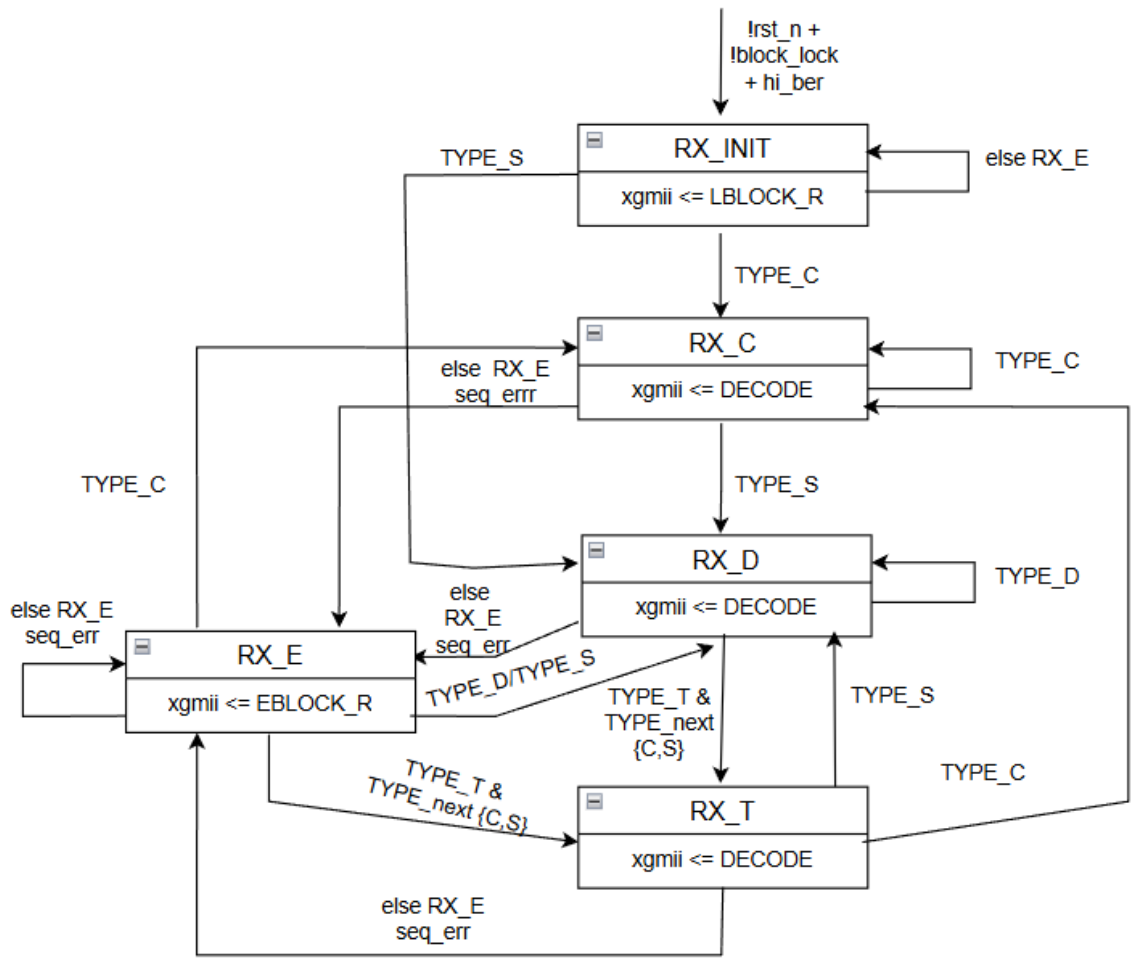
Đối với block điều khiển, khi sync_header bằng 2'b10, khởi sử dụng trường block_type để xác định định dạng của payload. Các block_type như 8'h1E, 8'h2D, 8'h4B, 8'h55 được xem là nhóm control block (TYPE_C). Các block_type 8'h78, 8'h33, 8'h66 được xem là nhóm Start block (TYPE_S). Các block_type 8'h87, 8'h99, 8'hAA, 8'hB4, 8'hCC, 8'hD2, 8'hE1, 8'hFF được xem là nhóm Terminate block (TYPE_T). Những giá trị block_type không thuộc các nhóm trên sẽ được xem là lỗi.

Các mã điều khiển 7-bit trong payload được giải mã ngược về mã điều khiển XGMII 8-bit theo bảng 3.2.

Đối với ordered set, mã O code 4-bit cũng được giải mã riêng. Trong thiết kế, O code = 4'h0 được giải mã thành 0x9C, tương ứng với Sequence Ordered Set. Ngoài ra, thiết kế cũng có hỗ trợ giải mã O code = 4'hF thành 0x5C.

Các ký tự Start /S/ và Terminate /T/ không được truyền trực tiếp bằng mã 7-bit mà được suy ra từ trường block type. Khi gặp Start block, khối gán byte tương ứng bằng 8'hFB. Khi gặp Terminate block, khối gán byte tương ứng bằng 8'hFD. Đồng thời, các bit tương ứng trong xgmii_rxc_sdr được gán bằng 1 để báo đó là byte điều khiển.

Khối sử dụng một máy trạng thái FSM gồm 5 trạng thái: RX_INIT, RX_C, RX_D, RX_T và RX_E để kiểm tra trình tự hợp lệ của luồng dữ liệu nhận theo hình 3.8.



Hình 3.8: Sơ đồ trạng thái khối giải mã Decoder 64B/66B

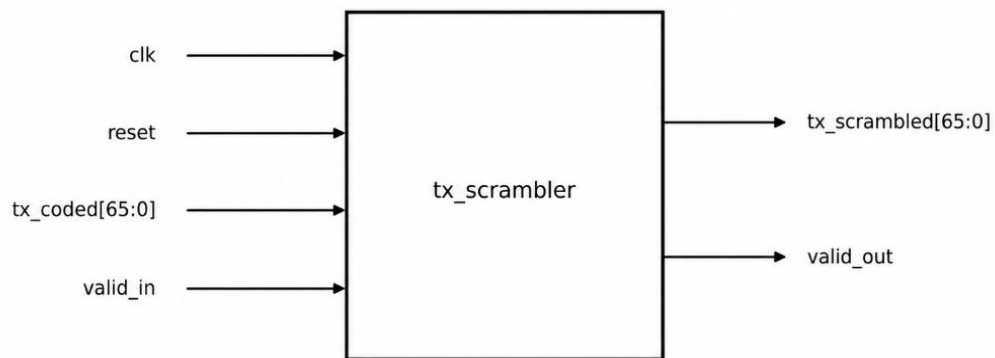
- Tại trạng thái RX_INIT, khối ở trạng thái khởi tạo hoặc chờ đường truyền ổn định. Nếu block_lock = 0 hoặc hi_ber = 1, FSM được đưa về RX_INIT. Trong trạng thái này, khối phát ra LBLOCK_R, tức block Local Fault, để báo với lớp phía trên rằng đường truyền thu chưa sẵn sàng. Từ trạng thái RX_INIT, nếu nhận được control block TYPE_C, FSM chuyển sang RX_C. Nếu nhận được Start block TYPE_S, FSM chuyển sang RX_D để bắt đầu nhận frame dữ liệu. Nếu nhận được loại block khác, FSM chuyển sang trạng thái lỗi RX_E.

- Tại trạng thái RX_C, FSM đang ở vùng nhận các control block hoặc idle block. Ở trạng thái này, ngõ ra XGMII nhận data từ quá trình giải mã block nhận được: $xgmii \leq \text{DECODE}$ nếu tiếp tục nhận được TYPE_C, FSM giữ nguyên ở RX_C. Nếu nhận được TYPE_S, FSM chuyển sang RX_D, vì đây là dấu hiệu bắt đầu một Ethernet frame mới. Nếu nhận được các loại block không đúng trình tự như data block TYPE_D hoặc terminate block TYPE_T khi chưa có Start block, FSM xem đây là lỗi trình tự, kích hoạt seq_err, và chuyển sang RX_E.
- Tại trạng thái RX_D, FSM đang trong quá trình nhận phần dữ liệu của frame. Ngõ ra XGMII tiếp tục được lấy từ kết quả giải mã: $xgmii \leq \text{DECODE}$. Nếu nhận được TYPE_D, FSM giữ nguyên ở RX_D, vì các data block liên tiếp nằm giữa Start block và Terminate block là hợp lệ. Nếu nhận được TYPE_S trong khi đang ở RX_D được xem là tình huống bắt đầu frame mới khi frame trước chưa kết thúc đúng cách chuyển sang RX_E. Khi nhận được TYPE_T điều kiện hợp lệ trong hình là: $\text{TYPE_T} \& \text{TYPE_next} \{C, S\}$ nghĩa là sau Terminate block, block kế tiếp phải là TYPE_C hoặc TYPE_S, FSM chuyển sang RX_T để xử lý kết thúc frame nếu không đúng, FSM kích hoạt seq_err và chuyển sang RX_E.
- Tại trạng thái RX_T, frame đã kết thúc. Sau đó, nếu khối nhận được control block thì FSM chuyển về RX_C; nếu nhận được Start block mới thì FSM chuyển sang RX_D để bắt đầu frame tiếp theo. Các loại block khác trong trạng thái này đều bị xem là sai trình tự chuyển sang RX_E.
- Trạng thái RX_E là trạng thái báo lỗi và phục hồi lỗi. Tại trạng thái này, khối không đưa dữ liệu giải mã sai lên giao tiếp XGMII. Thay vào đó, ngõ ra được gán bằng: $xgmii \leq \text{EBLOCK_R}$. EBLOCK_R là error block toàn bộ 8 byte dữ liệu được gán bằng mã lỗi 8'hFE, đồng thời toàn bộ bit control được đặt bằng 1. Điều này giúp lớp MAC hoặc lớp phía trên nhận biết rằng block hiện tại bị lỗi. Khi FSM chuyển vào RX_E, tín hiệu

errored_block_tick được kích hoạt trong một chu kỳ clock để phục vụ thông kê lỗi. Trong trạng thái RX_E, FSM tiếp tục kiểm tra các block tiếp theo để thoát lỗi. Nếu nhận được TYPE_C, FSM có thể chuyển về RX_C. Nếu nhận được TYPE_S hoặc TYPE_D FSM có thể chuyển sang RX_D. Nếu nhận được TYPE_T và block kế tiếp thuộc {TYPE_C, TYPE_S}, FSM chuyển sang RX_T. Nếu tiếp tục nhận được block sai loại, block không hợp lệ, hoặc sai trình tự, FSM vẫn ở RX_E và tiếp tục phát EBLOCK_R.

3.1.4. Khối xáo trộn và giải xáo trộn

3.1.4.1. Khối xáo trộn



Hình 3.9: Mô hình khối xáo trộn dữ liệu Scrambler

Chức năng của các tín hiệu trong khối:

Các ngõ vào:

- clk: Tín hiệu xung clock khối phát 156.25 MHz.
- reset: Tín hiệu reset tích cực mức thấp.
- tx_coded [65:0]: Block 66-bit đầu vào sau khi được mã hóa bởi khối encoder.
- valid_in: Tín hiệu báo dữ liệu đầu vào tx_coded hợp lệ.

Các ngõ ra:

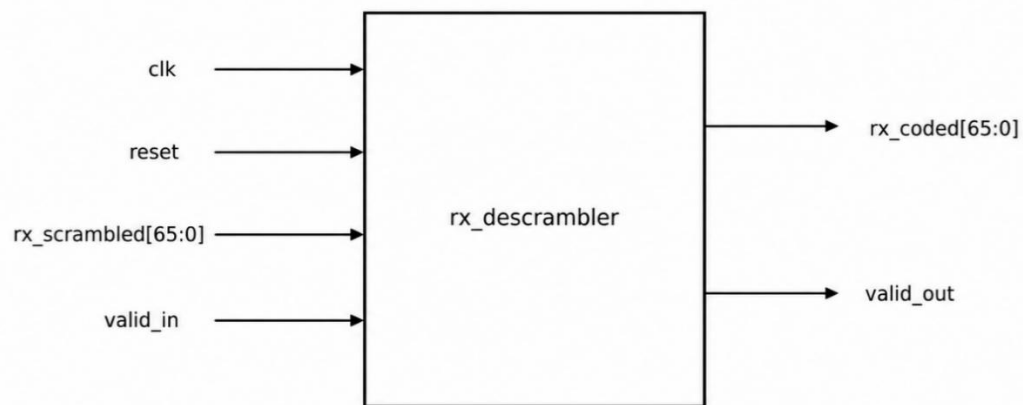
- tx_scrambled [65:0]: Block 66-bit đầu ra sau khi đã xáo trộn phần payload.
- valid_out: Tín hiệu báo dữ liệu đầu ra tx_scrambled hợp lệ.

Nguyên lý hoạt động:

Khi có dữ liệu hợp lệ, khối tách tx_coded [65:0] thành hai phần: tx_coded [1:0] là sync header và tx_coded [65:2] là payload 64 bit. Sync header được giữ nguyên, còn payload được xử lý qua scrambler.

Trong quá trình xáo trộn, mỗi bit payload đầu ra được tạo bằng phép XOR giữa bit dữ liệu đầu vào hiện tại với hai bit hồi tiếp tại vị trí trễ 39 bit và 58 bit. Thanh ghi history_q [57:0] được dùng để lưu lịch sử các bit đã xáo trộn trước đó, giúp quá trình scrambler được duy trì liên tục giữa các block.

3.1.4.2. Khối giải xáo trộn



Hình 3.10: Mô hình khối giải xáo trộn dữ liệu Desrambler

Các ngõ vào:

- clk: Tín hiệu xung clock khối thu 156.25 MHz.
- reset: Tín hiệu reset tích cực mức thấp.
- rx_scrambled [65:0]: Block 66 bit đầu vào sau khi đã đồng bộ block.
- valid_in: Tín hiệu báo dữ liệu đầu vào hợp lệ.

Các ngõ ra:

- rx_coded [65:0]: Block 66 bit sau khi đã giải xáo trộn payload.
- valid_out: Tín hiệu báo dữ liệu đầu ra hợp lệ.

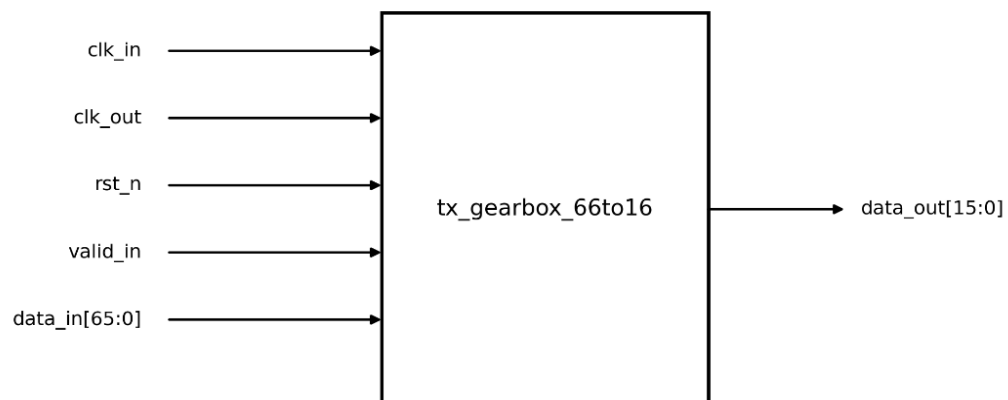
Nguyên lý hoạt động:

Khi có dữ liệu, khối tách rx_scrambled [65:0] thành hai phần: rx_scrambled [1:0] là sync header và rx_scrambled [65:2] là payload 64 bit đã bị xáo trộn. Sync header được giữ nguyên, còn payload được đưa vào mạch descrambler.

Trong quá trình giải xáo trộn, mỗi bit payload đầu ra được tạo bằng phép [9]p XOR giữa bit payload đầu vào hiện tại với hai bit hồi tiếp tương ứng tại vị trí trễ 39 bit và 58 bit. Thanh ghi history_q [57:0] được sử dụng để lưu lịch sử 58-bit payload đã nhận trước đó.

Đối với các bit đầu của block hiện tại, giá trị hồi tiếp được lấy từ thanh ghi history_q. Đối với các bit sau, giá trị hồi tiếp được lấy trực tiếp từ các bit payload đầu vào trước đó trong cùng block. Đây là điểm khác với khối scrambler truyền, vì descrambler sử dụng lịch sử của dữ liệu đã xáo trộn ở đầu vào để khôi phục lại payload ban đầu.

3.1.5. Khối chuyển độ rộng dữ liệu



Hình 3.11: Mô hình khối chuyển độ rộng dữ liệu Gearbox

Chức năng của các tín hiệu trong khối:

Các ngõ vào:

- clk_in: Xung clock dùng để ghi dữ liệu 66 bit vào FIFO 156.25 MHz.
- clk_out: Xung clock dùng để đọc dữ liệu và xuất dữ liệu 16 bit ra PMA 644.53 MHz
- rst_n: Tín hiệu reset tích cực mức thấp.
- valid_in: Tín hiệu báo dữ liệu đầu vào hợp lệ.
- data_in [65:0]: Block dữ liệu 66 bit đầu vào sau khối scrambler.

Ngõ ra:

- data_out [15:0]: Dữ liệu đầu ra 16 bit truyền xuống phía PMA.

Cấu tạo:

Bộ nhớ FIFO fifo_mem [0:127]: Dùng để lưu tạm các block dữ liệu 66 bit từ ngõ vào data_in [65:0]. FIFO có độ sâu 128 phân tử, mỗi phân tử có độ rộng 66 bit.

Khối ghi FIFO: Hoạt động theo xung clk_in. Khi valid_in ở mức 1 và FIFO chưa đầy, dữ liệu 66 bit được ghi vào FIFO. Con trỏ ghi wr_ptr dùng để xác định vị trí ghi dữ liệu.

Khối đọc FIFO: Hoạt động theo xung clk_out. Khi bộ đệm ngõ ra còn đủ chỗ và FIFO không rỗng, một block 66 bit sẽ được đọc ra từ FIFO. Con trỏ đọc rd_ptr dùng để xác định vị trí đọc dữ liệu.

Khối đồng bộ con trỏ Gray code: Do khối sử dụng hai miền clock khác nhau là clk_in và clk_out, các con trỏ đọc và ghi được chuyển sang dạng Gray code trước khi đồng bộ qua miền clock còn lại. Việc này giúp giảm lỗi khi truyền tín hiệu điều khiển giữa hai miền clock khác nhau.

Khởi kiểm tra trạng thái FIFO: Gồm hai tín hiệu `fifo_full` và `fifo_empty`. Tín hiệu `fifo_full` cho biết FIFO đã đầy để ngăn ghi thêm dữ liệu, còn `fifo_empty` cho biết FIFO đang rỗng để ngăn đọc dữ liệu không hợp lệ.

Thanh ghi đệm `buffer_reg [127:0]`: Dùng để gom các block 66-bit đọc ra từ FIFO thành một chuỗi bit liên tục. Từ thanh ghi này, dữ liệu được tách ra từng đoạn 16-bit để đưa ra ngõ `data_out [15:0]`.

Bộ đếm `bit_count`: Dùng để theo dõi số lượng bit hiện đang có trong `buffer_reg`. Khi đủ từ 16 bit trở lên, khối sẽ xuất 16 bit thấp nhất ra ngoài và dịch thanh ghi đệm sang phải 16 bit.

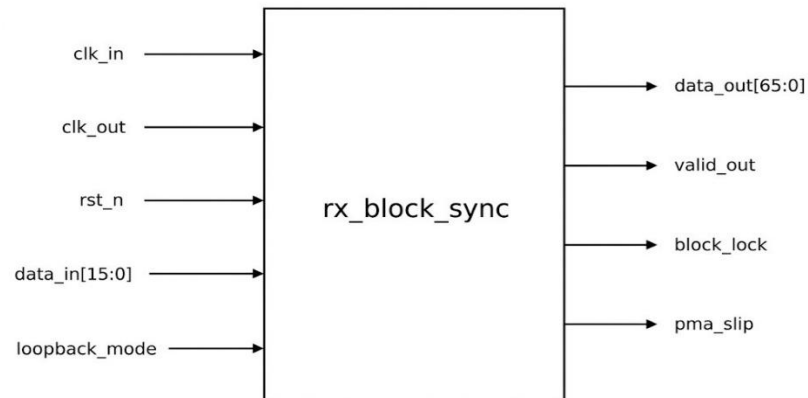
Nguyên lý hoạt động:

Khối gearbox sử dụng một FIFO để lưu tạm các block dữ liệu 66 bit. Khi `valid_in` ở mức 1 và FIFO chưa đầy, dữ liệu `data_in [65:0]` được ghi vào FIFO theo xung `clk_in`. Để đảm bảo việc truyền dữ liệu giữa hai miền clock khác nhau, khối sử dụng con trỏ đọc và con trỏ ghi dạng Gray code. Các con trỏ này được đồng bộ qua hai tầng thanh ghi trước khi dùng để xác định trạng thái FIFO đầy hoặc rỗng.

Ở miền `clk_out`, dữ liệu 66 bit được đọc từ FIFO khi bộ đếm bên trong còn đủ chỗ chứa. Dữ liệu đọc ra được đưa vào thanh ghi đệm `buffer_reg [127:0]`. Thanh ghi này có nhiệm vụ gom các block 66 bit liên tiếp và sắp xếp thành một chuỗi bit liên tục. Biến `bit_count` được sử dụng để theo dõi số lượng bit hiện đang có trong bộ đệm.

Tại mỗi chu kỳ `clk_out`, nếu trong bộ đệm có từ 16 bit trở lên, khối sẽ xuất 16 bit thấp nhất của `buffer_reg` ra ngõ `data_out [15:0]`. Sau đó, nội dung trong `buffer_reg` được dịch phải 16 bit và `bit_count` giảm đi 16. Khi số bit trong bộ đệm giảm xuống còn nhỏ hơn hoặc bằng 62 bit và FIFO chưa rỗng, khối tiếp tục đọc thêm một block 66 bit từ FIFO và ghép vào vị trí tiếp theo trong bộ đệm.

3.1.6. Khối đồng bộ khối



Hình 3.12: Mô hình khối đồng bộ khối

Chức năng của các tín hiệu trong khối:

Các ngõ vào:

- clk_in: Xung clock phía PMA, dùng để nhận dữ liệu 16 bit 644.53 MHz.
- clk_out: Xung clock phía PCS, dùng để xuất block 66 bit 156.25 MHz.
- rst_n: Tín hiệu reset tích cực mức thấp.
- data_in[15:0]: Dữ liệu 16 bit đầu vào từ PMA.
- loopback_mode: Tín hiệu chọn chế độ hoạt động, gồm chế độ normal và loopback.

Các ngõ ra:

- data_out[65:0]: Block dữ liệu 66 bit sau khi đã đồng bộ.
- valid_out: Tín hiệu báo dữ liệu đầu ra hợp lệ.
- block_lock: Tín hiệu báo khối đã khóa được biên block 66 bit.
- pma_slip: Tín hiệu yêu cầu PMA dịch cửa sổ dữ liệu thêm 1 bit khi chưa đồng bộ đúng.

Cấu tạo của khối:

Khối rx_block_sync gồm các thành phần chính sau:

Bộ đệm đầu vào `in_buf[127:0]`: Dùng để tích lũy dữ liệu 16 bit từ PMA thành chuỗi bit liên tục. Bộ đếm `in_count` dùng để theo dõi số bit hiện có trong bộ đệm.

Khởi tạo block 66 bit: Khi bộ đệm đầu vào có đủ từ 66 bit trở lên, khởi tách 66 bit thấp nhất để tạo thành một block dữ liệu đưa sang miền `clk_out`.

FIFO bất đồng bộ: Được sử dụng trong chế độ `loopback_mode = 1`. FIFO lưu các block 66 bit và hỗ trợ truyền dữ liệu giữa hai miền clock khác nhau. Các con trỏ đọc/ghi được mã hóa Gray code để giảm lỗi khi đồng bộ qua miền clock.

Khởi truyền dữ liệu trực tiếp CDC: Được sử dụng trong chế độ `normal`. Khi có block 66 bit mới, dữ liệu được chốt vào `direct_latch` và dùng cơ chế toggle synchronizer để báo sang miền `clk_out`.

Khởi yêu cầu slip: Khi phát hiện sync header không hợp lệ trong chế độ `normal`, khởi tạo yêu cầu slip và đồng bộ tín hiệu này ngược về miền `clk_in`. Sau đó, tín hiệu `pma_slip` được kích hoạt trong một chu kỳ để yêu cầu PMA dịch dữ liệu thêm 1 bit.

Bộ đệm đầu ra `out_buf[135:0]`: Dùng để lưu các block nhận được ở miền `clk_out` trước khi kiểm tra sync header và xuất ra ngoài.

Bộ kiểm tra sync header và FSM block lock: Dùng để kiểm tra 2 bit đầu của mỗi block. Nếu sync header có giá trị 2'b01 hoặc 2'b10, block được xem là hợp lệ. Khởi sử dụng các bộ đếm `sh_cnt` và `sh_inv_cnt` để xác định trạng thái khóa hoặc mất khóa block.

Nguyên lý hoạt động:

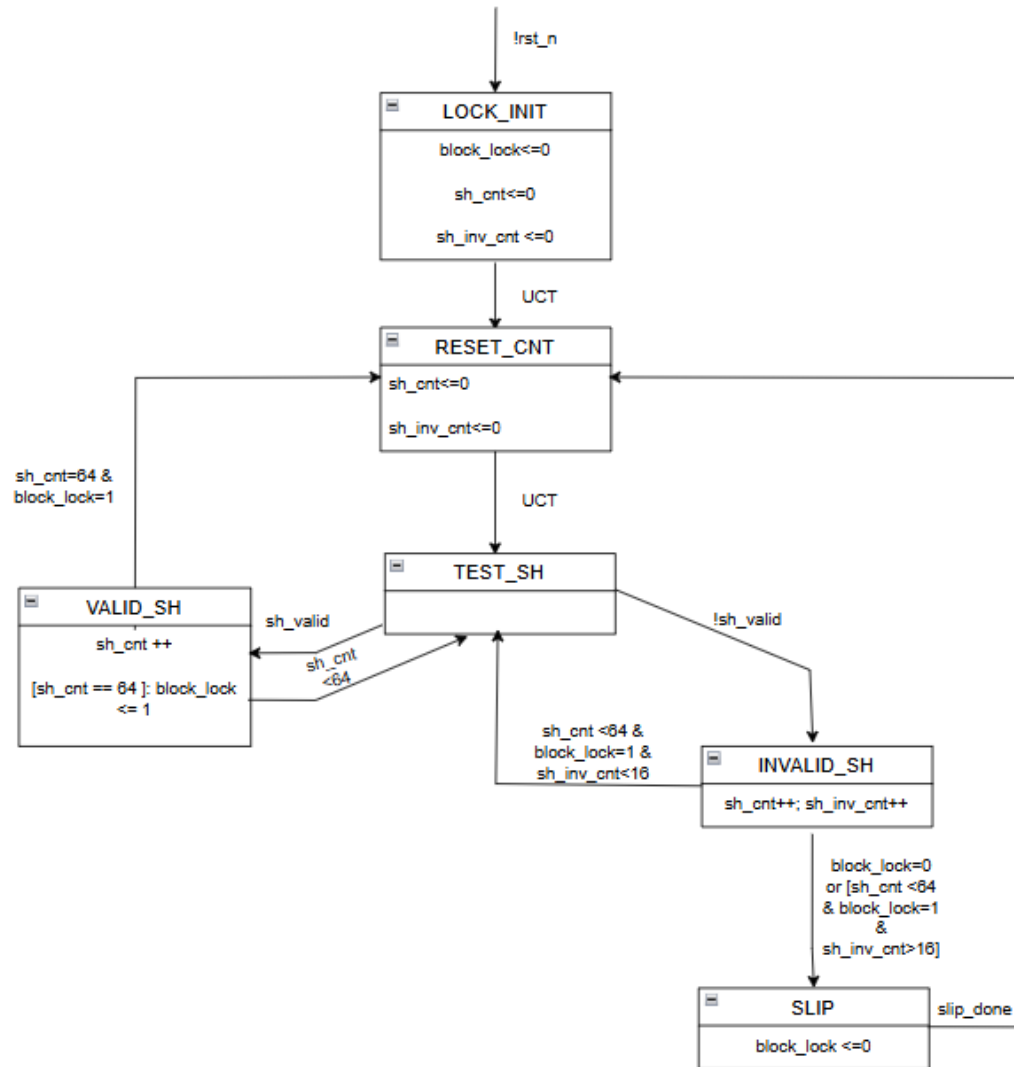
Ở xung `clk_in`, dữ liệu `data_in[15:0]` từ PMA được đưa liên tục vào bộ đệm `in_buf`. Mỗi chu kỳ clock, 16 bit mới được ghép vào bộ đệm và `in_count` được tăng thêm 16. Khi số bit trong bộ đệm đạt từ 66 bit trở lên, khởi tách ra một block 66 bit để đưa sang miền `clk_out`.

Trong chế độ loopback, block 66 bit được ghi vào FIFO bất đồng bộ. Ở miền clk_out, khi FIFO không rỗng và bộ đệm đầu ra còn chỗ trống, dữ liệu được đọc ra để kiểm tra đồng bộ. Nếu sync header sai, khối thực hiện slip nội bộ bằng cách dịch bộ đệm đi 1 bit để tìm lại biên block đúng.

Trong chế độ normal, block 66 bit không đi qua FIFO mà được truyền sang miền clk_out bằng cơ chế toggle synchronizer. Khi phát hiện có block mới, khối đưa dữ liệu vào bộ đệm đầu ra để tiếp tục kiểm tra sync header. Nếu sync header không hợp lệ khi chưa khóa block, khối tạo tín hiệu yêu cầu slip. Tín hiệu này được đồng bộ về miền clk_in và phát ra pma_slip trong một chu kỳ, yêu cầu PMA dịch cửa sổ dữ liệu thêm 1 bit.

Ở miền clk_out, khối kiểm tra 2 bit sync header của từng block. Nếu sync header bằng 2'b01 hoặc 2'b10, block được xem là hợp lệ. Khi khối chưa khóa block, nếu nhận đủ 64 sync header hợp lệ liên tiếp (vì xác suất dò sai block 66 bit khi dò 64 sync header hợp lệ là $\left(\frac{1}{2}\right)^{64}$ tín hiệu block_lock được đưa lên mức 1 để báo rằng biên block 66 bit đã được xác định chính xác.

Sau khi đã khóa block, khối tiếp tục giám sát sync header của các block kế tiếp. Trong mỗi nhóm 64 block, nếu số sync header không hợp lệ đạt từ 16 trở lên, khối sẽ xóa tín hiệu block_lock và quay lại quá trình tìm đồng bộ. Khi block hợp lệ được xuất ra, tín hiệu valid_out được kích hoạt trong một chu kỳ clk_out.



Hình 3.13: Sơ đồ trạng thái khởi đồng bộ khối 64B/66B

Khi tín hiệu reset tích cực mức thấp $\text{rst_n} = 0$ được kích hoạt, hệ thống chuyển vào trạng thái LOCK_INIT. Tại trạng thái này, tín hiệu block_lock được đưa về 0, đồng thời hai bộ đếm sh_cnt và sh_inv_cnt cũng được xóa về 0. Điều này cho biết khối Block Sync chưa đạt trạng thái đồng bộ block và cần bắt đầu quá trình tìm kiếm ranh giới block 66 bit. Sau trạng thái LOCK_INIT máy trạng thái chuyển không điều kiện, tức UCT – Unconditional Transition, sang trạng thái RESET_CNT.

Trạng thái RESET_CNT: các bộ đếm tiếp tục được xóa lại: $\text{sh_cnt} \leq 0$ $\text{sh_inv_cnt} \leq 0$. Trong đó, sh_cnt được dùng để đếm tổng số sync header đã kiểm tra

trong một chu kỳ kiểm tra, còn sh_inv_cnt dùng để đếm số sync header không hợp lệ. Sau đó, FSM tiếp tục chuyển không điều kiện sang trạng thái TEST_SH.

Trạng thái TEST_SH: là trạng thái kiểm tra sync header tại vị trí bit hiện tại. Trong mã hóa 64B/66B, mỗi block có 2 bit sync header. Sync header hợp lệ chỉ có thể là 01 đối với data block hoặc 10 đối với control block. Nếu sync header có giá trị hợp lệ, tín hiệu sh_valid được kích hoạt và máy trạng thái chuyển sang VALID_SH. Nếu sync header không hợp lệ, tức không phải 01 hoặc 10, tín hiệu !sh_valid được xác nhận và máy trạng thái chuyển sang INVALID_SH.

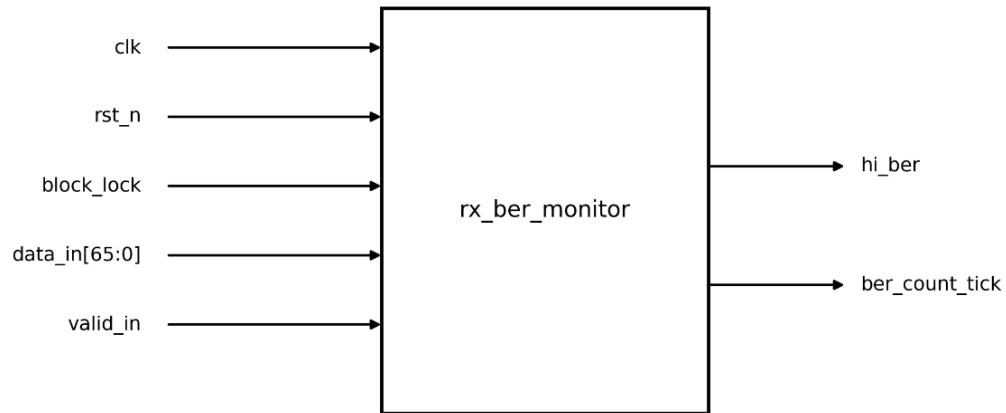
Trạng thái VALID_SH: bộ đếm sh_cnt được tăng lên một đơn vị: sh_cnt++. Trạng thái này cho biết hệ thống vừa kiểm tra được một sync header hợp lệ. Nếu hệ thống chưa đạt đủ số lượng sync header cần thiết, tức sh_cnt < 64, FSM sẽ quay lại TEST_SH để tiếp tục kiểm tra sync header tiếp theo tại cùng vị trí căn chỉnh block hiện tại. Khi số lượng sync header hợp lệ đạt ngưỡng 64, điều kiện: sh_cnt == 64 được thỏa mãn. Khi đó, tín hiệu: block_lock <= 1 được thiết lập. Điều này có nghĩa là khối Block Sync đã tìm được ranh giới block 66 bit chính xác. Từ thời điểm này, dữ liệu đầu vào có thể được tách thành các block 66 bit đúng định dạng để đưa sang các khối xử lý tiếp theo. Sau khi block_lock được thiết lập, hệ thống không dừng lại mà tiếp tục giám sát sync header để đảm bảo trạng thái đồng bộ vẫn còn hợp lệ. Khi đã kiểm tra đủ một cửa sổ 64 sync header trong trạng thái đã lock, tức điều kiện: sh_cnt = 64 & block_lock = 1 được thỏa mãn, FSM sẽ quay lại RESET_CNT để xóa sh_cnt và sh_inv_cnt, sau đó bắt đầu một chu kỳ kiểm tra mới. Cơ chế này giúp hệ thống liên tục theo dõi chất lượng đồng bộ trong quá trình hoạt động.

Trạng thái INVALID_SH, hệ thống phát hiện sync header không hợp lệ. Khi đó, cả hai bộ đếm đều được tăng: sh_cnt++, sh_inv_cnt++. Trong đó, sh_cnt vẫn tăng vì hệ thống đã kiểm tra thêm một sync header, còn sh_inv_cnt tăng để ghi nhận số lần phát hiện sync header lỗi. Nếu block_lock = 0, tức hệ thống chưa đồng bộ block, chỉ cần phát hiện một sync header không hợp lệ thì FSM sẽ chuyển sang trạng thái SLIP.

Lý do là trong giai đoạn tìm lock, hệ thống cần tìm được một vị trí căn chỉnh có chuỗi sync header hợp lệ liên tiếp. Nếu xuất hiện sync header sai, vị trí đang xét được xem là không đúng và cần dịch sang vị trí bit khác. Nếu $\text{block_lock} = 1$, tức hệ thống đã đồng bộ block, một vài sync header lỗi có thể được chấp nhận do nhiễu hoặc lỗi đường truyền. Khi đó, FSM không mất lock ngay lập tức. Nếu điều kiện: $\text{sh_cnt} < 64 \ \& \ \text{block_lock} = 1 \ \& \ \text{sh_inv_cnt} < 16$ được thỏa mãn, máy trạng thái sẽ quay lại TEST_SH để tiếp tục kiểm tra các sync header tiếp theo. Điều này có nghĩa là trong một cửa sổ 64 sync header, hệ thống cho phép số lượng sync header lỗi nhỏ hơn 16 mà vẫn duy trì trạng thái block lock. Tuy nhiên, nếu số lượng sync header không hợp lệ đạt hoặc vượt ngưỡng 16 trong khi vẫn chưa kiểm tra đủ 64 sync header, tức điều kiện: $\text{sh_cnt} < 64 \ \& \ \text{block_lock} = 1 \ \& \ \text{sh_inv_cnt} \geq 16$ được thỏa mãn, FSM sẽ chuyển sang SLIP.

Tại trạng thái SLIP, tín hiệu: $\text{block_lock} \leq 0$ được đưa về mức thấp. Trạng thái này cho biết hệ thống đã mất đồng bộ block hoặc vị trí căn chỉnh hiện tại không chính xác. Khối Block Sync thực hiện thao tác slip, tức dịch vị trí kiểm tra sync header sang bit kế tiếp để thử tìm lại ranh giới block 66 bit đúng. Sau khi thao tác slip hoàn tất, tín hiệu slip_done được kích hoạt. Khi đó, FSM quay lại RESET_CNT để xóa lại các bộ đếm và bắt đầu một chu kỳ kiểm tra mới tại vị trí căn chỉnh mới.

3.1.7. Khối giám sát BER và Sync Header



Hình 3.14: Mô hình khối giám sát BER và Sync Header

Chức năng của các tín hiệu trong khối:

Các ngõ vào:

- clk: Tín hiệu xung clock 156.25 MHz.
- rst_n: Tín hiệu reset tích cực mức thấp.
- block_lock: Tín hiệu báo khối đồng bộ block đã khóa được biên block 66 bit.
- data_in [65:0]: Block 66 bit đầu vào dùng để kiểm tra sync header.
- valid_in: Tín hiệu báo block đầu vào hợp lệ.

Các ngõ ra:

- hi_ber: Tín hiệu báo lỗi BER cao.
- ber_count_tick: Tín hiệu xung báo có một sync header lỗi được phát hiện.

Cấu tạo của khối:

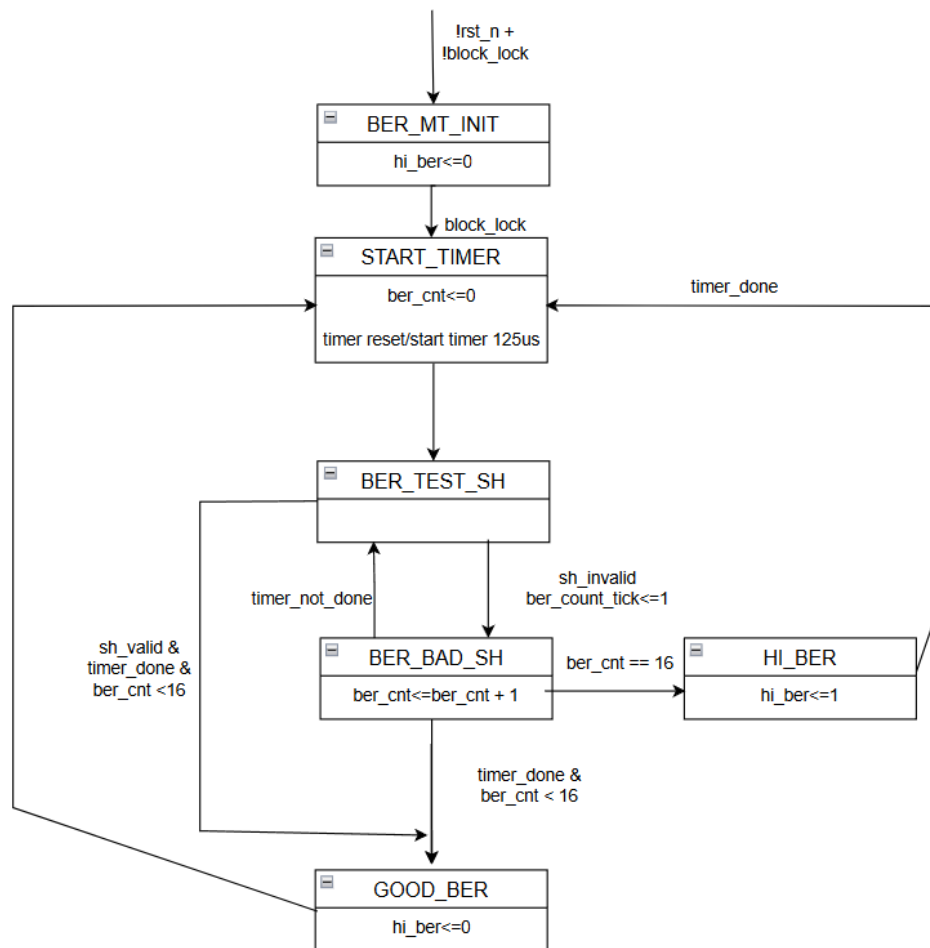
Khôi tách sync header: Lấy 2 bit thấp của block đầu vào data_in [1:0] để kiểm tra sync header.

Khối kiểm tra sync header lỗi: Xác định sync header không hợp lệ khi có giá trị 2'b00 hoặc 2'b11.

Bộ định thời timer_cnt: đếm thời gian giám sát BER. Trong thiết kế, giới hạn đếm được đặt bằng $TIMER_LIMIT = 19531$, tương ứng xấp xỉ của số kiểm tra 125 us.

Bộ đếm lỗi ber_cnt: Đếm số lần xuất hiện sync header không hợp lệ trong một cửa sổ thời gian.

Máy trạng thái FSM: Điều khiển quá trình khởi tạo, bắt đầu bộ định thời, kiểm tra sync header, đếm lỗi và xác định trạng thái BER cao.



Hình 3.15: Sơ đồ trạng thái của khối BER Monitor

Khi có tín hiệu reset `rst_n` hoặc `block_lock = 0`, FSM được đưa về trạng thái `BER_MT_INIT`. Tại trạng thái này, tín hiệu `hi_ber` được xóa về 0, cho biết chưa phát hiện tình trạng tỷ lệ lỗi bit cao. Khi `block_lock = 1` được thiết lập, FSM chuyển sang trạng thái `START_TIMER` để khởi tạo bộ đếm lỗi `ber_cnt` và bắt đầu chu kỳ giám sát trong khoảng thời gian 125 us.

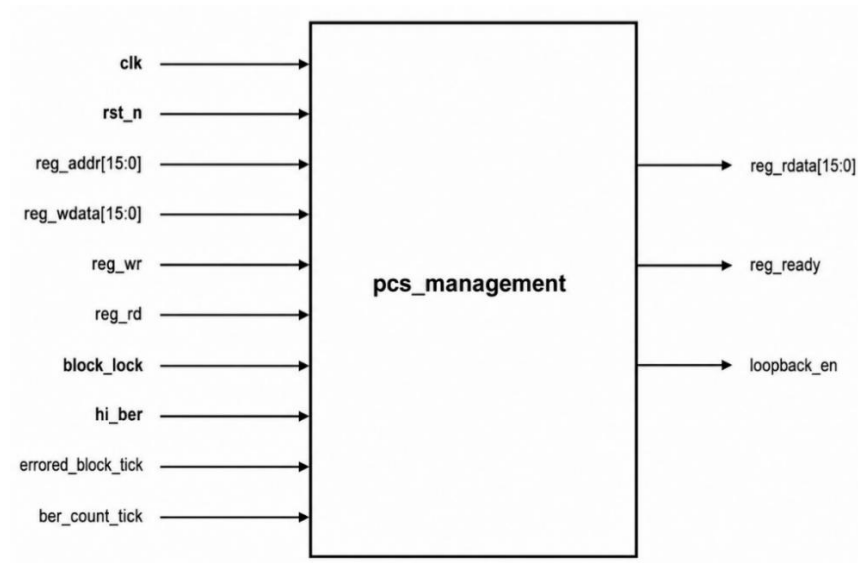
Sau đó, FSM chuyển sang trạng thái `BER_TEST_SH` để kiểm tra sync header của block nhận được. Trong mã 64B/66B, sync header hợp lệ có giá trị 01 đối với data block và 10 đối với control block. Nếu sync header nhận được có giá trị 00 hoặc 11, hệ thống xem đây là sync header không hợp lệ và FSM chuyển sang trạng thái `BER_BAD_SH`.

Tại trạng thái `BER_BAD_SH`, bộ đếm lỗi `ber_cnt` được tăng lên một đơn vị để ghi nhận một sync header lỗi. Nếu số lượng sync header lỗi đạt đến ngưỡng quy định (`ber_cnt == 16`), FSM chuyển sang trạng thái `HI_BER` và đặt tín hiệu `hi_ber = 1`, báo hiệu đường truyền đang có tỷ lệ lỗi bit cao. Trong thiết kế này, điều kiện `ber_cnt == 16` trước khi tăng bộ đếm tương ứng với việc phát hiện đủ 16 sync header lỗi trong chu kỳ giám sát.

Nếu bộ đếm thời gian kết thúc mà số lượng sync header lỗi vẫn nhỏ hơn ngưỡng, FSM chuyển sang trạng thái `GOOD_BER`. Tại trạng thái này, tín hiệu `hi_ber` được giữ ở mức 0, cho biết chất lượng tín hiệu vẫn nằm trong giới hạn cho phép. Sau đó, FSM quay lại `START_TIMER` để bắt đầu một chu kỳ giám sát mới.

3.1.8. Khối giám sát và quản lý trạng thái PCS

Khối `pcs_management` có nhiệm vụ quản lý, cấu hình và giám sát trạng thái hoạt động của PCS đóng vai trò là giao diện thanh ghi để hệ thống bên ngoài có thể đọc trạng thái hoặc ghi cấu hình điều khiển cho PCS.



Hình 3.16: Khối giám sát và quản lý trạng thái

Các ngõ vào:

- clk: Tín hiệu xung clock của hệ thống 156.25 MHz.
- rst_n: Tín hiệu reset tích cực mức thấp.
- reg_addr [15:0]: Địa chỉ thanh ghi cần truy cập.
- reg_wdata [15:0]: Dữ liệu ghi vào thanh ghi.
- reg_wr: Tín hiệu cho phép ghi thanh ghi.
- reg_rd: Tín hiệu cho phép đọc thanh ghi.
- block_lock: Tín hiệu báo khối Block Sync đã đồng bộ được biên block 66 bit.
- hi_ber: Tín hiệu báo trạng thái BER cao từ khối BER Monitor.
- errored_block_tick: Xung báo có một block lỗi được phát hiện.
- ber_count_tick: Xung báo bộ đếm BER tăng, tương ứng với một lần phát hiện lỗi liên quan đến sync header.

Các ngõ ra:

- reg_rdata[15:0]: Dữ liệu đọc ra từ thanh ghi.

- reg_ready: Tín hiệu báo thao tác đọc hoặc ghi thanh ghi đã được tiếp nhận.
- loopback_en: Tín hiệu cho phép chế độ loopback trong PCS.

Cấu tạo của khối:

Khối pcs_management gồm các thành phần chính: khối mã địa chỉ thanh ghi, thanh ghi điều khiển loopback, các thanh ghi trạng thái PCS, bộ đếm BER, bộ đếm block lỗi và mạch điều khiển đọc/ghi thanh ghi.

Khối giải mã địa chỉ thanh ghi có nhiệm vụ xác định thanh ghi cần đọc hoặc ghi dựa trên giá trị reg_addr. Khi có yêu cầu ghi thanh ghi, tức reg_wr = 1, khối sẽ kiểm tra địa chỉ ghi. Nếu địa chỉ tương ứng với thanh ghi PCS_CTRL1, bit 14 của reg_wdata sẽ được lưu vào thanh ghi reg_loopback để điều khiển tín hiệu loopback_en. Khi loopback_en = 1, chế độ loopback được kích hoạt để phục vụ kiểm tra nội bộ PCS. Khi loopback_en = 0, PCS hoạt động ở chế độ bình thường.

Khối đọc thanh ghi hoạt động khi reg_rd = 1. Tùy theo địa chỉ reg_addr, dữ liệu tương ứng sẽ được đưa ra ngõ reg_rdata[15:0]. Thanh ghi PCS_CTRL1 cho phép đọc lại trạng thái loopback. Thanh ghi PCS_STAT2 chứa trạng thái block_lock, cho biết PCS đã đồng bộ được biên block 66 bit hay chưa. Thanh ghi PCS_BASE_R_STAT chứa hai trạng thái quan trọng là hi_ber và block_lock, dùng để đánh giá trạng thái hoạt động của đường thu PCS.

Bộ đếm ber_cnt_reg có độ rộng 6 bit, dùng để tích lũy số lần xuất hiện xung ber_count_tick. Bộ đếm này tăng lên khi ber_count_tick = 1 và dừng ở giá trị lớn nhất 6'h3F, tức 63. Giá trị của bộ đếm được đọc thông qua thanh ghi PCS_BER_CNT tại địa chỉ 16'h0021. Sau khi thanh ghi này được đọc, bộ đếm sẽ được xóa về 0 theo cơ chế read-on-clear.

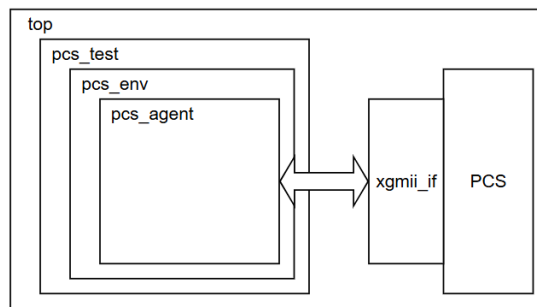
Bộ đếm err_blk_cnt_reg có độ rộng 8 bit, dùng để tích lũy số lần xuất hiện xung errored_block_tick. Bộ đếm này tăng lên khi phát hiện block lỗi và dừng ở giá trị lớn

nhất 8'hFF, tức 255. Giá trị của bộ đếm được đọc thông qua thanh ghi PCS_ERR_BLK_CNT tại địa chỉ 16'h0022. Tương tự bộ đếm BER, bộ đếm block lỗi cũng được xóa về 0 sau khi thanh ghi tương ứng được đọc.

Khi $rst_n = 0$, toàn bộ khối được đưa về trạng thái ban đầu. Thanh ghi loopback được xóa về 0, dữ liệu đọc reg_rdata được đưa về 16'h0000, tín hiệu reg_ready được xóa, đồng thời hai bộ đếm ber_cnt_reg và err_blk_cnt_reg cũng được đưa về 0. Sau khi thoát reset, khối bắt đầu chờ các yêu cầu đọc hoặc ghi thanh ghi. Khi có yêu cầu truy cập, tín hiệu reg_ready được kích hoạt để báo rằng thao tác đã được tiếp nhận.

3.2. UVM Testbench

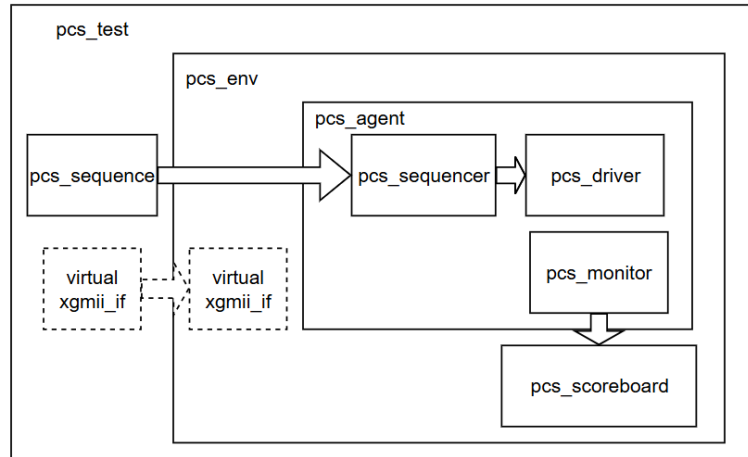
UVM Testbench của thiết kế này được xây dựng dựa trên kiến trúc thông thường của UVM. Top module được biểu diễn như hình 3.17.



Hình 3.17: Cấu trúc của top module

3.2.1. pcs_test, pcs_env, pcs_agent

Cấu trúc của các lớp pcs_test, pcs_env và pcs_agent được biểu diễn như hình 3.18.



Hình 3.18: Cấu trúc của pcs_test, pcs_env và pcs_agent

Lớp pcs_test được mở rộng từ uvm_test và được đăng ký vào cơ chế factory của UVM thông qua macro uvm_component_utils, giống như uvm_test.

Trong pha build_phase, virtual interface của XGMII được lấy từ top module qua cơ chế config_db, với mục đích đưa xuống lớp dưới. Pha này đồng thời khởi tạo lớp pcs_env.

Trong pha run_phase, pcs_sequence sẽ được khởi tạo và đưa trao đổi tới lớp pcs_sequencer bên trong lớp pcs_agent.

Lớp pcs_env được mở rộng từ uvm_env và được đăng ký vào cơ chế factory giống uvm_env.

Trong pha build_phase, lớp này tiếp tục lấy virtual interface từ config_db để đưa xuống lớp dưới, cũng như khởi tạo lớp pcs_agent và pcs_scoreboard.

Trong pha connect_phase, pcs_env kết nối các analysis_port (gồm có tx_port và rx_port) của agent đến scoreboard để phục vụ việc quan sát các tín hiệu trên XGMII.

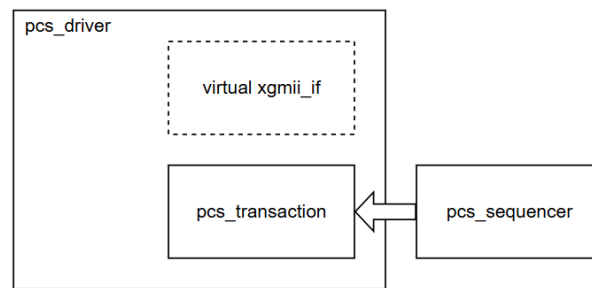
Lớp pcs_agent là một lớp được mở rộng từ uvm_agent và cần được đăng ký vào cơ chế factory giống uvm_agent.

Trong pha `build_phase`, lớp này khởi tạo các lớp `pcs_driver`, `pcs_sequencer`, `pcs_monitor`, đồng thời khởi tạo `uvm_analysis_port` (`tx_port` và `rx_port`), một TLM port truyền đi các trao đổi đã được thực hiện.

Trong pha `connect_phase`, lớp này kết nối sequencer đến driver để chuyển trao đổi cho hoạt động của driver. Ngoài ra `analysis_port` của agent được kết nối đến monitor để phát các hoạt động của trao đổi.

3.2.2. *pcs_driver*

. Lớp `pcs_driver` được biểu diễn như hình 3.19.



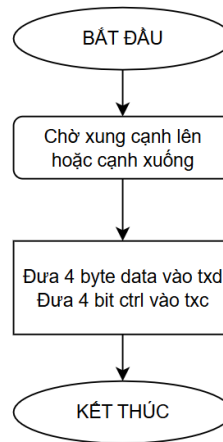
Hình 3.19: Cấu trúc của `pcs_driver`

Lớp `pcs_driver` được mở rộng từ `uvm_driver` và được đăng ký vào cơ chế factory giống `uvm_driver`.

Trong pha `build_phase`, lớp này tiếp tục lấy virtual interface từ `config_db` để phục vụ việc điều khiển tín hiệu.

Trong pha `run_phase`, lớp này khởi tạo lớp `pcs_transaction`. Trao đổi sau đó được truyền giữa sequencer và driver thông qua hàm `get_next_item` của `seq_item_port`, một TLM port dùng cho giao tiếp driver-sequencer.

Driver điều khiển tín hiệu bằng hàm tự định nghĩa. Hàm này mô phỏng hoạt động truyền dữ liệu trên giao diện XGMII, và có lưu đồ như hình 3.20.



Hình 3.20: Lưu đồ hàm điều khiển tín hiệu trên driver

Driver nhận 4 byte tín hiệu dữ liệu (data) và 4 bit tín hiệu điều khiển (ctrl) từ pcs_sequencer. Tín hiệu của 4 byte data cùng 4 bit ctrl được gửi đi sau mỗi cạnh lên hoặc cạnh xuống của xung xgmii_tx_clk.

3.2.3. *pcs_transaction, pcs_sequence, pcs_sequencer*

Các lớp pcs_transaction, pcs_sequence, pcs_sequencer có nhiệm vụ tạo ra một và một chuỗi các trao đổi cho một giao tiếp trên XGMII. Lớp pcs_transaction được mở rộng từ uvm_sequence_item, pcs_sequence được mở rộng từ uvm_sequence và pcs_sequencer được mở rộng từ uvm_sequencer.

Lớp pcs_transaction định nghĩa thông tin truyền với biến *data* cho tín hiệu dữ liệu 32 bit trên xgmii_txd, biến *ctrl* cho tín hiệu điều khiển 4 bit trên xgmii_txc và một mảng động data_q nhằm mục đích tạo dữ liệu truyền ngẫu nhiên trong dải dữ liệu của XGMII.

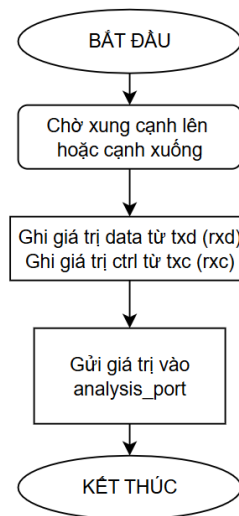
Từ các thông tin trên pcs_transaction, lớp pcs_sequence tạo ra các chuỗi trao đổi (transaction) cho driver. Đầu tiên, lớp này gửi liên tục các khối Idle để PCS được đồng bộ hóa. Sau đó, lớp này gửi đi các tín hiệu theo dải dữ liệu XGMII, gồm có Start, preamble, sfd, dữ liệu ngẫu nhiên được tạo bởi pcs_transaction và cuối cùng là

Terminate. Sau khung Terminate thì pcs_sequence gửi thêm một khung Idle. Các trao đổi này được đưa đến sequencer và được thực hiện bởi driver.

3.2.4. pcs_monitor

Lớp pcs_monitor là một lớp được mở rộng từ uvm_monitor và cần được đăng ký vào cơ chế factory giống uvm_monitor. Trong pha build_phase, lớp này tiếp tục lấy virtual interface từ config_db để phục vụ việc điều khiển tín hiệu, đồng thời khởi tạo uvm_analysis_port.

Trong pha run_phase, pcs_monitor ghi lại trao đổi của một hoạt động được thực hiện bằng hàm lấy mẫu tự định nghĩa. Hàm lấy mẫu này có lưu đồ như sau.



Hình 3.21: Lưu đồ hàm lấy mẫu tín hiệu trên monitor

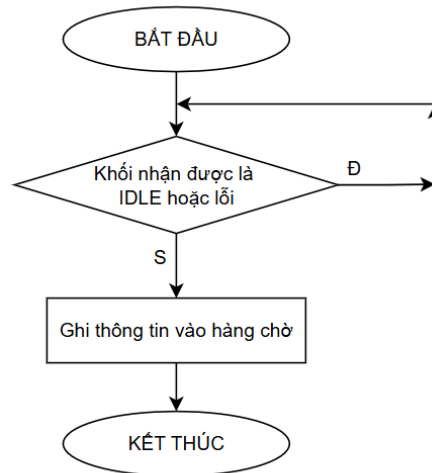
Hàm lấy mẫu này được thực hiện liên tục trong mô phỏng, do đó ghi lại mọi giá trị trên phía truyền và nhận của XGMII. Sau đó, monitor ghi lại các giá trị này vào các analysis_port tương ứng (tx_port hoặc rx_port).

3.2.5. pcs_scoreboard

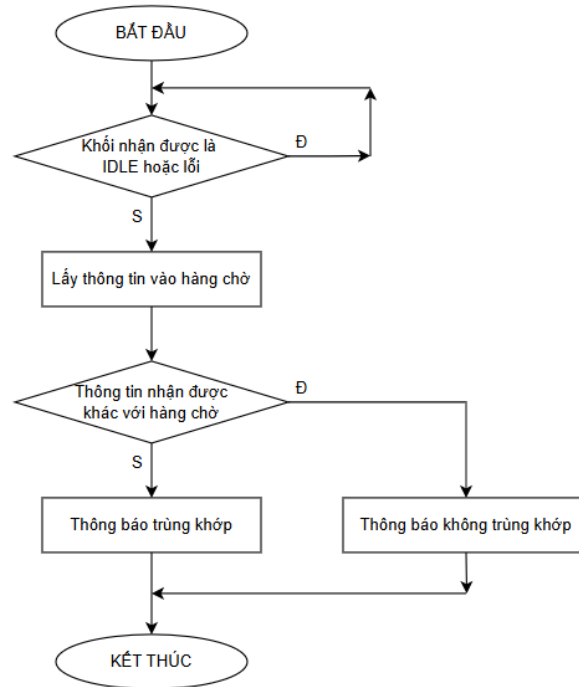
Lớp pcs_scoreboard là một lớp được mở rộng từ uvm_scoreboard và cần được đăng ký vào cơ chế factory giống uvm_scoreboard. Lớp này nhận thông tin của

analysis_port từ pcs_env để thu thập thông tin của các trao đổi, thông qua tx_export và rx_export.

Lớp này tạo một hàng chờ (queue) có tên *expected_q*, dùng để chứa các giá trị mong đợi. Hoạt động kiểm tra được thực hiện bởi hai hàm write_tx và write_rx. Các hàm này có lưu đồ như hình 3.22 và 3.23.



Hình 3.22: Lưu đồ hàm write_tx



Hình 3.23: Lưu đồ hàm write_rx

Hàm write_tx nhận tín hiệu từ tx_port của monitor, lọc các khối Idle hoặc lỗi, rồi đẩy các dữ liệu trao đổi hợp lệ vào hàng chờ *expected_q*.

Hàm write_rx nhận tín hiệu từ rx_port của monitor và lọc các khối Idle và lỗi. Ngoài ra, nó so sánh tín hiệu nhận được với dữ liệu tương ứng từ hàng chờ *expected_q*. Nếu các dữ liệu này trùng khớp thì scoreboard sẽ gửi thông báo vào log. Nếu không, scoreboard sẽ gửi thông báo lỗi. Hoạt động trên các hàm này giúp kiểm tra hoạt động của PCS trên các tín hiệu truyền nhận XGMII.

3.2.6. pcs_stress_test và pcs_stress_sequence

Nhằm đánh giá khả năng hoạt động của thiết kế RTL trong điều kiện tải cao, nhóm bổ sung kịch bản kiểm tra chịu tải thông qua hai lớp pcs_stress_test và pcs_stress_sequence.

Lớp `pcs_stress_test` được mở rộng từ `uvm_test`, kế thừa toàn bộ kiến trúc kết nối của hệ thống gốc (Hình 3.18). Trong pha `run_phase`, thay vì khởi tạo `pcs_sequence`, lớp này khởi tạo `pcs_stress_sequence` và truyền đến `pcs_sequencer`. Toàn bộ các thành phần còn lại gồm `pcs_driver`, `pcs_monitor`, `pcs_scoreboard` và các kết nối TLM Port giữ nguyên hoàn toàn.

Lớp `pcs_stress_sequence` được mở rộng từ `uvm_sequence` và hoạt động theo ba giai đoạn. Giai đoạn đầu tiên truyền khối Idle liên tiếp để khối Block Sync phía thu hoàn tất quá trình đồng bộ và đạt trạng thái `block_lock`. Giai đoạn thứ hai bơm liên tục 20.000 gói tin, mỗi gói gồm tín hiệu Start (0xFB), preamble, SFD (0xD5), payload ngẫu nhiên có độ dài thay đổi và tín hiệu Terminate (0xFD). Điểm đặc trưng của kịch bản này là khoảng cách giữa hai gói tin liên tiếp. Giai đoạn cuối truyền thêm 100 khối Idle để đảm bảo dữ liệu còn trong pipeline được xả hết trước khi mô phỏng kết thúc.

CHƯƠNG 4: KẾT QUẢ MÔ PHỎNG

4.1. Mô phỏng chức năng

4.1.1. Truyền và gửi dữ liệu theo bảng mã hóa 64b/66b

Sau khi ghi lại được một hoạt động trên phía truyền hoặc phía nhận, scoreboard sẽ gửi một dòng thông báo về hoạt động ghi lại được, bao gồm phía truyền hoặc nhận, giá trị trên `xgmii_txc(rxc)` và giá trị trên `xgmii_txd(txd)`. Đồng thời, scoreboard sẽ thông báo kết quả so sánh của các giá trị `data` và `ctrl` ghi lại được sau khi ghi tín hiệu trên phía nhận và trả về các giá trị đó. Nếu giá trị được ghi lại bên phía nhận đúng với mong đợi từ phía truyền, scoreboard sẽ gửi một thông báo trùng khớp. Nếu không, scoreboard sẽ gửi một thông báo lỗi. Các thông báo được biểu diễn như hình 4.1, 4.2, 4.3 và 4.4.

```
uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] TX saved: ctrl=0x00 data='{hec, 'h76, 'h49, 'h5c}
```

Hình 4.1: Thông báo của một hoạt động ghi lại được trên phía truyền

```
uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] RX saved: ctrl=0x00 data='{h4b, 'h66, 'hc6, 'h14}
```

Hình 4.2: Thông báo của một hoạt động ghi lại được trên phía nhận

```
# UVM_INFO pcs_scoreboard.svh(86) @ 3788800000: uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] MATCH  
# TX ctrl=0x00 data='{h4b, 'h66, 'hc6, 'h14}  
# RX ctrl=0x00 data='{h4b, 'h66, 'hc6, 'h14}
```

Hình 4.3: Thông báo của một hoạt động phía nhận trùng khớp với phía truyền

```
# UVM_ERROR pcs_scoreboard.svh(103) @ 3315200000: uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] MISMATCH  
# TX expected: ctrl=0x00 data='{had, 'ha7, 'h6e, 'h24}  
# RX actual : ctrl=0x00 data='{h68, 'h95, 'hfe, 'h8d}
```

Hình 4.4: Thông báo lỗi của một hoạt động phía nhận không trùng khớp với phía truyền

Khi một thông báo lỗi được nhận, UVM sẽ tăng số đếm thông báo lỗi (UVM_ERROR) trong thống kê số lượng thông báo được gửi cuối mô phỏng, thống kê này được biểu diễn trong hình 4.5.

```
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 520
# UVM_WARNING : 0
# UVM_ERROR : 10
# UVM_FATAL : 0
# ** Report counts by id
# [PACKET] 60
# [PCS_SB] 466
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# ** Note: $finish : C:/modeltech64_10.6d/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 5398400 ps Iteration: 55 Instance: /top
# End time: 01:19:38 on Jun 23,2026, Elapsed time: 0:00:03
# Errors: 0, Warnings: 0
```

Hình 4.5: Thống kê các số lượng thông báo trên UVM

Trước khi truyền dữ liệu, sequence tạo ra 300 khối Idle cho driver truyền vào phía truyền của XGMII, để block sync có thể khóa đúng khung của data 66 bit. Khi đó, block sync phải phát hiện đúng 64 header liên tiếp. Nếu sai sync header, block sync trượt 1 bit và tiếp tục dò khung đến khi đúng 64 lần.

Tiếp theo, sequence thực hiện tạo ngẫu nhiên 30 dữ liệu truyền có độ dài ngẫu nhiên từ 5 đến 15 byte. Các dữ liệu này được truyền theo dải dữ liệu XGMII và có các giá trị được ghi lại vào bảng 4.1.

Bảng 4.1: Các dữ liệu ngẫu nhiên được gửi đi trên XGMII

Dữ liệu truyền (biểu diễn theo hệ thập lục phân)	Độ dài (byte)
f6 25 6c 05 1f 85 e0 fc d5 59 2a 5c e3	13
4c a8 90 07 38	5
d4 bc f0 49 db 16	6
3f 50 28 da 57	5
ac 86 2b 58 90 0b 11	7
60 ba d4 42 20 53 92 59 e8 85 56 56 55 8e 03	15

e8 7b 63 c0 8b db e8 b3 b5 33 da f0	12
51 93 42 11 f3 bf 98 c9 3f	9
ec 48 c9 7a c5 f4 6c c7 53 a9 ae d1 88 8d 07	15
36 c6 90 11 e0 41 3f e3 5e fa 67 fa 0b	13
f4 7d 75 11 88	5
3e c6 8d 96 9f 34	6
db a0 e2 dc 6c 22	6
89 dd 13 95 43 d7 0b	7
25 51 36 c7 9f ec 4c 0f	8
cf d1 ac 5d 11 2f a4 e0 aa 63	10
d5 a0 ce 43 3f 4c 25 cf f2 64 38 3a d1 e9 10	15
ab 7b ab 46 2e d9 fc f0 85 1e dd 85 c0 fa	14
7c 39 81 9e 21	5
9a 28 43 fc e2 6c ca d9 eb 28 9a d8 30 ef	14
82 36 ca 93 16 c3 af 68 80 29	10
9d dd a9 79 a7 95 05 55 1e	9
d5 54 11 52 8a be	6
6f d6 cb 74 1f af a4 9e c4 d7 de f8 54 9a 6d	15
e0 94 19 6d 36 37 f6 bc 9f 44 6d c7 17	13
4b 66 c6 14 48 dc 83	7
1d 06 35 64 1c 05 1b 2f c0 96 db f0 c5 59 04	15
5a 2c 6d 8b 2a 7b 7b 91 d2 0f 02	11
bb f4 b0 e5 c5 84 3d f0 cb 4a a1	11
7a 76 64 18 ec 76 49 5c	8

Việc thay đổi kích thước Payload ngẫu nhiên giúp tín hiệu kết thúc gói tin /T/ dịch chuyển qua lại trên toàn bộ 8 Lane từ Lane 0 đến Lane 7, qua đó tự động tạo ra

và kiểm chứng thành công cả 8 loại Terminate Block từ 0x87 đến 0xFF trong bảng hình 2.7.

Nhóm truyền thêm các block chứa mã O (orderset) để kiểm chứng các Block Type có chứa mã O theo hình 2.7.

Bảng 4.2: Các dữ liệu kiểm tra mã hóa ký tự Ordered Set (/O/) gửi đi XGMII

BlockType	XGMII TXC (8bit)	XGMII TXD (64bit)	Cấu trúc
0x4B	8'hF1 (1111_0001)	64'h07070707_0000009C	Lane 0 là /O/ = 0x9C control, lane 1-3 là 3 byte data của Ordered Set; lane 4-7 là IDLE control.
0x2D	8'h1F (0001_1111)	64'h0000009C_07070707	Lane 0-3 là IDLE control; lane 4 là /O/ = 0x9C control, lane 5-7 là 3 byte data của Ordered Set.
0x33	8'h1F (0001_1111)	64'h555555FB_07070707	Lane 0-3 là IDLE control; lane 4 là /S/ = 0xFB control, lane 5-7 là data đầu frame,

			thường là 55 55 55.
0x55	8'h11 (0001_0001)	64'h0000009C_0000009C	Có 2 Ordered Set: /O/ ở lane 0 và /O/ ở lane 4; các lane còn lại trong mỗi nhóm là data của Ordered Set.
0x66	8'h11 (0001_0001)	64'h555555FB_0000009C	Lane 0 là Ordered Set /O/ = 0x9C; lane 4 là Start /S/ = 0xFB; lane 5-7 là data đầu frame 55 55 55

Từ các thông tin trên, scoreboard sẽ gửi các thông báo về các hoạt động được thực hiện trên phía truyền và nhận của XGMII. Kết quả của các hoạt động này được biểu diễn trên hình 4.6.

```

# RX ctrl=0x01 data='{h9c, 'h0, 'h0, 'h0}
# UVM_INFO pcs_scoreboard.svh(83) @ 3926480000: uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] RX saved: ctrl=0x01 data='{hfb, 'h55, 'h55, 'h55}
# UVM_INFO pcs_scoreboard.svh(93) @ 3926480000: uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] MATCH
# TX ctrl=0x01 data='{hfb, 'h55, 'h55, 'h55}
# RX ctrl=0x01 data='{hfb, 'h55, 'h55, 'h55}
# UVM_INFO pcs_scoreboard.svh(83) @ 3929600000: uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] RX saved: ctrl=0x0f data='{hfd, 'h7, 'h7, 'h7}
# UVM_INFO pcs_scoreboard.svh(93) @ 3929600000: uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] MATCH
# TX ctrl=0x0f data='{hfd, 'h7, 'h7, 'h7}
# RX ctrl=0x0f data='{hfd, 'h7, 'h7, 'h7}
# UVM_INFO pcs_scoreboard.svh(83) @ 3936000000: uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] RX saved: ctrl=0x01 data='{h9c, 'h0, 'h0, 'h0}
# UVM_INFO pcs_scoreboard.svh(93) @ 3936000000: uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] MATCH
# TX ctrl=0x01 data='{h9c, 'h0, 'h0, 'h0}
# RX ctrl=0x01 data='{h9c, 'h0, 'h0, 'h0}
# UVM_INFO pcs_scoreboard.svh(83) @ 3939200000: uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] RX saved: ctrl=0x01 data='{h9c, 'h0, 'h0, 'h0}
# UVM_INFO pcs_scoreboard.svh(93) @ 3939200000: uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] MATCH
# TX ctrl=0x01 data='{h9c, 'h0, 'h0, 'h0}
# RX ctrl=0x01 data='{h9c, 'h0, 'h0, 'h0}
# UVM_INFO pcs_scoreboard.svh(83) @ 3948800000: uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] RX saved: ctrl=0x01 data='{h9c, 'h0, 'h0, 'h0}
# UVM_INFO pcs_scoreboard.svh(93) @ 3948800000: uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] MATCH
# TX ctrl=0x01 data='{h9c, 'h0, 'h0, 'h0}
# RX ctrl=0x01 data='{h9c, 'h0, 'h0, 'h0}
# UVM_INFO pcs_scoreboard.svh(83) @ 3952000000: uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] RX saved: ctrl=0x01 data='{hfb, 'h55, 'h55, 'h55}
# UVM_INFO pcs_scoreboard.svh(93) @ 3952000000: uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] MATCH
# TX ctrl=0x01 data='{hfb, 'h55, 'h55, 'h55}
# RX ctrl=0x01 data='{hfb, 'h55, 'h55, 'h55}
# UVM_INFO pcs_scoreboard.svh(83) @ 3955200000: uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] RX saved: ctrl=0x0f data='{hfd, 'h7, 'h7, 'h7}
# UVM_INFO pcs_scoreboard.svh(93) @ 3955200000: uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] MATCH
# TX ctrl=0x0f data='{hfd, 'h7, 'h7, 'h7}
# RX ctrl=0x0f data='{hfd, 'h7, 'h7, 'h7}
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1268) @ 5462400000: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO pcs_scoreboard.svh(114) @ 5462400000: uvm_test_top.pcs_env_h.pcs_scoreboard_h [PCS_SB] All TX transactions were successfully matched with RX!
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 557
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
#
# ** Report counts by id
# [PACKET] 60
# [PCS_SB] 493
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
#
# ** Note: $finish : C:/modeltech64_10.6d/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)

```

Hình 4.6: Kết quả mô phỏng thực hiện gửi 30 lần các dữ liệu ngẫu nhiên và mã hóa Ordered Set

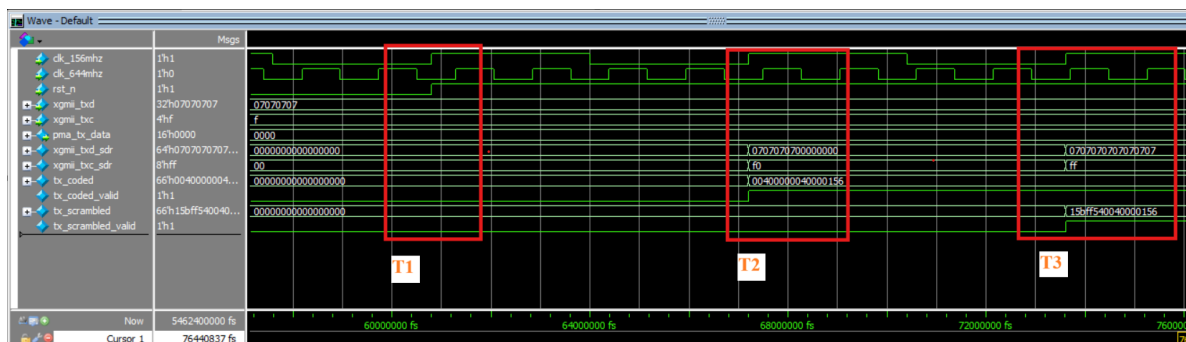
Từ kết quả mô phỏng, do không xuất hiện các dòng thông báo lỗi tín hiệu không trùng khớp, ta có thể thấy module PCS gửi các tín hiệu phía nhận đúng với mong đợi từ phía truyền.

4.2. Mô phỏng dạng sóng

4.2.1. Quá trình truyền dữ liệu

Mô tả dữ liệu XGMII vào TX adapter, encoder, scrambler, gearbox

Các tín hiệu của 10GBASE-R được thiết lập như trong hình 4.7:

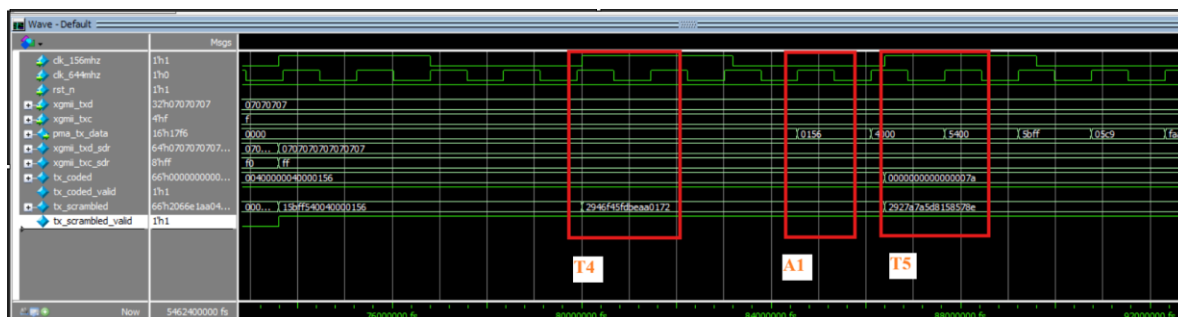


Hình 4.7: Dạng sóng tín hiệu trong quá trình truyền dữ liệu khởi tạo ban đầu ở phía phát

Mạch được kích hoạt để hoạt động ở chế độ loopback. Tại thời điểm T1 tín hiệu reset được nhả, khối được kích hoạt và dữ liệu Idle được đưa liên tục vào đầu vào của mạch.

Tại thời điểm T2, sau khi nhả reset, thanh ghi xgmii_txd_sdr bắt đầu thực hiện chức năng ghép dữ liệu từ bus 32 bit thành bus 64 bit. Lúc này, thanh ghi đã bắt được nhóm dữ liệu 32 bit đầu tiên, tạo thành giá trị 0x0707070700000000. Đồng thời, tín hiệu xgmii_txc_sdr cũng đã ghép được các bit điều khiển đầu tiên với giá trị 0xFF. Giá trị tx_coded (tín hiệu từ endcoder) lúc này là block Local Fault LBLOCK_T trước khi qua khối scrambler có giá trị 66'h00400000040000156. Giá trị này được tạo ra bằng cách ghép payload 64 bit của LBLOCK_T là 0x0100000010000055 với Sync Header dạng Control 2'b10.

Tại thời điểm T3, thanh ghi xgmii_txd_sdr đã ghép thành công dữ liệu Idle thành một block 64 bit hoàn chỉnh với giá trị 0x0707070707070707. Tín hiệu xgmii_txc_sdr cũng đã ghép đầy đủ các bit điều khiển với giá trị 0xFF. Sau khi đi qua khối scrambler, giá trị tx_coded ban đầu 66'h00400000040000156 được mã hóa thành chuỗi đã xáo trộn có giá trị 66'h15BFF540040000156 lúc này tx_scrambler_valid được kéo lên 1 báo scrambler đã xáo trộn bit thành công.



Hình 4.8: Dạng sóng tín hiệu sau quá trình xáo trộn và và chuyển đổi độ rộng dữ liệu phía phát

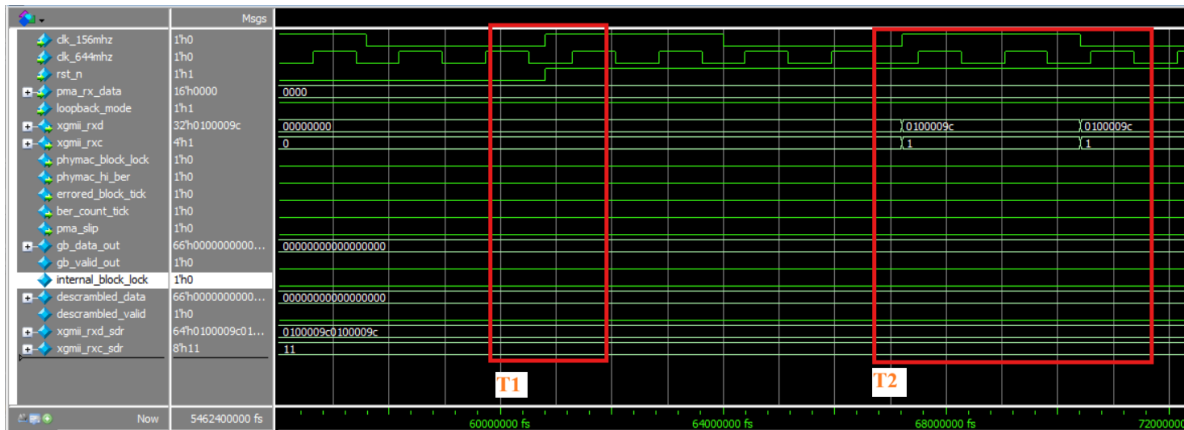
Tại T4 khối scrambler tiếp tục nhận các block dữ liệu 66 bit từ tín hiệu tx_coded và thực hiện xáo trộn chuỗi bit. Kết quả sau khi xáo trộn được quan sát tại tín hiệu tx_scrambled, với giá trị 66'h2946F45FDBEAA0172.

Tại A1 tại xung nhịp tần số 644Mhz khối gearbox chia dữ liệu tx_scrambled = 66'h15BFF540040000156 thành các word 16 bit để truyền xuống PMA. Bốn word đầu tiên quan sát được là 0x0156, 0x4000, 0x5400 và 0x5BFF, tương ứng với 64 bit thấp của block 66 bit. Do block 66 bit không chia hết cho 16 bit, sau bốn lần truyền vẫn còn dư 2 bit. Hai bit này được ghép với 14 bit của block kế tiếp để tạo thành word tiếp theo như 0x05C9. Từ thời điểm này dữ liệu trên pma_tx_data có thể bị lệch so với cách quan sát từng block tx_scrambled riêng lẻ, do gearbox xử lý dữ liệu theo luồng bit liên tục qua nhiều block 66 bit.

Tại T5 tx_coded (tín hiệu mã hóa của encoder) đã mã hóa thành công tín hiệu IDLE thành mã 66'h0000000000000007a trong đó tx_coded = {Payload 64-bit, Sync_Header} ta có Payload 64-bit là: {56'h00...00, 8'h1E} giá trị này là 64'h0000_0000_0000_001E, Sync Header cho một Control Block là 2'b10, 0x1E trong nhị phân là: 0001_1110 nên khi ghép thêm 10 sẽ thành 0001_111010 tương đương 007A.

4.2.2. Quá trình nhận dữ liệu

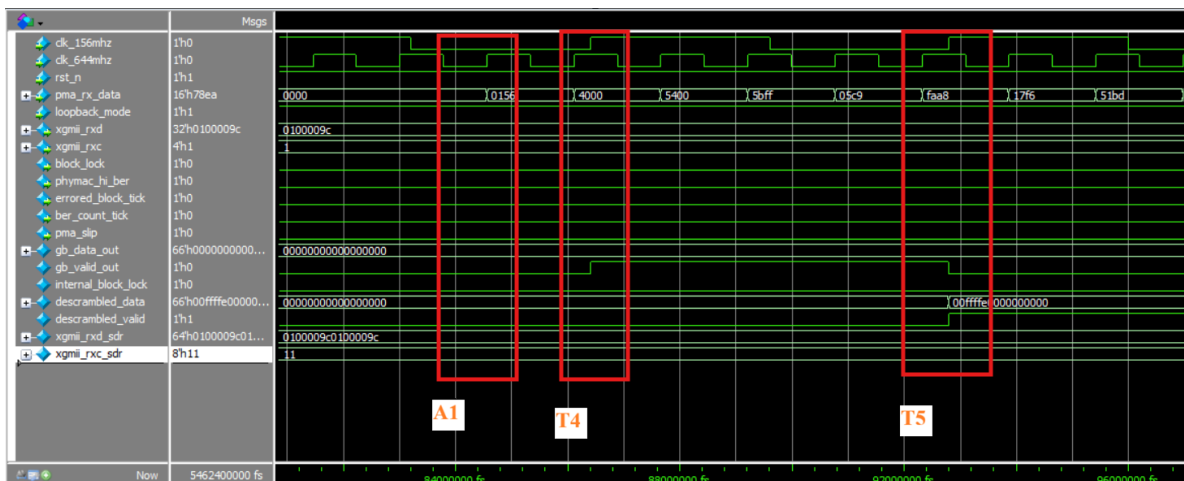
Mô tả dữ liệu từ block sync, descrambler, decoder, RX adapter



Hình 4.9: Dạng sóng tín hiệu trong quá trình truyền dữ liệu khởi tạo ban đầu ở phía thu

Tại T1 nhả tín tín hiệu rst_n mạch bắt đầu xử lý dữ liệu lúc này tín hiệu phy_mac_block_lock = 0 báo bên thu vẫn chưa khóa được khung block. Lúc này mạch đã bắt đầu thực hiện gom các bit trên đường tín hiệu pma_rx_data trên ở miền tần số 644Mhz.

Tại T2, ở chu kỳ tiếp theo mạch phát lên xgmii_rxd block là 01009C xgmii_rxc là 1, báo rằng mạch vẫn chưa gửi dữ liệu đúng tại thời điểm này.



Hình 4.10: Dạng sóng tín hiệu của phía thu khi bắt đầu nhận được dữ liệu từ phía phát

Tại thời điểm A1 tại miền tần số clk_644Mhz dữ liệu bên phía phát đã truyền được đến phía thu.

Tại thời điểm T4 lúc này tín hiệu gb_valid_out (tín hiệu báo đã gom thành công 66bit từ miền clock 644Mhz về miền clock 156Mhz) được kéo lên 1 báo đã gom thành công 66bit.

Tại thời điểm T5 tín hiệu gb_data_out (data 66bit được nhận từ pma_rx_data) xuất dữ liệu đã gom được từ phía phát, tín hiệu descrambler_valid được kéo lên 1 để báo mạch đã đẩy data ra thành công cho phép descrambler ghi data. Nếu lúc này block 66bit vừa xuất ra bị sai synheader mạch tiến hành gửi tín hiệu slip dịch khung gom 66bit qua một đơn vị. Khung gom lúc này sẽ tiếp tục gom 66bit mới với giá trị được dịch lên trên 1 đơn vị (trượt 1 bit).

Trong khoảng thời gian tín hiệu phản hồi trượt bit truyền đi và về giữa 2 miền clock, miền 644MHz vẫn liên tục đẩy các khối 66bit (mang ranh giới cũ) vào bộ đệm Asynchronous FIFO, sau khi đã trượt bit tín hiệu slip_done lại mất thêm thời gian để truyền ngược về. Để tránh vô tình để mất khối 66bit đã được dịch khung trong quá trình gửi tín hiệu slip từ miền 156Mhz về 644Mhz trong khoảng thời gian chờ slip_done. Thiết kế RTL của khối PCS đã ứng dụng cơ chế đánh dấu tọa độ con trỏ ghi (Pointer Snapshotting). Khi xung yêu cầu dịch bit (slip_pulse_clkin) vừa tới miền 644MHz, phần cứng thực hiện dịch thanh ghi đầu vào 1 bit để định hình lại ranh giới. Ngay trong cùng chu kỳ đó, mạch thực hiện sao lưu giá trị của con trỏ ghi hiện tại (wr_ptr) vào một thanh ghi (wr_ptr_at_slip) đánh dấu địa chỉ bộ nhớ của khối 66bit đầu được đóng gói theo ranh giới mới. Trong khoảng thời gian chờ tín hiệu xác nhận slip_done gửi qua cầu CDC về lại miền 156MHz, miền 644MHz tiếp tục đóng gói các khối 66-bit mới và nạp liên tục vào FIFO ở các địa chỉ ngay phía sau mốc wr_ptr_at_slip.

Tại miền 156MHz Khi FSM ở miền 156MHz nhận được tín hiệu slip_done, FIFO hiện đang chứa khối 66bit (cắt theo ranh giới cũ) khối 66bit (cắt theo ranh giới

mới). Khối ép con trỏ đọc nhảy thẳng tới đúng tọa độ đã được đánh dấu tại wr_ptr_at_slip. Toàn bộ các khối block 66bit chưa dịch khung nằm giữa con trỏ đọc cũ và mốc wr_ptr_at_slip bị bỏ qua.



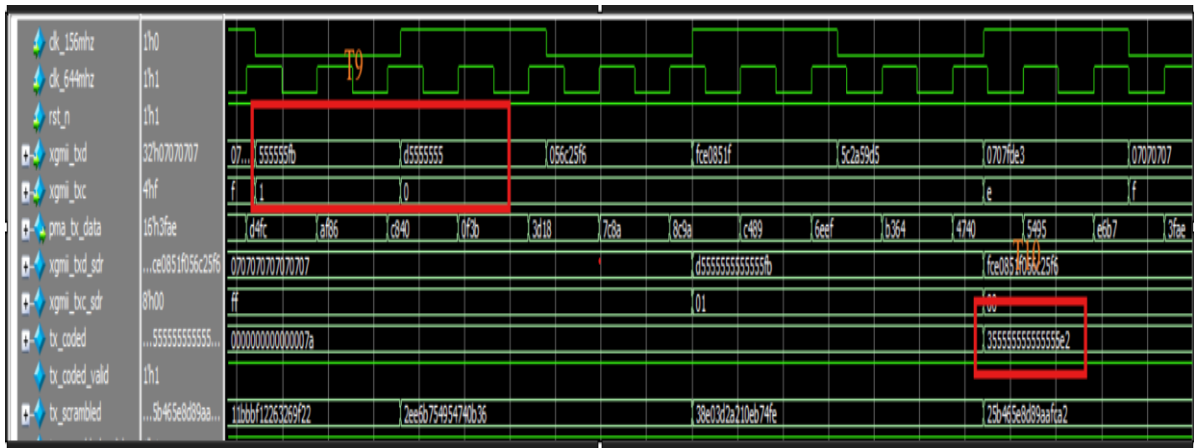
Hình 4.11: Dạng sóng xử lý dữ liệu ở phía thu khi đã khóa được khung

Tại T6 sau khi nhận dữ liệu từ pma_rx_data và mạch đã dò được chính xác biên của block 66 bit. Dữ liệu 66bit là gb_data_out = 34d190eed6901b7f2. Đây là dữ liệu đã được xáo trộn bởi khối scrambler ở phía phát. Dữ liệu này tiếp tục được đưa vào khối descrambler để thực hiện quá trình giải xáo trộn, cho kết quả tại ngõ ra là descrambled_data = 0000000000000007a.

Tại T7 dữ liệu sau giải xáo trộn descrambled_data = 0000000000000007a được đưa qua khối giải mã 64b/66b. Kết quả giải mã cho thấy dữ liệu XGMII ở dạng SDR là xgmii_rxd_sdr = 0707070707070707, đồng thời tín hiệu điều khiển tương ứng là xgmii_rxc_sdr = ff cho thấy toàn bộ 8 byte dữ liệu đều là ký tự điều khiển IDLE 0x07 cho thấy phía thu đã nhận được tín hiệu chờ từ phía phát.

Tại T8 dữ liệu XGMII ở dạng SDR 64 bit được chuyển sang giao tiếp XGMII dạng DDR 32 bit. Trong quá trình này, dữ liệu 64 bit được tách thành hai phần 32 bit và truyền lần lượt trên hai cạnh của xung clock, 32 bit thấp xgmii_rxd_sdr [31:0] được truyền tại cạnh lên của clock, còn 32 bit cao xgmii_rxd_sdr [63:32] được truyền tại

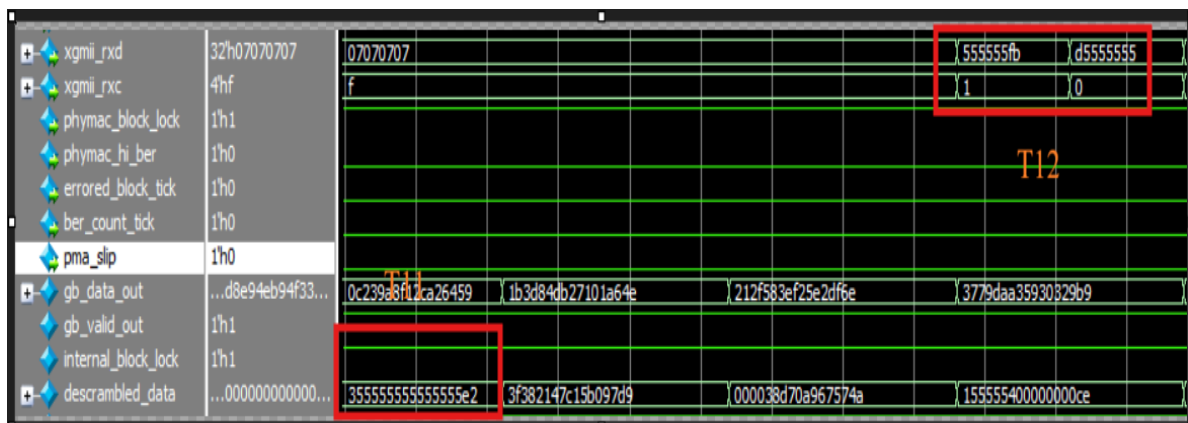
cạnh xuống của clock. Do dữ liệu trong trường hợp này là 0707070707070707, cả hai nửa 32 bit đều có giá trị 07070707, vì vậy tín hiệu quan sát được tại ngõ ra xgmii_rxd là 07070707 trên cả hai pha truyền. Tương tự, tín hiệu điều khiển xgmii_rxc_sdr = ff được tách thành hai phần 4 bit, tạo thành xgmii_rxc = f tại cả cạnh lên và cạnh xuống của clock.



Hình 4.12: Dạng sóng mô phỏng quá trình truyền frame đầu tiên

Tại thời điểm T9 nhóm bắt đầu truyền một frame Ethernet từ giao tiếp XGMII. Dữ liệu DDR 32 bit tại xgmii_txd lần lượt có giá trị 555555fb và d5555555, kết hợp với tín hiệu điều khiển tương ứng xgmii_txc = 1 và xgmii_txc = 0. Trong đó, byte FB là ký tự điều khiển Start của XGMII, các byte 55 là preamble và byte D5 là SFD. Sau khi được gom từ dạng DDR 32 bit sang SDR 64 bit, dữ liệu thu được là xgmii_txd_sdr = d5555555555555fb và xgmii_txc_sdr = 01.

Tại T10 xgmii_txd_sdr = d5555555555555fb và xgmii_txc_sdr = 01 được đưa vào khối mã hóa 64b/66b để tạo block mã hóa tương ứng, thể hiện tại tín hiệu tx_coded = 35555555555555e2.



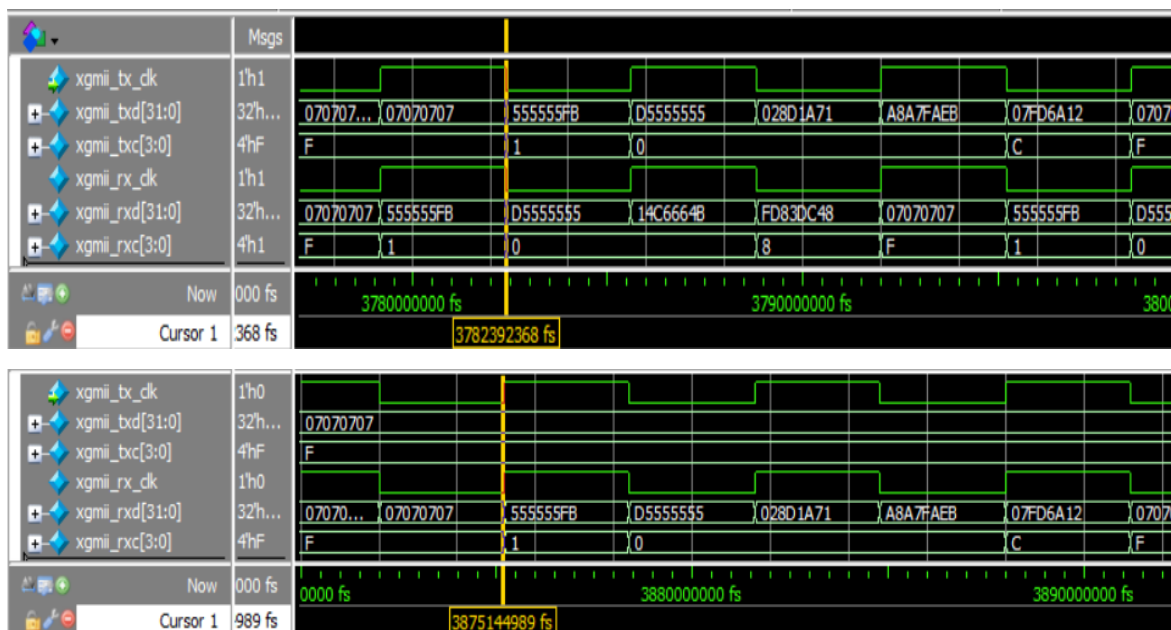
Hình 4.13: Dạng sóng mô phỏng quá trình nhận frame đầu tiên từ phía phát

Tại thời điểm T11 sau khi đi qua khối descrambler, dữ liệu thu được là descrambled_data = 35555555555555e2 hoàn toàn trùng khớp dữ liệu sau giải mã tại T10 là tx_coded = 35555555555555e2.

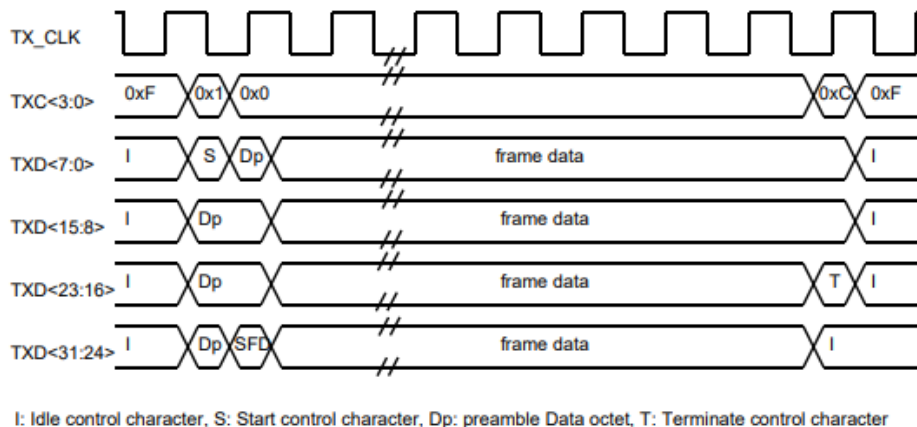
Tại T12 cho thấy kết quả tại ngõ ra cho thấy xgmii_rxd lần lượt có giá trị 555555fb và d5555555, với tín hiệu điều khiển tương ứng là xgmii_rxc = 1 và xgmii_rxc = 0 hoàn toàn trùng khớp với dữ liệu gửi đi ban đầu tại T9.

4.2.3. Kiểm tra dạng khung truyền nhận trên XGMII

Nhóm kiểm tra dạng sóng của hoạt động truyền và nhận trên XGMII và so sánh chúng với dạng sóng được mô tả cho TXC và TXD trong [4]. Kết quả dạng sóng của các hoạt động trên các phía truyền và phía nhận cùng dạng sóng lý thuyết được biểu diễn trong hình 4.14 và 4.15.



Hình 4.14: Kết quả mô phỏng dạng sóng của một hoạt động truyền dữ liệu trên phía truyền và phía nhận



Hình 4.15: Một khung truyền dẫn thông thường trên XGMII [4]

Mô phỏng trên hình thực hiện một hoạt động truyền một chuỗi dữ liệu có giá trị 0x6a12_a8a7_faeb_028d_1a71. Dữ liệu này có độ dài 10 byte. Theo dải dữ liệu XGMII, dữ liệu này được truyền theo bảng 4.3.

Bảng 4.3: Tín hiệu gửi dữ liệu 0x6a12_a8a7_faeb_028d_1a71 theo XGMII

Mã điều khiển	Đường 0	Đường 1	Đường 2	Đường 3
0x1	0xfb (S)	0x55 (Dp)	0x55 (Dp)	0x55 (Dp)
0x0	0x55 (Dp)	0x55 (Dp)	0x55 (Dp)	0xd5 (SFD)
0x0	0x71	0x1a	0x8d	0x02
0x0	0xeb	0xfa	0xa7	0xa8
0xC	0x12	0x6a	0xfd (T)	0x07 (I)

Tại thời điểm 3,782,392ps, xgmii_txd = 0x555555fb và xgmii_txc = 0x1, tương ứng với hoạt động đầu tiên trong dải dữ liệu trên, gồm một mã Start (S) và ba dữ liệu preamble (Dp). Sau nửa chu kỳ xung, tức 3200ps, xgmii_txd = 0xd5555555 và xgmii_txc = 0x0, tương ứng với ba Dp và một dữ liệu sfd. Dữ liệu 10 byte được truyền mỗi 4 byte vào cạnh xung tiếp theo, tương ứng với xgmii_txd = 0x028d1a71 và xgmii_txc = 0x0 cùng với xgmii_txd = 0xa8a7faeb và xgmii_txc = 0x0. Tại cạnh xung cuối cùng, 2 byte còn lại được truyền vào đường 0 và 1 của txd. Đường 3 của xgmii_txd sẽ truyền mã Terminate (T) và đường 4 sẽ truyền mã Idle (I). Do đó, xgmii_txd = 0x07fd6a12 và xgmii_txc = 0xc.

Tương tự với phía nhận, tại thời điểm 3,875,144 ps, xgmii_rxd = 0x555555fb và xgmii_rxc = 0x1. Kế tiếp mỗi nửa chu kỳ là xgmii_rxd = 0x d5555555 và xgmii_rxc = 0x0; xgmii_rxd = 0x028d1a71 và xgmii_rxc = 0x0; xgmii_rxd = 0xa8a7faeb và xgmii_rxc = 0x0; xgmii_rxd = 0x07fd6a12 và xgmii_rxc = 0xc.

Từ các kết quả trên, ta có thể thấy module PCS có kết mô phỏng dạng sóng đúng với mong đợi từ một khung truyền dẫn trên XGMII.

4.3. Mô phỏng kịch bản kiểm tra chịu tải.

Ngoài kịch bản mô phỏng chức năng, nhóm thực hiện thêm kịch bản kiểm tra chịu tải nhằm đánh giá khả năng hoạt động của module PCS trong điều kiện tải cao.

Kịch bản kiểm tra chịu tải được thực hiện với các thông số như sau:

- Số gói tin: 20000
- IPG: 4byte

Hình 4.16 trình bày kết quả log UVM của kịch bản kiểm tra chịu tải:

```
# RX ctrl=0x00 data='{h6f, 'h7c, 'h3, 'hfd}'
# UVM_INFO pcs_scoreboard.svh(83) @ 393289600000: uvm_test_top.pcs_env.h.pcs_scoreboard_h [PCS_SB] RX saved: ctrl=0x01 data='{hfb, 'h55, 'h55, 'h55}'
# UVM_INFO pcs_scoreboard.svh(93) @ 393289600000: uvm_test_top.pcs_env.h.pcs_scoreboard_h [PCS_SB] MATCH
# TX ctrl=0x01 data='{hfb, 'h55, 'h55, 'h55}'
# RX ctrl=0x01 data='{hfb, 'h55, 'h55, 'h55}'
# UVM_INFO pcs_scoreboard.svh(83) @ 393292800000: uvm_test_top.pcs_env.h.pcs_scoreboard_h [PCS_SB] RX saved: ctrl=0x00 data='{h55, 'h55, 'h55, 'hd5}'
# UVM_INFO pcs_scoreboard.svh(93) @ 393292800000: uvm_test_top.pcs_env.h.pcs_scoreboard_h [PCS_SB] MATCH
# TX ctrl=0x00 data='{h55, 'h55, 'h55, 'hd5}'
# RX ctrl=0x00 data='{h55, 'h55, 'h55, 'hd5}'
# UVM_INFO pcs_scoreboard.svh(83) @ 393296000000: uvm_test_top.pcs_env.h.pcs_scoreboard_h [PCS_SB] RX saved: ctrl=0x00 data='{h39, 'h53, 'hc5, 'h87}'
# UVM_INFO pcs_scoreboard.svh(93) @ 393296000000: uvm_test_top.pcs_env.h.pcs_scoreboard_h [PCS_SB] MATCH
# TX ctrl=0x00 data='{h39, 'h53, 'hc5, 'h87}'
# RX ctrl=0x00 data='{h39, 'h53, 'hc5, 'h87}'
# UVM_INFO pcs_scoreboard.svh(83) @ 393299200000: uvm_test_top.pcs_env.h.pcs_scoreboard_h [PCS_SB] RX saved: ctrl=0x0c data='{he4, 'hc5, 'hfd, 'h7}'
# UVM_INFO pcs_scoreboard.svh(93) @ 393299200000: uvm_test_top.pcs_env.h.pcs_scoreboard_h [PCS_SB] MATCH
# TX ctrl=0x0c data='{he4, 'hc5, 'hfd, 'h7}'
# RX ctrl=0x0c data='{he4, 'hc5, 'hfd, 'h7}'
# UVM_INFO pcs_stress_test.svh(39) @ 399926400000: uvm_test_top [STRESS_TEST] =====
# UVM_INFO pcs_stress_test.svh(40) @ 399926400000: uvm_test_top [STRESS_TEST] GEARBOX STRESS TEST COMPLETED! CHECKING SCOREBOARD...
# UVM_INFO pcs_stress_test.svh(41) @ 399926400000: uvm_test_top [STRESS_TEST] =====
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1268) @ 399926400000: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO pcs_scoreboard.svh(114) @ 399926400000: uvm_test_top.pcs_env.h.pcs_scoreboard_h [PCS_SB] All TX transactions were successfully matched with RX!
#
--- UVM Report Summary ---
#
** Report counts by severity
# UVM_INFO :345642
# UVM WARNING : 0
# UVM ERROR : 0
# UVM FATAL : 0
#
** Report counts by id
# [PACKET] 40000
# [PCS_SB] 305629
# [Questa UVM] 2
# [RNTST] 1
# [STRESS_SEQ] 3
# [STRESS_TEST] 6
# [TEST_DONE] 1
```

Hình 4.16: Kết quả log UVM của kịch bản kiểm tra chịu tải

Kết quả cho thấy Scoreboard xác nhận toàn bộ transaction TX khớp với RX. Không có transaction nào tồn đọng trong queue của Scoreboard khi mô phỏng kết thúc, xác nhận không có dữ liệu bị mất hay lệch pha trong suốt quá trình truyền 20000 gói tin liên tiếp.

Kết quả kịch bản kiểm tra chịu tải cho thấy module PCS 10GBASE-R hoạt động ổn định với IPG 4byte (IPG tối thiểu Ethernet thường là 12 octet [10]) 4 byte chỉ là kịch bản kiểm tra (IPG càng nhỏ, băng thông càng đạt đỉnh), khối Gearbox xử lý bất đồng bộ giữa miền xung nhịp 156.25 MHz và 644.5 MHz vẫn duy trì đồng bộ dữ liệu chính xác qua toàn bộ 20000 gói tin mà không xảy ra tràn bộ đệm FIFO.

CHƯƠNG 5: TỔNG KẾT

5.1. Kết quả

Nhóm đã thiết kế các chức năng chính của PCS 10GBASE-R bằng SystemVerilog theo nguyên lý Clause 49 với kiến trúc 2 xung nhịp (156.25 MHz và 644.53 MHz), bao gồm 3 khối chính:

- Khối Phát (Tx): Chuyển đổi XGMII, mã hóa 64B/66B, trộn dữ liệu (Scrambler) và ép xung (Gearbox 66-to-16) để đẩy ra PMA.
- Khối Thu (Rx): Đồng bộ luồng dữ liệu (Block Sync & Gearbox 16-to-66), giám sát lỗi (BER Monitor), giải trộn (Descrambler), giải mã 64B/66B và trả về MAC.
- Khối Quản lý: Hỗ trợ giám sát trạng thái hệ thống (Block Lock, Hi BER) và điều khiển chế độ nội vòng (Internal Loopback).

Nhóm thực hiện thành công việc thiết kế một UVM testbench có khả năng thực hiện truyền thông tin trên giao diện XGMII và đánh giá tín hiệu nhận được trên phía nhận của PCS.

Đề tài này còn nhiều hạn chế trong việc thiết kế và kiểm thử do hạn chế về kiến thức, công cụ và thời gian của nhóm.

Hạn chế của thiết kế PCS: Thiết kế RTL hiện tại vẫn còn mang tính mô hình chức năng, chưa bao phủ đầy đủ toàn bộ đặc tả PCS trong chuẩn Ethernet. Hệ thống mới được kiểm tra bằng mô phỏng UVM nên chưa phản ánh hoàn toàn các yếu tố timing và tài nguyên khi triển khai FPGA hoặc ASIC.

Đề tài có hạn chế trong việc kiểm thử chức năng của thiết kế. Một tiêu chí đánh giá thường được sử dụng trong kiểm thử, được gọi là độ bao phủ (coverage), không được áp dụng trong testbench này. Độ bao phủ có thể giúp đánh giá nhiều tính năng của thiết kế. Ví dụ có thể kể đến như: một mã điều khiển nhất định đã được truyền và

nhận trong mô phỏng hay chưa; tất cả các dòng trong thiết kế đã được mô phỏng hay chưa; v.v.

5.2. Hướng phát triển

Các hạn chế trên mở ra một vài hướng phát triển khác cho đề tài. Một số hướng có thể kể đến:

- Hoàn thiện đầy đủ toàn bộ đặc tả của PCS trong chuẩn Ethernet Clause 49 và Clause 45.
- Trên nền tảng kiến trúc mã hóa 64B/66B hiện tại, mở rộng nghiên cứu sang các kỹ thuật mã hóa có hiệu suất cao hơn như 256B/257B.
- Tích hợp hoàn chỉnh Ethernet: Ghép nối lõi PCS với các lớp lân cận bao gồm khối MAC ở phía trên và khối PMA ở phía dưới, hình thành một hệ thống truyền nhận Ethernet 10 Gigabit hoàn chỉnh từ đầu đến cuối.
- Thêm độ bao phủ vào tiêu chí đánh giá trong kiểm thử.

TÀI LIỆU THAM KHẢO

- [1] Intel, "10 Gigabit Ethernet Technology Overview," Intel Corporation, 2003.
- [2] A. Athavale and C. Christensen, "High-Speed Serial I/O Made Simple," Xilinx, Inc, 2005.
- [3] D. G. Cunningham, "The Status of the 10-gigabit Ethernet Standard," Agilent Technologies, Inc, 2001.
- [4] IEEE Computer Society, "IEEE Standard for Ethernet (IEEE Std 802.3™-2022)," IEEE SA Standards Board, 2022.
- [5] IEEE Computer Society, "IEEE Standard for Universal Verification Methodology Language Reference Manual (IEEE Std 1800.2™-2020)," IEEE SA Standards Board, 2020.
- [6] S. S. Luckoor, "Introduction to 10 Gigabit 64b/66b (Clause 49)," 22 October 2001. [Online]. Available: https://www.iol.unh.edu/sites/default/files/knowledgebase/10gec/10GbE_Cl49.pdf. [Accessed 5 April 2026].
- [7] Accellera, "Universal Verification Methodology (UVM) 1.2 Class Reference," Accellera Systems Initiative, 2014.
- [8] Accellera, "Universal Verification Methodology (UVM) 1.2 User's Guide," Accelera Systems Initiative, 2015.
- [9] IEEE, "Management Data Input/Output (MDIO) Interface," 6 January 2003. [Online]. Available: https://www.ieee802.org/3/ak/public/jan03/WP2p0_cls45.pdf. [Accessed 5 April 2026].
- [10] F. D. a. N. Folkens, " "IPG Considerations," IEEE 802.3ba Task Force," May 2008. [Online]. Available: https://www.ieee802.org/3/ba/public/may08/folkens_01_0508.pdf.

PHỤ LỤC

Phụ lục A MÃ NGUỒN CHƯƠNG TRÌNH

Link github: [10gbase_R](https://github.com/10gbase_R)

Phụ lục B HƯỚNG DẪN SỬ DỤNG SẢN PHẨM

Hệ thống được thiết kế để sử dụng công cụ mô phỏng ModelSim hoặc QuestaSim.

Mô phỏng sử dụng môi trường UVM (Thư mục `uvm_cu`)

Môi trường kiểm thử UVM hoàn chỉnh cho khối PCS (10GBASE-R) được đặt trong thư mục `uvm_cu`. Thư mục này đã có sẵn một Makefile giúp dễ dàng biên dịch và chạy mô phỏng.

Cách chạy:

Mở terminal (ví dụ: PowerShell, Command Prompt, hoặc Git Bash) và di chuyển vào thư mục `uvm_cu`:

```
bash
```

```
cd uvm_cu
```

Sau đó, có thể chạy các lệnh sau:

Biên dịch mã nguồn (RTL + UVM Testbench):

```
bash
```

```
make build
```

Chạy mô phỏng ở chế độ Console (Không mở GUI): Lệnh này sẽ chạy test mặc định (là `pcs_test`) và hiển thị log ngay trên màn hình dòng lệnh.

```
bash
```

```
make run
```

Chạy kịch bản Ép tải tối đa (`pcs_stress_test`) truyền 20,000 gói tin liên tiếp, sử dụng lệnh sau:

```
make run TEST=pcs_stress_test
```

Chạy mô phỏng có hiển thị dạng sóng (Waveform): Lệnh này sẽ tự động biên dịch, mở giao diện đồ họa (GUI) của ModelSim, nạp toàn bộ dạng sóng (add wave -r /*) và chạy mô phỏng.

bash

make wave

Dọn dẹp file rác: Sau khi mô phỏng xong, để xóa các thư mục work, file log, file waveform và tiết kiệm dung lượng, hãy chạy:

bash

make clean